From OT to OLE with Subquadratic Communication*

Jack Doerner† Iftach Haitner‡ Yuval Ishai§ Nikolaos Makriyannis¶

September 22, 2025

Abstract

Oblivious Linear Evaluation (OLE) is an algebraic generalization of oblivious transfer (OT) that forms a critical part of a growing number of applications. An OLE protocol over a modulus q enables the *receiver* party to securely evaluate a line $a \cdot X + b$ chosen by the *sender* party on a secret point $x \in \mathbb{Z}_q$. Motivated by the big efficiency gap between OLE and OT and by fast OT extension techniques, we revisit the question of *reducing* OLE to OT, aiming to improve the communication cost of known reductions.

We start by observing that the Chinese Remainder Theorem (CRT) can be combined with a prior protocol of Gilboa (Crypto '99) to reduce its communication cost from $O(\ell^2)$ to $\tilde{O}(\ell)$ bits, for $\ell = \log q$. Unfortunately, whereas Gilboa's protocol is secure against a semi-honest sender and a malicious receiver, a direct application of the CRT technique is only semi-honest secure (it is insecure against malicious receivers). Thus, we employ number-theoretic techniques to protect our CRT-based protocol against malicious receivers, while still retaining a concrete advantage over Gilboa's protocol (e.g., 10.2× less communication for $\ell = 256$). Furthermore, we obtain a fully malicious OLE-to-OT reduction by applying either information-theoretic techniques with moderate overhead, or RSA-based cryptographic techniques with very low overhead.

We demonstrate the usefulness of our results in the context of OLE applications, including a post-quantum oblivious pseudorandom function (OPRF) and distributed signatures. In particular, assuming pre-existing random OT correlations, we can use our malicious-receiver OLE protocol to realize (a single instance of) the power-residue based OPRF candidate with security against a malicious client and a semi-honest server using only 1.14 KB of communication, a 16× improvement over the best previous protocol in this setting. Using our RSA-based fully malicious OLE protocol, we achieve a 5× communication improvement over previous OT and EC-based distributed ECDSA protocols. Compared to other ECDSA protocols (including ones that use Paillier and class groups), the communication gains are more modest, but come at only a fraction of the computational cost as we avoid all expensive group operations.

*This is the full version of a paper appearing in ACM CCS 2025 [DHIM25].

†University of Virginia. Email: j@ckdoerner.net

‡Stellar Development Foundation and The Blavatnik School of Computer Science, Tel Aviv University. Email: iftachh@gmail.com.

§Technion and AWS. Email: yuvali@cs.technion.ac.il. This paper is not associated with Amazon.

¶Fireblocks. Email: nikos@fireblocks.com

Contents

1 Introduction

Oblivious Transfer (OT) [Rab81; EGL85] is a two-party protocol that enables the *receiver* R to select one of two messages (bits or strings) supplied by the *sender* S. *Oblivious Linear Evaluation (OLE)* [NP06] is a natural algebraic generalization of OT in which the sender supplies the secret coefficients a and b of a line $a \cdot X + b$ (defined over some finite ring $\mathcal{R}$), and the receiver learns the evaluation of this line on a chosen secret point x. OLE can be used to perform multiplication of secrets over $\mathcal{R}$ in a manner analogous to the way in which OT is used to perform secure multiplication of boolean secrets (i.e., "secure AND") [GMW87; Kil88]. This makes OLE a useful building block in a large variety of applications, including threshold cryptography [DKLs18; HLNR23; DKLs24] and secure computation of general arithmetic circuits [IPS09; KOS16]. In this paper, we focus on the rings $\mathbb{Z}_q$ for "large" primes $q \in \mathbb{N}$ that are exponential in the security parameter. Such rings are common in applications, typically with $128 \leq \log q \leq 512$.

The primary motivation behind this work is the observation that, despite substantial optimization efforts, existing OLE protocols lag far behind their OT counterparts. Highly efficient OLE protocols do exist in the *amortized* setting from lattice assumptions [JVC18; HIMV19; BEPST20; CJV21; LXYY25; CKKLMS24] and error-correcting codes [NP06; IPS09; GNN17]. Still, these only achieve their efficiency guarantees when a *large batch* of OLE relations is computed at once—typically thousands or more. Additional techniques are known for the direct implementation of single OLE instances using homomorphic encryption schemes, such as Paillier's encryption [NP06] or lattice-based encryption [CHIVV22], but these also incur significant overhead. For applications that require only a small number of OLE instances, no highly efficient solution is known.

OLE in the OT-hybrid model. What is typically the most efficient approach to realizing a small number of OLEs is via *reduction* to OT. Specifically, this is because of the existence of highly efficient OT-extension protocols [Bea96; IKNP03; ALSZ15; KOS15; BCGIKS19; YWLZW20; BCGIKS20; RRT23; Roy22; BCMPR24; BBCCDS24; CDDKS24; LXYY25; KPRR25]. Reductions from OLE to OT are commonly specified in the *OT-hybrid model*: the parties have access to an ideal oracle that implements multiple instances of OT.[1] When measuring communication, we consider by default a variant of the OT-hybrid model in which parties have access to correlated randomness consisting of *random* instances of OT, with random messages of the desired length and a random selection bit. These *ROT* correlations can be generated in an offline phase, before the inputs are known, and by default are not counted towards communication costs. This metric is motivated by "silent" OT extension techniques [BCGIKS19], including the recently introduced *public-key* silent OT, which requires little or no communication beyond a shared public key infrastructure (PKI) [BCMPR24; CDDKS24]. Ordinary OT with ℓ-bit chosen messages can be reduced generically to ROT using $2\ell + 1$ bits of communication [Bea91], but it is often possible to achieve better efficiency by designing protocols to use ROT correlations directly.

The feasibility of information-theoretic OLE in the OT-hybrid model is implied by general feasibility results for OT-based secure computation [GMW87; Kil88; IPS08; IKOPS11]. Combined with known results on the asymptotic circuit complexity of modular multiplication [RT90; HH22], OLE over an ℓ-bit modulus can be implemented with $O(\ell \log \ell)$ bits of communication in the OT-hybrid model, with perfect security against semi-honest parties. A similar result with $2^{-\kappa_{\mathsf{s}}}$-statistical security against malicious parties and $O(\ell \log \ell) + \mathrm{poly}(\kappa_{\mathsf{s}}, \log \ell)$ communication follows from [IPS08, Theorem 2]. However, the OLE protocols that follow from these asymptotic results are concretely inefficient for realistic values of ℓ and require a super-constant number of rounds.

The first concretely efficient OLE protocol was proposed by Gilboa [Gil99], who gave a simple and round-optimal protocol for OLE over an ℓ-bit modulus that requires $\approx 2\ell^2$ bits of communication in the OT-hybrid model (with perfect security), or $\approx \ell^2$ bits in the ROT-hybrid model. This protocol is natively secure against a malicious receiver, who might behave in arbitrary ways, but it is only secure against a *semi-*

[1] Using standard composition theorems for secure computation [Can00; Can01], an OLE in the OT-hybrid model can be combined with efficient OT extension techniques to yield an OLE in the plain model. This can be done with little overhead, and thus does not constitute a concrete or asymptotic bottleneck.

honest sender, who is guaranteed to follow the protocol's instructions. A subsequent line of work attempted to achieve security when either participant is malicious, while minimizing overhead. Keller et al. [KOS16] (implicitly) achieved this with a communication overhead factor of roughly 6 relative to Gilboa's baseline. Haitner et al. [HMRT22] reduced this overhead factor to less than 2, either by settling for a relaxed OLE functionality (which suffices for some applications) or by relying on computational security and the DDH assumption. Purely OT-based OLE protocols with overhead factors between 2 and 4 were given by Doerner et al. [DKLs18; DKLs19; DKLs24].

Breaking the quadratic communication barrier? All of the above works build on Gilboa's protocol as a baseline and inherit its quadratic communication in the modulus length ℓ. This raises the question: is it possible to build *subquadratic* and *practical* OLE protocols in the OT-hybrid model, even when settling for security against semi-honest parties? If so, can such improvements be extended to achieve security against malicious parties?

1.1 Our Contribution

We answer the above questions affirmatively, improving the communication costs of OLE in the OT-hybrid model. We present three types of new protocols: (1) a semi-honest secure protocol; (2) a protocol secure against a malicious receiver and a semi-honest sender; (3) fully secure protocols, i.e., secure against malicious behavior by either participant. All protocols have subquadratic (in fact, nearly linear) communication cost in the modulus length ℓ and offer a concrete improvement even for small values of ℓ that arise in applications; we provide cost estimates for practically-relevant parameter combinations in Section 1.2. In the remainder of this section, we give a brief and intuitive overview of the above protocols and some of their applications; a more detailed technical overview appears in Section 2.

Semi-honest OLE. We begin with the observation that the Chinese Remainder Theorem (CRT) can be used to improve the communication complexity of Gilboa's protocol from $O(\ell^2)$ to $\tilde{O}(\ell)$ bits. Concretely, the typical improvement ranges from 7× (for $\ell = 128$) to 26× (for $\ell = 512$) when 40 bits of statistical security are required (i.e., $2^{-\kappa_s}$ distinguishing advantage for statistical parameter $\kappa_s = 40$). At a high level, we apply a standard randomization technique to reduce the problem of OLE modulo q to the problem of OLE modulo a smooth integer $n \approx q^2 \cdot 2^{\kappa_s}$, where n is the product of a set $\{n_i\}_{i\in[t]}$ of small, distinct prime moduli, and κ_s is a statistical security parameter. The parties apply Gilboa's protocol over each modulus n_i individually, and the receiver uses the CRT to reconstruct the output modulo q from the intermediate outputs modulo each n_i. Using this approach, the communication grows quadratically in the length of each n_i. But if we choose n_i to be the t smallest primes, then it suffices to use $t = O(\ell/\log \ell)$ primes n_i where each $n_i \in O(\ell)$, and the overall communication is $O(\ell \log \ell)$. (Here and in the following, we assume that $\ell \geq \kappa_s$.) Like Gilboa's protocol, and unlike the generic approaches discussed above, this CRT-based protocol is minimally interactive: it requires only one message in each direction (sequentially), given random OT correlations.

Malicious-receiver OLE. While Gilboa's protocol natively achieves security against a malicious receiver, the above CRT-based protocol is only secure against a semi-honest receiver; since $n \gg q$, a malicious receiver can choose its input x outside the expected range and thereby obtain information about the sender's input that cannot be simulated via an ideal OLE call. Our second protocol matches Gilboa's security properties while retaining almost the same efficiency advantage as our first protocol; the main tradeoff is that we must use a slightly larger $n \approx q^2 \cdot 2^{5\kappa_s}$. We eliminate the "out of range" attack by instructing the sender to perform a simple statistical consistency check of the information learned by the receiver. We present a method to perform this check with minimal communication overhead, and without increasing the number of rounds (keeping the number of rounds intact, requires the use of a random oracle). Specifically, the receiver reveals its input/output pair modulo a fixed prime, and the sender checks for consistency (to preserve the receiver's security, it uses a slightly larger input in the semi-honest protocol). While the resulting protocol is simple, its analysis is subtle and relies on a variant of Thue's lemma.

Fully malicious OLE. Our final protocol extends the above malicious-receiver protocol also to provide security against malicious senders. Our extension requires the receiver to perform a similar check to the one the sender did in the malicious-receiver protocol, now applied to the input a, b of the sender. Since a malicious sender has an additional attack vector, i.e., using inconsistent inputs to baseline Gilboa's protocol, the consistency check uses a random prime modulus r, and the sender commits to its input a *in advance*, and when revealing $a \bmod r$ it proves it is consistent with the committed value. We devise two methods for doing this *commit and modular reveal check*:

1. The first method is based information-theoretically upon OT via cut-and-choose techniques. As such, it can be founded on many assumptions and is plausibly post-quantum secure. Unfortunately, its bandwidth complexity grows with the product of ℓ and the statistical parameter, and we do not know how to leverage the CRT to improve this asymptotic figure as we are able to elsewhere. While this variant does meaningfully inflate the concrete bandwidth cost of the overall protocol, it remains more bandwidth-efficient than prior works, both asymptotically and concretely.

2. The second method is based upon an efficient RSA-based integer commitment scheme, along with a simple sigma protocol. This method has a communication cost that is independent of ℓ and is, concretely, far lower. Still, it cannot offer information-theoretic security in the OT-hybrid model and cannot plausibly be post-quantum secure.

Alternatively, we explore the case that the sender does *not* commit to a in advance. The resulting protocol is highly efficient, gaining 0.77 KB compared to the provable RSA-based protocol and avoiding the costly exponentiations in the RSA group. Furthermore, since it is purely based on OT, it allows for post-quantum security. The security of this variant reduces to a natural but apparently new number-theoretic conjecture (see Conjecture 5.37) whose study may be of independent interest.

Under all three of the approaches we explore, our fully malicious protocol requires the size of n to satisfy $n'/\log_2 n \approx q^2 \cdot 2^{12\kappa_s}$, where n' is the product of all but the largest κ_s factors of n. For comparison, the other cases admit much smaller choices of n, and that term dominates the communication complexity.

VOLE. We extend our fully malicious scheme to length-m VOLE (i.e., m OLEs with the same receiver input). First, we present (in Appendix B) a new information-theoretic reduction of VOLE to OLE, realizing length-m VOLE modulo a big prime q from $m+1$ instances of OLE modulo q and a short commitment (which can be realized by an additional OLE). This improves on a similar reduction from [DKM12], which requires more than $2m$ OLE instances. For short VOLEs, which are useful for some applications (see below), we present a direct generalization of our OLE protocol that beats this general reduction. In concrete terms, the total communication is 1.5× smaller at $m = 2$ compared to the general reduction, tapering to about 1.1× at $m = 10$; for larger m, the generic approach becomes more efficient.

Applications. We demonstrate the usefulness of our results in the context of several OLE applications. The first is a construction of a plausibly post-quantum *oblivious pseudorandom function* (OPRF) based on the power-residue PRF candidate [Dam88]. An OPRF protocol enables a client to securely evaluate a PRF on a secret key held by a server. Yang et al. [YBHKR25] show that when settling for security against a malicious client and a semi-honest server, which suffices for some use cases, the power-residue OPRF requires just a single OLE invocation, with a concrete modulus size of 384 bits. In this context, one can utilize our malicious-receiver OLE protocol, which, in the ROT-hybrid model, requires only 1.14 KB of communication over two rounds —a 16× improvement over Gilboa's protocol. This application further motivates the design of post-quantum *public-key* silent OT protocols [BCMPR24; CDDKS24] that minimize the ROT setup cost.

Second, we observe that recent concretely efficient threshold signing schemes for the ECDSA [DKLs24] and BBS+ signature schemes [DKLsT23] are formulated in the hybrid model of a small number of OLE or short *Vector* OLE invocations,[2] and these invocations dominate the overall cost of threshold signing. We

[2]Concretely, two invocations per pair of parties, and [DKLs24] requires a vector length of two. In contrast, the other cited works do not require vectorization.

propose a simple vector extension of our OLE protocols to suit these applications. In the case that our purely-OT-based fully malicious protocol is used, we reduce the bandwidth consumption of the protocols in question by at least 1.8× and as much as 3× in the ROT-hybrid model,[3] while using only the general assumption of ROT correlations. (In the ROT-hybrid model, our conjecture-based variant improves communication to 14× in this model.) In the case that our integer-commitment-based fully malicious protocol is used, we reduce the bandwidth consumption of prior works by as much as 5.5× at the cost of introducing the strong RSA assumption.[4] Many prior works have constructed threshold ECDSA and BBS+ signing schemes under number-theoretic assumptions like DCR or class groups; however, these have universally employed homomorphic encryption schemes [Lin17; GG18; CGGMP20; ABCJM24; CCLST19; CCLST20] that are far more computationally expensive than our integer commitments (see Remark 1.2).

1.2 Summary of Concrete Costs

In Table 1.1, we present the concrete communication costs for our protocols in the ROT-hybrid model, which is our key metric of interest. We provide further discussion of *other* costs in Appendix A. For comparison, we also provide communication costs under equivalent parameterizations for the protocols of Gilboa [Gil99], Haitner et al. [HMRT22], and Doerner et al. [DKLs24]. All protocols are marked with the flavor of security they achieve: either semi-honest (SH), malicious-receiver (MR), or fully-malicious (FM). They are also marked with any computational assumptions or conjectures they require. Let OT+ Cnj stand for our most efficient fully malicious OLE protocol, which does not use the commit&prove functionality (but whose security only holds assuming Conjecture 5.37 is true, for the right choice of parameters).

Protocol	Gilboa	**Ours**	**Ours**	**Ours**	**Ours**	**Ours**	HMRT	DKLs
Security	MR	SH	MR	FM	FM	FM	FM	FM
Assumptions	OT	OT	OT	OT + Cnj	OT + RSA	OT	OT + DDH	OT
$\|q\| = 32$	0.13	0.08	0.28	1.30	2.07	5.63	0.82	1.81
$\|q\| = 64$	0.51	0.14	0.34	1.33	2.10	5.66	1.96	2.92
$\|q\| = 128$	2.05	0.29	0.50	1.42	2.19	6.12	4.94	7.86
$\|q\| = 256$	8.19	0.58	0.80	1.77	2.54	7.78	13.98	24.77
$\|q\| = 384$	18.43	0.90	1.14	2.17	2.94	9.51	27.12	50.29
$\|q\| = 512$	32.77	1.24	1.48	2.55	3.32	11.23	44.35	84.32

Table 1.1: Communication cost in KB, with statistical security parameter $\kappa_{\mathsf{s}} = 40$ and computational security parameter $\kappa_{\mathsf{c}} = 128$. We denote by $|q|$ the bit-length of the OLE modulus q. Our protocols achieve a concrete distinguishing advantage no greater than $2^{-\kappa_{\mathsf{s}}}$, with no hidden constants. Communication is measured in the ROT-hybrid model; i.e., we assume pre-existing random OT correlations at zero cost.

Computational complexity. The computational cost of our protocols is dominated by: (i) modular-arithmetic operations (this is the lion's share of the computation); (ii) sampling a random prime of size roughly κ_{s} (FM variants only); and (iii) RSA group operations (for the OT+RSA protocol). Regarding (i) and (ii), to illustrate: our FM protocols require two CRT encodings (one per party), one CRT decoding (for the receiver), one sampling of a random prime of size κ_{s} (receiver only), and three modular reductions (two by small primes of size κ_{s} and one by the target prime of size $|q|$). The primes used in CRT encoding and decoding are of size $O(\log \log q)$—in practice, at most 10 bits long—so all operations in (i) and (ii) are extremely lightweight. Specifically, CRT encoding involves t modular reductions $a \bmod p_i$ with small p_i, while CRT decoding computes $\sum_{i=1}^{t} a_i c_i \bmod n$ with constants c_i, and small a_i (encoding and decoding each cost $O(t^2)$ when the p_i's are small enough). By comparison, a single exponentiation in an EC group of size q

[3] These performance improvements are calculated relative to the OLE/VOLE protocols proposed in [DKLs24], and assume a parameterization that yields 128 bits of computational security and 40 bits of statistical security.

[4] This implies that the security of the overall protocol can ultimately be founded on a wide range of specific assumptions.

costs $\omega(|q|^2)$. Concretely, for $|q| = 256$, which is associated with $t = 177$ (see Table A.1 in Appendix A), the entire modular-arithmetic cost of our protocol is much lower than a single exponentiation in a q-sized group. The cost of sampling a random small prime is also modest: our benchmarks[5] show about 100 μs for an 80-bit prime and 700 μs for a 256-bit prime. This is comparable to the cost of a few operations in Secp256k1 (the Bitcoin curve). Finally, the RSA-group operations in our OT+RSA protocol (three multi-exponentiations in total) dominate its computational cost by at least an order of magnitude. This is because full exponentiation in an RSA group is significantly more expensive than an EC group at the 128-bit security level.

Remark 1.2. Perhaps a more appropriate comparison for our OT+RSA protocol is with protocols based on number-theoretic assumptions such as DCR. Several DCR-based OLE protocols achieve communication of only a few KB, but their reliance on Paillier encryption makes them computationally expensive. Security-wise, a protocol from [HL09] realizes the OLE functionality, but it involves cut-and-choose techniques which makes it comparatively heavy in both computation and communication compared to more recent protocols. On the flip side, the recent protocols from [Lin17; GG18; CGGMP20; ABCJM24] have not been shown to realize the OLE functionality, though several works could potentially be adapted to do so (with some modifications). Complexity-wise, ignoring the non-RSA costs of our protocol which are negligible in this comparison, our protocol is expected to be about an order of magnitude faster than these, since the commit-and-prove RSA-based protocol (essentially a subprotocol in those works) accounts for less than a tenth of the cost (their dominant overhead being Paillier encryption and decryption). For $q = 256$, the most optimized variant (from [ABCJM24], which relies on comparatively strong assumptions) requires roughly 3KB of communication and 50ms of computation. Thus, our work achieves comparable—or better—communication at only a fraction of the computational cost. A similar comparison holds for class-group-based protocols [CCLST19; CCLST20].

1.3 Realizing the ROT-hybrid Model

Since we cast our results in the ROT-hybrid model, a concrete instantiation of our work requires an additional protocol to sample a sufficient number of ROT correlations efficiently. Since OT correlations are a fundamental building block of multiparty computation, such protocols have received intense interest, and several different approaches exist. Classic OT *extension* protocols consume a small number of "seed" OT instances to generate a polynomially large number of OT correlations using only symmetric cryptography. Traditional OT extension techniques [Bea96; IKNP03; ALSZ15; KOS15] have communication costs greater than the product of the security parameter and the number of OT instances. Roy [Roy22] improved this communication cost by up to a logarithmic factor in the security parameter, while still relying only on symmetric cryptography. On the other hand, there are practical "silent" OT protocols [BCGIKS19; BCGIKS20; RRT23; BBCCDS24; LXYY25; KPRR25], based on *pseudorandom correlation generators* (PCGs) for OT, in which the total communication cost is sublinear in the number of ROT instances. Such protocols are typically based on LPN-style assumptions and have moderately higher setup costs than non-silent OT extension techniques. Of special note are *public-key* silent OT protocols, which allow parties to generate ROT correlations using only a shared public-key infrastructure (PKI) [BCMPR24], possibly with a small amount of additional communication [CDDKS24]. Our work further motivates the design of faster (and post-quantum) public-key silent OT protocols.

Organization

A high-level overview of our techniques is given in Section 2. In Section 3, we review the necessary preliminaries. In Section 4, we describe our semi-honest and malicious-receiver OLE protocols. In Section 5, we describe our fully-secure OLE protocol. In Section 7, we describe and prove the OT-based and RSA-based sub-protocols that our fully-secure OLE requires. In Section 6, we prove several theorems and lemmas introduced in the previous sections. In Section 8 we describe several applications of our protocol and give concrete performance results in context. In Appendix A we report in-depth parameterizations and concrete

[5]Conducted on a 13th Gen Intel(R) Core(TM) i7-1365U CPU using OpenSSL 3.0.13.

performance results for our protocol when it stands alone. A generic reduction from VOLE to OLE is given in Appendix B. Finally, missing proofs are given in Appendix C.

2 Technical Overview

In this section, we provide a high-level overview of our techniques. For a modulus $n \in \mathbb{N}$, recall the definitions of the following functionalities:

- **Oblivious Transfer ($\mathsf{OT_n}$)**: Takes two inputs $(a_0, a_1) \in \mathbb{Z}_n^2$ from the sender S and a selection bit $i \in \{0, 1\}$ from the receiver R. It returns a_i to R.
- **Oblivious Linear Evaluation ($\mathsf{OLE_n}$)**: Takes two inputs $(a, b) \in \mathbb{Z}_n^2$ from the sender S and an input $x \in \mathbb{Z}_n$ from the receiver R. It returns $ax + b \bmod n$ to R.
- **Vector Oblivious Linear Evaluation ($\mathsf{VOLE_{m,n}}$)**: Takes two inputs $(\mathbf{a}, \mathbf{b}) \in (\mathbb{Z}_n^m)^2$ from the sender S and an input $x \in \mathbb{Z}_n$ from the receiver R. It returns $\mathbf{a}x + \mathbf{b} \bmod n$ to R.

2.1 Gilboa's Protocol

Similar to many other works in this area [KOS16; DKLs18; DKLs24; HMRT22], our starting point is a protocol due to Gilboa [Gil99], a simple malicious-receiver $\mathsf{OLE_n}$ protocol in the $\mathsf{OT_n}$-hybrid model defined as follows. (In the following, all ring operations in $\mathbb{Z}_n$ are reduced modulo n).

Protocol 2.1 (Gilboa's OLE protocol)**.**

Parties: Sender S and receiver R.

Parameters: $n \in \mathbb{N}$. Let $\ell \leftarrow \lceil \log(n) \rceil$.

Oracle: $\mathsf{OT_n}$.

S's input: $(a, b) \in \mathbb{Z}_n^2$.

R's input: $x \in \mathbb{Z}_n$. Let $x_0, \ldots, x_{\ell-1}$ be the bit decomposition of x (i.e., $x = \sum_{j=0}^{\ell-1} x_j \cdot 2^j$).

Operations:

1. S: Sample $b_0, \ldots, b_{\ell-2} \xleftarrow{R} \mathbb{Z}_n$ and set $b_{\ell-1} \leftarrow b - \sum_{j=0}^{\ell-2} b_j$.
2. For $j = 0$ to $\ell - 1$ (in parallel): The parties jointly call $\mathsf{OT_n}$, with S using input $(b_j, b_j + a \cdot 2^j)$ and R using input x_j. Let z_j denote R's output.
3. R: Set $y = \sum_{j=0}^{\ell-1} z_j$.

Output: R outputs y. (S outputs nothing).

To see that the above protocol achieves security against malicious receivers, notice that there is no opportunity for R to cheat, as the only possible deviation involves selecting a different input, i.e., choosing the selection bits $x_0, \ldots, x_{\ell-1}$ wrongly, during the OT phase (this does not violate the protocol's security, as it is equivalent to running the protocol with a different input x'). The protocol, however, is not secure against malicious senders.[6]

[6] Consider a malicious sender who uses input $(0, 1)$ in the first OT call (i.e., $j = 0$) and input $(0, 0)$ in the other calls, causing R to output x_0. For $n \geq 3$, this attack clearly cannot be emulated given access to (ideal) OLE since the receiver's input/output pairs $\{(0, 0), (1, 1), (2, 0), \ldots\}$ are not on the same line.

2.1.1 Reducing Communication Complexity via CRT

To achieve communication-efficient OT-based OLE, we first observe a more efficient OLE protocol for the case where n is a *smooth modulus*: $n = \prod_i n_i$ for some small, distinct primes n_i. For such an n, the OLE can be performed as follows: (1) For each n_i, the parties perform $\mathsf{OLE}_{\mathsf{n}_i}$ over their inputs modulo n_i. (2) The receiver reconstructs the output from the small outputs using the Chinese Remainder Theorem (CRT). This approach significantly reduces the communication complexity from $\approx \log(n)^2$ to $\approx \log(n) \cdot \log\log(n)$, assuming that n is the product of, say, the first consecutive primes.

The above gives rise to the following more efficient OLE over a prime modulus q, assuming the parties have access to $\mathsf{OLE}_{\mathsf{n}}$ for a smooth modulus n significantly larger than q. Specifically, if $n > q^2 \cdot 2^{\kappa_{\mathsf{s}}}$, where κ_{s} denotes a statistical security parameter, then $\mathsf{OLE}_{\mathsf{n}}$ can effectively be used as an OLE *over the integers* without any wraparound: the sender uses (a, b') as its input to $\mathsf{OLE}_{\mathsf{n}}$, where b' is sampled uniformly from the range $[0, n - q^2)$ subject to $b' = b \bmod q$. This standard randomization technique ensures that almost no information about a or b modulo q is leaked beyond the output. Note that for the honest receiver who uses its prescribed input $x \in \mathbb{Z}_q$, it holds that the value $y' = ax + b' \bmod n$ it received from $\mathsf{OLE}_{\mathsf{n}}$ does not wrap around n, since $ax + b' < n$, and it can reduce y' modulo q to obtain the desired value $y = ax + b \bmod q$. Asymptotically, the above protocol achieves $O(\log(q) \cdot \log\log(q))$ communication complexity compared to $\log(q)^2$ of Gilboa's protocol, a near-quadratic improvement. Unfortunately, unlike Gilboa's protocol, the CRT-based protocol is insecure against malicious receivers.

The large-input attack. Consider a malicious receiver $\widetilde{\mathsf{R}}^*$ that uses $x^* = \lfloor 2n/q \rfloor$ in the $\mathsf{OLE}_{\mathsf{n}}$ call instead of its prescribed input. Since $x^* \cdot a \geq n$ if and only if $a > n/\lfloor 2n/q \rfloor \approx q/2$, such a receiver causes the value $ax^* + b$ to wrap around n *based on the sender's input* a, and this behavior cannot be simulated in the ideal model with access to an $\mathsf{OLE}_{\mathsf{q}}$ functionality.[7]

2.2 Efficient OT-Based Malicious-Receiver OLE

To address the large-input attack, we instruct S to use *random* inputs in the $\mathsf{OLE}_{\mathsf{n}}$-call (of larger domains), and only *after* testing that R did not use a large input, S sends R the required information so it can retrieve the prescribed output. Specifically, we use the following protocol.

Protocol 2.2 (Malicious-receiver $\mathsf{OLE}_{\mathsf{q}}$)**.**

Additional parameter: Large enough prime $p \neq q$ such that $p \nmid n$.

Operation:

1. S samples $a' \xleftarrow{\mathrm{R}} (s_a)$, for sufficiently large s_a, and $b' \xleftarrow{\mathrm{R}} (n - s_a s_x)$.
2. R sets $x' \in [q \cdot p]$ such that $x' = x \bmod q$ and $x' = 0 \bmod p$.
3. The parties run $\mathsf{OLE}_{\mathsf{n}}$ on inputs (a', b') and x'. Let y' denote R's output.
4. R sends $y_0 \leftarrow y' \bmod p$ to S.
5. S:

[7] Consider an honest sender that uses uniform $(a, b) \xleftarrow{\mathrm{R}} \mathbb{Z}_q^2$ as input, and let y be the output obtained by the malicious receiver $\widetilde{\mathsf{R}}^*$, using input x^*, at the end of the execution. Assume that after the execution of Protocol 2.1 ends, the parties call an ideal function $\mathsf{IsConsist}$ to verify consistency: $\mathsf{IsConsist}$ gets (a, b) from S and (x, y) from R, and returns true iff $ax + b = y \bmod q$. Instruct $\widetilde{\mathsf{R}}^*$ to use $(x^* \bmod q, y)$ as its input to $\mathsf{IsConsist}$. Clearly, $\mathsf{IsConsist}$ accepts if $a < n/x^* - 1$, but rejects with high probability if $x \geq n/x^*$. When using an ideal OLE, on the other hand, the call to $\mathsf{IsConsist}$ either always accepts, if the malicious receiver is using the prescribed input and output, or rejects with very high probability (over S's input). We conclude that Protocol 2.1 is insecure against the malicious receiver $\widetilde{\mathsf{R}}^*$.

(a) Aborts if $b' \neq y_0 \bmod p$.

(b) Sends $(\delta_a, \delta_b) \leftarrow (a - a', b - b') \bmod q$.

(The above ensures that the parties obtain the correct correlation if no cheat is detected.)

6. R outputs $y \leftarrow y' + \delta_b + x \cdot \delta_a \bmod q$.

Namely, the test S uses is $b' \stackrel{?}{=} y_0 \bmod p$ for a large-enough prime p. Note that by taking $n > q^2$, the protocol is correct in an all-honest execution. It is also clear that the protocol is secure against semi-honest senders, i.e., the test leaks no information about x. Arguing security against malicious receivers is more challenging. Our strategy consists of proving the following regarding a malicious receiver R^* using input x^* in the OLE_n call:

1. If $x^* \leq 2^{\kappa_\mathsf{s}} \cdot q$, then the protocol is simulateable (using oracle to OLE_q).
2. Otherwise, R^* passes the test of Step 5a with only negligible probability.

Item 1 is easy to prove: for small x^*, the value of $y' \leftarrow x^* \cdot a' + b' \bmod n$ leaks only κ_s bits of information about a', intuitively, its most significant bits. Thus, by CRT, it leaks almost no information about a. However, as stated, Item 2 is *incorrect*: consider the malicious receiver R^* that uses $x^* \leftarrow p \cdot 2^{-1} \bmod n$ (we assume for simplicity that n is an odd number), and after obtaining y', it sends $y_0 = y' \bmod p$. It is easy to see that x^* is a very large input, and yet, the outcome R^* passes the test with probability $1/2$: if a' is even, then $y' = pa/2 + b$ (as integers). Thus, $y' \equiv b \bmod p$, and the test passes.

While such a value of x^* contradicts Item 2, a closer look reveals that the obtained leakage is inconsequential to the protocol's security. Specifically, due to CRT, the parity of a' does not reveal any information about $a' \bmod q$, and thus the protocol is secure for such a maliciously chosen x^*. More generally, for $x^* = c \cdot d^{-1} \bmod n$, where both c and d are small (e.g., $c \leq 2^{\kappa_\mathsf{s}} \cdot n/s_a$ and $d \leq 2^{\kappa_\mathsf{s}}$), hereafter *small* (c, d), we show in the technical sections that using such x^* leaks at most the values of $\lfloor ca'/n \rfloor$ and $a' \bmod d$. For a suitable choice of parameters, these values are completely independent of $a' \bmod q$.[8] Furthermore, given a small pair (c, d), the above method allows us to simulate the interaction of a corrupted receiver R^* with the sender, using an oracle to OLE_q. This simulation locally emulates the leakage of $\lfloor ca'/n \rfloor$ and $a' \bmod d$, both of which depend solely on ephemeral randomness and are independent of $a' \bmod q$ and δ_a, and thus independent of a. Given this observation, to complete the proof, we need to prove the following two facts:

1. There exists an efficient method to compute a small pair (c, d) from x^*, if such x^* exists.
2. If x^* admits no small pair, then the test in Step 5a of Protocol 2.2 passes with only negligible probability.

Finding a suitable pair (c, d) via rational reconstruction. Thankfully, computing small (c, d) from x^* corresponds to the classic *rational reconstruction problem*. Given $z \in \mathbb{Z}_n$ and bounds $r_c, r_d \in \mathbb{N}$ such that $n \geq 2r_c r_d$ (which our bounds on small (c, d) satisfy), the task is to find (c, d) such that $d \cdot z = c \bmod n$, $d \in [r_d]$ and $|c| \leq r_c$ where the ratio $c/d \in \mathbb{Q}$ is unique when viewed as a rational number. When a solution exists, the rational reconstruction problem can be solved efficiently as a byproduct of the extended GCD algorithm.

Unpredictability of modular multiplication. Note that the simulatable inputs, i.e., x^* for which a small (c, d) exists, consist of the elements in the following set:

$$\mathcal{X}_{n, s_a, \kappa_\mathsf{s}} := \{x \in \mathbb{N} \colon \exists c \in \mathbb{Z}, d \in [2^{\kappa_\mathsf{s}}] \text{ s.t. } |c| \leq n \cdot 2^{\kappa_\mathsf{s}}/s_a \wedge d \cdot x = c \bmod n\}.$$

The following lemma (proven using the so-called Thue's lemma, see Lemma 3.8) implies that the test in Step 5a of Protocol 2.2 passes with only negligible probability.

[8] Intuitively, $\lfloor ca'/n \rfloor$ leaks the higher-order bits of a', and $a' \bmod d$ leaks the lower-order bits of a', but both are independent of $a' \bmod q$.

Lemma 2.3 (Modular multiplication unpredictability, informal)**.** *Let $x^* \notin \mathcal{X}_{n,s_a,\kappa_s}$ and let $p > 2^{\kappa_s}$ be a prime such that $p \not| n$. Then, for any $y' \in \mathbb{Z}_n$ and $y_0 \in \mathbb{Z}_p$:*

$$\Pr_{a',b'}[y_0 = b' \bmod p \mid y' = x^* \cdot a' + b' \bmod n] \leq 2^{-\kappa_s}.$$

2.3 Towards Fully Malicious OLE

The case of a malicious sender is more complex, as, besides using large inputs, a malicious sender has an additional attack vector. To see that, let us take a closer look at how the protocol secures against large-input attacks, for both parties, is implemented over the prime divisors of n.

Protocol 2.4 (Preliminary fully malicious OLE protocol)**.**

Additional parameters: Let $n_1, \ldots, n_t$ denote the different prime divisors of n.

1. R samples $x' \xleftarrow{R} (s_x)$, for sufficiently large s_x.
2. S samples $a' \xleftarrow{R} (s_a)$, for sufficiently large s_a, and $b' \xleftarrow{R} (n - s_a s_x)$.
3. For $i = 1, \ldots, t$ (in parallel):

   Run Gilboa's protocol for OLE_{n_i} on input $(a', b') \bmod n_i$ for S and input $x' \bmod n_i$ for R. Let y'_i denote R's output.
4. R computes $y' \bmod n$ via CRT-reconstruction using $y'_1, \ldots, y'_t$.
5. The parties perform "consistency-tests" to detect large-input attacks. If no cheating is detected, R sends $\delta_x = x - x' \bmod q$, and S sends back $(\delta_a, \delta_b) = (a - a', b' - \delta_x \cdot a' - b) \bmod q$.
6. R outputs $y \leftarrow y' - x \cdot \delta_a - \delta_b \bmod q$.

We do not know how to analyze the security Protocol 2.4 against a malicious sender that induces corruption in the executions of the small factors OLE sub-protocols; consider the following selective failure attack: For $i = 1, \ldots, 10$, in the first OT-call inside the implementation of OLE_{n_i}, the sender sets the input to $(\delta_{i,0}, \perp)$ instead of $(\delta_{i,0}, \delta_{i,0} + a' \bmod n_i)$. Effectively, the sender is guessing that the parity bit of $a' \bmod n_i$ is zero for $i = 1, \ldots, 10$. Since R's input is random, the probability that S guesses correctly is approximately $1/1000$. In this case, it becomes challenging to analyze the distribution of $x' \bmod q$, given the leakage of the parity bits of $x' \bmod n_1, \ldots, x' \bmod n_{10}$ (or some other arbitrary leakage pattern).

To overcome the above issue, our security analysis assumes the entire value $x' \bmod n_i$ is leaked. This requires that the range of x', denoted s_x in Protocol 2.4, is suitably increased. Additionally, the product of the largest κ_s primes must be sufficiently smaller than s_x to ensure that the leakage remains independent of $x' \bmod q$.[9]

Input corruptions. A more subtle attack consists of *true* input corruptions rather than selective failures. Namely, in the first OT-call inside the implementation of OLE_{n_i}, the sender sets the input to $(\delta_{i,0}, \delta_{i,0} + a'' \bmod n_i)$ instead of $(\delta_{i,0}, \delta_{i,0} + a' \bmod n_i)$ for some unrelated value a''. Here, we benefit from the following key observations:

1. For any $\mathcal{I} \subseteq [t]$ such that $\prod_{i \in \mathcal{I}} n_i \leq s_x / 2^{\kappa_s}$, the receiver's inputs $(x' \bmod n_i)$ are statistically independent in each call to OLE_{n_i} for $i \in \mathcal{I}$. (This is guaranteed by the size of s_x and since x' is uniformly distributed in (s_x).)

[9] The rationale for using the largest κ_s primes is that this represents the strongest attack strategy, and the attacker will be caught with high probability if they cheat in more than κ_s primes.

2. Input corruptions in a call to $\mathsf{OLE}_{\mathsf{n}_i}$ make guessing the value of $a_0 \cdot x' - y' \bmod n_i$, for $x' \xleftarrow{R} (s_x)$, succeed with probability at most $1/2$, for any $a_0 \in \mathbb{Z}_{n_i}$.[10]

From the above items, we deduce that if the malicious sender cheats in ℓ different $\mathsf{OLE}_{\mathsf{n}_i}$ executions, then the probability of guessing $a_0x' - y' \bmod n$, i.e., $\max_{z \in (n)}\{\Pr[z = a_0x' - y' \bmod n]\}$, is bounded above by $2^{-\ell}$ (for any $a_0 \leq n$). In particular, this implies that guessing the value of $a_0x' - y'$ *over the integers* is also bounded above by $2^{-\ell}$ (the latter is strictly more challenging to prove than the former).

The above observation leads to the following potential solution for detecting input corruptions: have the attacker provide a guess $(a_0, z) \in \mathbb{Z}_n^2$ and check whether $z = a_0x' - y'$ over the integers. Of course, revealing the honest (a', b') is out of the question, as it would leak the honest sender's secrets. However, we can leverage the consistency test, which, *when the input corruptions are independent of the prime* p, essentially tests whether $z = a_0x' - y'$ over the integers. (For a high-entropy value w, guessing $w \bmod p$ for a random prime p is only slightly easier than guessing a uniform value modulo p.) To ensure that the input corruptions are independent of the prime, we instruct the receiver to sample a fresh random prime and hand it to the sender for the consistency test *only after* the sender's inputs to the OTs have been recorded.

Achieving full security via commit&modular-reveal. The final piece of the puzzle is enforcing that the attacker's guess—specifically the value of a_0—is independent of the random prime. To achieve this, we use a "commit&modular-reveal" functionality that, on input v provided by the sender, reveals $(w, c \bmod w)$, for a uniformly sampled prime w, to the receiver. Combining this functionality with Protocol 2.4 yields a fully secure protocol.[11]

2.4 Vector OLE

There are generic reductions from vector OLE (VOLE) to OLE. To construct the $\mathsf{VOLE}_{\mathsf{m,n}}$ protocol, however, the best such reduction costs $(m+1)$ times the cost of the $\mathsf{OLE}_{\mathsf{n}}$ protocol, see Appendix B. Yet, as in many other constructions, one can modify our OLE protocol to get a $\mathsf{VOLE}_{\mathsf{m,n}}$ in the cost of m times the cost of our $\mathsf{OLE}_{\mathsf{n}}$ protocol. The construction is straightforward for malicious-receiver VOLE. For a fully secure VOLE, the analysis requires some non-trivial work, and the construction induces some additional cost in communication. See details in the sections.

3 Preliminaries

3.1 Notation

We use calligraphic letters to denote sets, uppercase for parties and random variables, and lowercase for values and functions. All logarithms considered here are base 2. Unless stated explicitly otherwise, all ring operations of elements of $\mathbb{Z}_n$ are reduced modulo n. Let $\mathbb{N}$ denote the set of natural numbers. For $a \in \mathbb{R}$, let $\mathsf{abs}(a)$ denote its absolute value. For $n \in \mathbb{N}$, let $[n] := \{1, \ldots, n\}$, $(n) := \{0, \ldots, n\}$, and let $\mathbb{Z}_n = \mathbb{Z}/n\mathbb{Z}$ denote the ring of integers modulo n. For an algorithm, protocol, or functionality A that has a parameter p, let A_v denote A with parameter p set to v. We use $\leftarrow$ to indicate deterministic assignment. For a vector $\mathbf{a} = (a_1, \ldots, a_\ell)$, let $\mathbf{a}_i$ denote its i^{th} entry, i.e., a_i. For a set $\mathcal{S} \subseteq \mathbb{Z}$, let $\pm\mathcal{S} := \{a, -a \colon a \in \mathcal{S}\}$.

3.2 Entropy and Random Variables

All distributions considered in this work are finite. For a distribution X, let $x \xleftarrow{R} X$ denote sampling x according to X. Similarly, for a set $\mathcal{X}$, $x \xleftarrow{R} \mathcal{X}$ denotes sampling x uniformly from $\mathcal{X}$. The support of a distribution X over a discrete set $\mathcal{X}$, is defined by $\mathrm{Supp}(X) := \{x \in \mathcal{X} : X(x) > 0\}$. For two distributions/random

[10] Actually, this requires the receiver to randomly, and unbeknownst to the sender, permute the powers of two in the baseline Gilboa's protocol. We will not address this issue in this high-level overview.

[11] While enforcing using commit&modular-reveal to enforce independence of a_0 is critical for our current security proof, we speculate that the protocol remains secure without it. See discussion in Section 5.3.5.

variables A, B, let $\mathrm{SD}(A, B)$ denote their *statistical distance*. We use the standard notion of min-entropy and collision probability. To extend the notion of min-entropy to the conditional case, as common in this setting, we move from min-entropy to *predictability*, which in the non-conditional case is just the exponent of the min-entropy, i.e., $\mathrm{P}_\infty(X) := 2^{-\mathrm{H}_\infty(X)}$. We also use the natural definition of conditional collision probability.

Definition 3.1 (Conditional predictability and collision probability)**.** *For random variables X and Y, let*

$$\mathrm{P}_\infty(X \mid Y) := \mathrm{E}_{y \xleftarrow{R} Y}\left[2^{-\mathrm{H}_\infty(X|_{Y=y})}\right]$$
$$\mathrm{CP}(X \mid Y) := \mathrm{E}_{y \xleftarrow{R} Y}[\mathrm{CP}(X|_{Y=y})]$$

3.3 Primes and CRT

Let $\mathcal{P}$ denote the set of all primes, and for $\kappa \in \mathbb{N}$, let $\mathcal{P}_\kappa \subset \mathcal{P}$ denote the set of all κ-bit primes. We extensively use the Chinese Remainder Theorem.

Theorem 3.2 (Chinese Reminder Theorem (CRT))**.** *For any $\ell \in \mathbb{N}$, there exists an efficiently computable function $\mathsf{crt} : (\mathbb{Z}^{\geq 0}, \mathbb{Z}^{\geq 0})^\ell \mapsto \mathbb{Z}^{\geq 0}$ such that for relatively prime integers $\{n_1, \ldots, n_\ell\}$ and any $(z_1, \ldots, z_\ell) \in (n_1) \times \ldots (n_\ell)$, it holds that $z \leftarrow \mathsf{crt}((z_i, n_i)_{i\in[\ell]}) \in \left(\prod_{i\in[\ell]} n_i\right)$, and $z = z_i \bmod n_i$ for every $i \in [\ell]$.*

It is easy to see that crt is an injective and onto function from $(n_1) \times \ldots (n_\ell)$ to $(\prod_{i\in[\ell]} n_i)$. We also use the following fact.

Fact 3.3. *Let $q, p \in \mathbb{N}$ be relatively prime, Let $\mathcal{S} \subset \mathbb{Z}$ be an s-size set of consecutive numbers, let X and Q be uniformly distributed over $\mathcal{S}$ and $\mathbb{Z}_q$, respectively. Then, for any $x \in \mathbb{Z}_p$:*

$$\mathrm{SD}(X|_{X=x \bmod p} \bmod q, Q) \leq (pq-1)/s.$$

Proof. Let $s' \leftarrow \lfloor s/(pq) \rfloor \cdot pq$, let $\mathcal{S}'$ be the first s' elements of $\mathcal{S}$ and let $X' \xleftarrow{R} \mathcal{S}$. Since $pq \mid s'$, by CRT we have $X'|_{X'=x \bmod p} \bmod q$ is uniform over $\mathbb{Z}_q$. Since $\mathrm{SD}(X, X') \leq (pq-1)/s$, this concludes the proof . □

3.3.1 Number of Primes

For $n \in \mathbb{N}$, let $\pi(n)$ denote the number of primes in $[n]$, and let $\tau(\kappa) := |\{\mathcal{P}_\kappa\}|$, i.e., number of primes whose bit length is exactly κ. The following fact states a lower and upper bound on $\pi(n)$.

Fact 3.4 ([RS62])**.** *For any $n \geq 17$: $\frac{n}{\log n} < \pi(n) < 1.25506 \cdot \frac{n}{\log n}$.*

Fact 3.4 yields the following bound.

Proposition 3.5. *For any $\kappa \geq 6$: $\tau(\kappa) > 0.24 \cdot 2^\kappa/\kappa$.*

Proof. Clearly, $\tau(\kappa) = \pi(2^\kappa) - \pi(2^{\kappa-1})$. Hence, by Fact 3.4

$$\tau(\kappa) \geq \frac{2^\kappa}{\kappa} - \frac{1.25506}{2} \cdot \frac{2^\kappa}{\kappa-1} = \frac{2^\kappa}{\kappa} \cdot (1 - \frac{1.25506}{2} \cdot \frac{\kappa}{\kappa-1}) > \frac{2^\kappa}{\kappa} \cdot 0.24.$$

□

We also make use of the following bound.

Proposition 3.6. *Let $\mathcal{P}_{s_w}$ be the set of all primes of length $s_w \in \mathbb{N}$, and let $d_v \in \mathbb{N}$. If $s_w \geq 5$, then $\forall (v, v') \in (d_v)^2$ s.t. $v \neq v'$:*

$$\Pr_{w \xleftarrow{R} \mathcal{P}_{s_w}}[v \bmod w = v' \bmod w] < \frac{5 \log_2 d_v}{2^{s_w}}.$$

Proof. Without loss of generality, assume $v > v'$, and then let $\delta \leftarrow v - v'$. We have $\delta \in [d_v]$ and we know that $v \bmod w = v' \bmod w \implies \delta \bmod w = 0$. For any fixed $\delta \in [d_v]$, there can be no more than $\log_2 d_v/(s_w - 1)$ distinct values of w such that $\delta \bmod w = 0$.

On the other hand, per Proposition 3.5, we have $|\mathcal{P}_{s_w}| > 0.24 \cdot 2^{s_w}/s_w$ when $s_w \geq 6$. The probability that a value w is sampled such that $v \bmod w = v' \bmod w$ is therefore no more than

$$\frac{\log_2 d_v/(s_w - 1)}{|\mathcal{P}_{s_w}|} < \frac{s_w \cdot \log_2 d_v}{0.24 \cdot (s_w - 1) \cdot 2^{s_w}}$$

and since $s_w \geq 6$ the proposition follows. □

3.4 Thue's Lemma and Rational Reconstruction

Intuitively, the lemma below states that every $x \in \mathbb{Z}_n$ can be written as a 'modular' ratio $x = c \cdot d^{-1} \bmod n$ for $d \leq r_d$ and $c \leq r_c$, as long as $r_c \cdot r_d \geq n$. Furthermore, c and d are efficiently computable.

Lemma 3.7 (Effective Thue's Lemma)**.** *There exists a poly-time function* $\widetilde{\gcd}$ *that on input* $n, x, r_c, r_d \in \mathbb{N}$ *such that* $r_c \leq n < r_c r_d$*, outputs* $c, d \in \mathbb{Z}$ *such that:*

1. $d \in [r_d - 1]$, $\mathsf{abs}(c) \leq r_c - 1$,
2. $d \cdot x = c \bmod n$*, and*
3. $\gcd(d, \frac{dx-c}{n}) = 1$.

Proof. The above theorem, without the constraint on the gcd, corresponds to the effective Thue's lemma ([Sho06, Theorem 4.7]). To address the gcd constraint $\gcd(d, \lambda) = 1$ for $\lambda \leftarrow \frac{dx-c}{n}$, consider the parameters n, x, c, d as in the theorem statement, even if they do not satisfy $\gcd(d, \lambda) = 1$. Define $\lambda' = \gcd(d, \lambda)$; then n, x, c/λ', d/λ', λ/λ' satisfy all conditions of the theorem, including the gcd constraint. We mention that $\widetilde{\gcd}$ is essentially the extended GCD algorithm. □

The next lemma is somewhat of a strengthening of Lemma 3.7. Intuitively, it states if there exists $x \in \mathbb{Z}_n$ such that $x = c \cdot d^{-1} \bmod n$ for $d \leq r_d$ and $c \leq d \cdot r_c$, for $n \geq 2r_c r_d^2$, then c and d are efficiently computable.

Lemma 3.8 (Rational reconstruction)**.** *There exists poly-time function* $\widetilde{\gcd}$ *such that, on input* $n, x, r_d \in \mathbb{N}$ *and* $r_c \in \mathbb{R}^+$*, where* $n \geq 2r_c r_d^2$*,* $\widetilde{\gcd}$ *outputs* $(c, d) \in \mathbb{Z}^2$ *if such a pair exists that satisfies the following properties:*

1. $d \in [r_d]$ *and* $\mathsf{abs}(c) \leq d \cdot r_c$*, and*
2. $d \cdot x = c \bmod n$.

If no such pair (c, d) *exists, then* $\widetilde{\gcd}$ *outputs* $\perp$.

Proof. The above follows from the rational reconstruction problem ([Sho06, Theorem 4.9, p. 90]), which states that for $n \geq 2r'_c r_d$, $\widetilde{\gcd}$ outputs $c, d \in \mathbb{Z}$ such that $d \in [r_d]$ and $|c| \leq r'_c$ and $d \cdot x = c \bmod n$ and c/d is unique as a rational number if such a pair exists. Assume that such a pair exists for $r'_c = r_c \cdot r_d$, and that without loss of generality c and d are coprime. Either $c \leq d \cdot r_c$, in which case we are done. Otherwise, by the uniqueness of c/d as a rational number, no such pair exists. As in Lemma 3.7, we mention that $\widetilde{\gcd}$ is essentially the extended gcd algorithm. □

3.5 Universal Composability

We prove the security of our protocols in the universal composability (UC) framework [Can01], and in all cases we assume that the adversary has the following restrictions and capabilities:

- Security with *abort*: After getting its output, the adversary can instruct the ideal functionality to send $\perp$ instead of the actual output to any subset of the honest parties.
- *Static* corruptions: The adversary must choose which parties to corrupt before the interaction starts.

In the remainder of this work, "UC security" implies the above adversarial properties. When proving UC security, we utilize the following fact: for two-party, non-reactive functionalities, it suffices to provide *non-rewinding* (also known as "straight-line") simulators [KLR10, Thm. 5].[12] That is, to prove security, it suffices to prove correctness, and to provide a PPT straight-line (i.e., non-rewinding) simulator for the case that either of the parties is corrupt (for all possible inputs of the honest party).

Definition 3.9 (Statistical vs. computational security)**.** *We say that a protocol UC-realizes a functionality with* statistical [resp., *computational*] security ε *if the protocol has a straight-line simulator such that the statistical [resp., computational] distance between the real and emulated executions is at most* ε*.*

The security bound ε is typically a function of the security parameter given to the protocol. Where differentiation is important, we use κ_{s} to denote the parameter that determines statistical distance, and κ_{c} to denote the parameter that determines computational distance.

3.6 OT and OLE

We use the standard OT, Random OT (ROT), and Correlated OT (COT) functionalities. For the latter two, we use their so-called *corruptible* variant, which allows both parties, when corrupt, to determine their outputs. These weaker variants suffice for our applications, and in some settings easier to construct.

Functionality 3.10 ($\mathsf{OT_n}$)**.**

Parameters: $n \in \mathbb{N}$.

S's input: $(a_0, a_1) \in \mathbb{Z}_n^2$.

R's input: $i \in \{0,1\}$.

Output: Send a_i to R.

Functionality 3.11 ($\mathsf{ROT_n}$)**.**

Parameters: $n \in \mathbb{N}$.

Corrupt parties' input:
A corrupt sender can provide $a_0, a_1 \in \mathbb{Z}_q$. A corrupt receiver can provide $i \in \{0,1\}$ and $a_i \in \mathbb{Z}_q$.

Operation:
If (a_0, a_1) are set, sample $i \xleftarrow{R} \{0,1\}$.
Else, if (i, a_1) are set, sample $a_{1-i} \xleftarrow{R} \mathbb{Z}_q$;
else, sample $(a_0, a_1) \xleftarrow{R} \mathbb{Z}_n^2$ and $i \xleftarrow{R} \{0,1\}$.

Output:
Send (a_0, a_1) to S and (i, a_i) to R.

Functionality 3.12 ($\mathsf{COT_n}$)**.**

Parameters: $n \in \mathbb{N}$.

S's input: $\delta \in \mathbb{Z}_n$. A corrupt sender can also provide $a_0 \in \mathbb{Z}_q$.

R's input: $i \in \{0,1\}$. A corrupted receiver can also provide $a_i \in \mathbb{Z}_q$.

Operation: If a_0 is set, let $a_1 \leftarrow a_0 + \delta$. Else,

1. If a_i is not set, sample $a_i \xleftarrow{R} \mathbb{Z}_q$.

[12] [KLR10, Thm. 5] requires an implicit *start-synchronization* phase: all parties are active in the first round. But for two-party functionalities, it is easy to see that start-synchronization is not needed (it does not affect the adversary's abilities).

2. Let $a_{1-i} \leftarrow a_i + (-1)^i \delta$.

Output: Send a_0 to S and a_i to R.

We utilize the standard OLE and Vector OLE (VOLE) functionalities over the ring $\mathbb{Z}_n$.

Functionality 3.13 (OLE_n)**.**

Parameters: $n \in \mathbb{N}$.

S's input: $(a, b) \in \mathbb{Z}_n^2$.

R's input: $x \in \mathbb{Z}_n$.

Output: Send $y \leftarrow ax + b \bmod n$ to R.

Functionality 3.14 ($\mathsf{VOLE}_{m,n}$)**.**

Parameters: $m, n \in \mathbb{N}$.

S's input: $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_n^m$.

R's input: $x \in \mathbb{Z}_n$.

Operation: For each $i \in [m]$: let $y_i \leftarrow \mathbf{a}_i \cdot x + \mathbf{b}_i \bmod n$.

Output: Send $\mathbf{y} \leftarrow (y_1, \ldots, y_m)$ to R.

ROT to COT. All protocols in this paper can be implemented in the COT-hybrid model or the OT-hybrid; in the hybrid models, there is no difference in cost, because nothing extra must be transmitted. However, it is typical to instantiate both OT and COT in the ROT-hybrid model, because doing so enables preprocessing and the use of efficient techniques for generating batches of pseudorandom OT instances.[13] While OT_n can be implemented from ROT_n using $2\log n + 1$ bits of additional communication, COT_n requires only $\log n + 1$ bits. We therefore *present* our protocols in the COT-hybrid model, and analyze their costs in the ROT-hybrid model, assuming that the following COT protocol in the ROT-hybrid model is used.

Protocol 3.15 (Implementing COT using ROT)**.**

Parameters: $n \in \mathbb{N}$.

Oracle: ROT_n.

S's input: $\delta \in \mathbb{Z}_n$.

R's input: $i \in \{0, 1\}$.

Operations:

1. The parties jointly call ROT_n. Let (a_0, a_1) denote S's output and (i', z) denote R's output.
2. R: Send $f \leftarrow i \oplus i'$ to R.
3. S:
   (a) If $f = 1$, swap the values of a_0 and a_1.
   (b) Send $o \leftarrow a_1 - a_0 - \delta$ to R.

Output. S outputs a_0. R outputs z if $i = 0$, and $z - o$ otherwise.

Theorem 3.16 (Security of Protocol 3.15)**.** *Protocol 3.15* perfectly *UC-realizes* COT_n *in the* ROT_n*-hybrid model.*

Proof sketch: **Correctness.** By construction, S outputs a uniform element in $\mathbb{Z}_q$, as needed. By the swap done by S, it is effectively always the case that $i = i'$. Hence, R outputs a_0 if $i = 0$, and $a_0 + \delta$, otherwise.

Malicious sender. The simulator emulates a random execution of the protocol. Let (a_0, a_1) be the value the emulated call to the ROT functionality returned, assume for concreteness that the emulated receiver sent $f = 0$, and let o be the value the sender sent to the receiver. The simulator calls the COT oracle

[13] For example, Silent OT extension can generate m pseudorandom OT correlations with $o(m)$ communication.

with input $(a_0, \delta \leftarrow a_1 - a_0 - o)$. Clearly, the sender's view in both executions is the same, and by the setting of (a_0, δ), this is also the case when adding the receiver's output.

Malicious receiver. The simulator acts as follows:

1. Emulate a random execution of the protocol until the end of Step 2. Let $(i', a_{i'})$ be the output the emulated call to the ROT functionality returned to the receiver. Assume for concreteness that the receiver sent $f = 0$.
2. Call the COT oracle with input (i', a_0) if $i' = 0$, and with input i' otherwise. Let $a'_{i'}$ be the returned output.
3. If $i' = 0$, sample $o \xleftarrow{R} \boldsymbol{z}_q$; otherwise, let $o \leftarrow a_1 - a'_1$. Send o as the message S sends in Step 3.
4. Continue the emulation of the receiver til its end.

Assume $i' = 0$. In the real execution, the value of a_1 is hidden from the receiver. Thus, o is uniformly distributed and independent of the rest of the receiver's view and the sender's input and output. Since this is also the case in the emulated execution, the emulation is perfect.

Assume $i' = 1$. In the real execution, it holds that $a_1 - o = a_0 + \delta$, and S's output is a_0. In the emulated execution, it holds that $a_1 - o = a'_0 + \delta$, for a'_0 being S's output. Hence, given (a_1, o, δ), the outputs S's in the two executions are the same. Finally, we note that given the rest of the receiver's view and δ, the value of o in both executions is uniformly distributed. In the real execution, this holds since no information has leaked about a_0, and in the emulated execution, this holds since a'_1 is chosen uniformly.

□

4 Semi-Honest and Malicious-Receiver OLE

We begin this section by recalling Gilboa's malicious-receiver OLE protocol [Gil99]. In Section 4.2, we describe a semi-honest CRT-based OLE protocol, and in Section 4.2, we describe a CRT-based OLE protocol with security against malicious receivers. Both protocols are defined in the hybrid model of ideal OLE functionalities over *small* moduli;[14] these can be instantiated using Gilboa's protocol.

4.1 Gilboa's Malicious-Receiver OLE

We start by describing Gilboa's protocol using our notation.

Protocol 4.1 (Gilboa's malicious-receiver OLE)**.**

Parameters: $n \in \mathbb{N}$. Let $\ell \leftarrow \lceil \log n \rceil$.

Oracle. OT_n.

S's input: $a, b \in \mathbb{Z}_n$.

R's input: $x \in \mathbb{Z}_n$. Let $(x_0, \ldots, x_{\ell-1}) \in \{0,1\}^\ell$ be the bit-decomposition of x, i.e., $x = \sum_{i=0}^{\ell-1} x_i \cdot 2^i$.

Operations:

1. S: Sample $(b_0, \ldots, b_{\ell-2}) \xleftarrow{R} \mathbb{Z}_n^{\ell-1}$ and let $b_{\ell-1} = b - \sum_{i=0}^{\ell-2} b_i$.
2. For all $i \in (\ell - 1)$ (in parallel): the parties jointly call $\mathsf{OT}_n((b_i, b_i + 2^i \cdot a \bmod n), x_i)$. Let z_i denote R's output.

[14] Here *small* means roughly logarithmic in the modulus of the main protocol.

Output. R outputs $\sum_{i=0}^{\ell-1} z_i \bmod n$.

Note that when S is honest, any choice bits x_i of (a possibly malicious) R define a valid input $x = \sum_{i=0}^{\ell-1} x_i \cdot 2^i \bmod n$, such that the OT outputs obtained by R form a random additive secret sharing of the correct OLE output $ax + b$. This implies the following theorem.

Theorem 4.2 (Security of Protocol 4.1)**.** *For any $n \in \mathbb{N}$, Protocol 4.1* perfectly *UC-realizes OLE_n against semi-honest senders and malicious receivers, in the OT_n-hybrid model.*

4.1.1 Using Correlated OT

It is easy to convert Protocol 4.1 to work in the COT-hybrid model, which in turn can be cheaply implemented using random OT via Theorem 3.16. This follows from the observation that in all ℓ invocations of OT except the last one, the sender's first input is uniformly random. The same protocol can also be proved secure under the "endemic" variant of COT, which allows a malicious S to choose its own output and compute the output of R accordingly. This flavor of COT captures the functionality realized by pseudorandom correlation generators for COT [BCGIKS19; YWLZW20]. We formally describe the COT-based OLE protocol below.

Protocol 4.3 (Gilboa's malicious-receiver OLE, using correlated OT)**.**

Parameters: $n \in \mathbb{N}$. Let $\ell \leftarrow \lceil \log n \rceil$.

Oracle: COT_n.

S's input: $a, b \in \mathbb{Z}_n$.

R's input: $x \in \mathbb{Z}_n$. Let $(x_0, \ldots, x_{\ell-1}) \in \{0,1\}^\ell$ be the bit-decomposition of x.

Operations:

1. For all $i \in (\ell - 1)$ (in parallel): the parties jointly call COT_n, S using input $2^i \cdot a$ and R using input x_i. Let b_i denote S's output and z_i denote R's output.
2. S: Send $o \leftarrow b - \sum_{i=0}^{\ell-1} b_i$ to R.

Output. R outputs $o + \sum_{i=0}^{\ell-1} z_i \bmod n$.

The only difference between the view of R in Protocol 4.3 and Protocol 4.1 is that in Step 1 of Protocol 4.3 S uses a *random* value of $b_{\ell-1}$. Correctness is maintained because S transmits o, which is the difference between the values of $b_{\ell-1}$ in Protocol 4.3 and Protocol 4.1, and security is maintained because the receiver learns the same information from $z_i + o$ in Protocol 4.3 as it learned from z_i alone in Protocol 4.1.

Theorem 4.4 (Security of Protocol 4.3)**.** *For any $n \in \mathbb{N}$, Protocol 4.3* perfectly *UC-realizes OLE_n against semi-honest senders and malicious receivers, in the COT_n-hybrid model.*

Proof sketch: Correctness holds by constructions. Security against a malicious receiver holds since by controlling its output, which is allowed by the ROT functionality, only adds a known offset to $\sum b_i$, compared to the case where it let the functionality choose its output, and this offset can then be reduced from the value of b sent by S. Security against a malicious sender holds since moving from OT to COT does not give the sender additional power, and the offset o it sends in Step 2 is equivalent to changing the value of b. □

4.2 CRT-based Semi-Honest OLE

We now describe a simple OLE protocol for *smooth* moduli; that is, moduli that are the product of many small primes. This protocol achieves perfect security in the hybrid model of OLE instances over each of the small primes that divide the main smooth modulus. Afterward, we demonstrate that our OLE over smooth moduli can be used to construct OLE over *any* modulus q, and the resulting protocol is more efficient than using Protocol 4.3 over q directly.

4.2.1 Perfectly Secure OLE over Smooth Integers

Protocol 4.5 (Perfectly secure OLE over smooth integers)**.**

Parameters: $\{n_i\}_{i\in[t]}$. Let $n \leftarrow \prod_i n_i$.

Oracles: $\{\mathsf{OLE}_{\mathsf{n}_i}\}_{i\in[t]}$.

S's input: $a, b \in \mathbb{Z}_n$.

R's input: $x \in \mathbb{Z}_n$.

Operation. For all $i \in [t]$ (in parallel): The parties jointly call $\mathsf{OLE}_{\mathsf{n}_i}$, S using input $(a_i, b_i) \leftarrow (a, b) \bmod n_i$ and R using input $x_i \leftarrow x \bmod n_i$. Let y_i denote the output of R for this call.

Output. R: output $\mathsf{crt}((y_i, n_i)_{i\in[t]})$.

Theorem 4.6 (Security of Protocol 4.5)**.** *Let $\{n_i\}_{i\in[t]}$ be relatively prime, and let $n \leftarrow \prod_i n_i$. Protocol 4.5 with parameter $\{n_i\}_{i\in[t]}$,* perfectly *UC-realizes $\mathsf{OLE}_{\mathsf{n}}$ in the $\{\mathsf{OLE}_{\mathsf{n}_i}\}_{i\in[t]}$-hybrid model (against malicious adversaries).*

Proof. Correctness and security against semi-honest adversaries easily follow from CRT. Security against malicious parties follows correctness since the protocol gives no leeway for active adversaries to deviate from the correct execution. □

4.2.2 Semi-Honest OLE over Arbitrary Integers

The following protocol realizes $\mathsf{OLE}_{\mathsf{q}}$ for an arbitrary integer q against semi-honest adversaries. The protocol uses an oracle to $\mathsf{OLE}_{\mathsf{n}}$ for smooth $n \gg q$, which can be instantiated using Protocol 4.5. The reduction works by using the standard "statistical smudging" technique to reduce arithmetic modulo q to arithmetic over the integers, using n as an upper bound on the resulting integers to prevent wraparound. Unlike the previous protocol, however, here both parties have room to cheat; thus, we only get security against semi-honest adversaries.

Protocol 4.7 (Semi-honest OLE over arbitrary integers)**.**

Parameters: 1^{κ_s} and $q, n \in \mathbb{N}$.

Oracle: $\mathsf{OLE}_{\mathsf{n}}$.

S's input: $a, b \in \mathbb{Z}_q$.

R's input: $x \in \mathbb{Z}_q$.

Operation:

1. S: Sample $b' \xleftarrow{R} (n - q^2)$ conditioned on $b' = b \bmod q$.
2. The parties jointly call $\mathsf{OLE}_{\mathsf{n}}$, S using input (a, b') and R using input x. Let y denote the output of R for this call.

Output: R: Output $y \bmod q$.

Namely, the parties use $\mathsf{OLE}_{\mathsf{n}}$ for the receiver to compute $y = ax + b \bmod n$. Since $ax + b < n$, it holds that $y = ax + b$, and thus $y = ax + b \bmod q$.

Theorem 4.8 (Security of Protocol 4.7)**.** *For any $q, n \in \mathbb{N}$ with $n \geq q^2 \cdot 2^{\kappa_s}$, Protocol 4.7 UC-realize Functionality 3.13 against semi-honest adversaries, in the $\mathsf{OLE}_{\mathsf{n}}$-hybrid model, with statistical security $2^{-\kappa_s}$.*

Proof. Correctness and security against the semi-honest sender (who does not receive any messages) are straightforward. For the semi-honest receiver, consider the following simulator:

Algorithm 4.9 (Simulator for the semi-honest receiver)**.**

Oracle: $\mathsf{OLE_q}$.

Input: $x \in \mathbb{Z}_q$.

Operation:

1. Call $\mathsf{OLE_q}$ with input x, let y_q be the result.
2. Sample $y \xleftarrow{R} \{y_q, \ldots, n\}$ conditioned on $y = y_q \bmod q$, and send it to R as the answer of the $\mathsf{OLE_n}$ call.

Fix the input (a, b) of S and the input x of R, let $y \leftarrow ax + b$ and $y_q \leftarrow ax + b \bmod q$. Let B' be the value of b' sampled by S in the real execution, and let $Y \leftarrow ax + B'$, i.e., the output of $\mathsf{OLE_n}$ call. Note that Y is uniform over $\{y, \ldots, y + n - q^2 - q\}$ conditioned on $Y = y \bmod q$. In the simulated execution, the value of y is uniform over $\{y_q, \ldots, n\}$ conditioned on $y = y_q \bmod q$. Hence, the statistical distance between the two executions is bounded by $(y - y_q)/n \leq q^2/n \leq 2^{-\kappa_s}$. □

4.3 CRT-based Malicious-Receiver OLE

The CRT-based, malicious-receiver OLE protocol follows similar lines to the semi-honest protocol presented in Section 4.2, while immunizing it against the only effective attack a malicious receiver can apply: using a too large input x in the $\mathsf{OLE_n}$-call. At a high level, we enforce the malicious receiver to use "small" input x by asking it to provide $x_p, y'_p \in \mathbb{Z}_p$, for large enough prime p, and the receiver verifies that $a \cdot x'_p + b' = y'_p \bmod p$. An honest receiver, using $x \in \mathbb{Z}_q$, is guaranteed to pass the test. For x's not in $\mathbb{Z}_q$, we show that for some x's, the probability that a malicious receiver passes the test is negligible. For the other x's, for which it might pass the test, we prove that (although they are not in $[q]$) there is a way to simulate the receiver's actions. The protocol is formally defined below.

Protocol 4.10 (Malicious-receiver OLE)**.**

Parameters: 1^{κ_s} and $p, q, n \in \mathbb{N}$.

Derived parameters: Let $s_x \leftarrow qp$, $r_d \leftarrow 2^{\kappa_s+8}$, $s_a \leftarrow \max\{\max\{q, p\} \cdot r_d \cdot 2^{2\kappa_s+8}, p^2 \cdot 2^{\kappa_s+2}\}$ and $s_b \leftarrow n - s_x s_a - 1$.

Oracle: $\mathsf{OLE_n}$.

S's input: $a, b \in \mathbb{Z}_q$.

R's input: $x \in \mathbb{Z}_q$.

Operation:

1. S: Sample $a' \xleftarrow{R} (s_a)$ and $b' \xleftarrow{R} (s_b)$.
2. R: Let $x' \leftarrow \mathsf{crt}((x, q), (0, p))$.[a]
3. The parties jointly call $\mathsf{OLE_n}$, S using input (a', b') and R using input x'. Let y' denote R's output.
4. R: Send $y'_p \leftarrow y' \bmod p$ to S.
5. S:
   (a) Abort if $y'_p \neq b' \bmod p$.
   (b) Send $(\delta_a, \delta_b) \leftarrow (a - a', b - b') \bmod q$ to R.

Output: R: Output $y \leftarrow y' + \delta_b + \delta_a \cdot x \bmod q$.

[a]Hence, $x' = x \bmod q$ and $x' = 0 \bmod p$.

Note that, unlike Protocol 4.7, Protocol 4.10 is (also) parameterized by an integer p (which will later be set to be relatively prime to n and q), and that the value of a used as inputs to OLE_n is randomized. As stated, Protocol 4.10 has three rounds. In Section 4.3.3, we present a variant of the protocol that has only a single round (two rounds, when taking into account that the instantiation of the OLE_n protocol takes two rounds).

Theorem 4.11 (Security of Protocol 4.10)**.** *Let $\kappa_\mathsf{s}, q, p, n \in \mathbb{N}$, and define s_x, s_a as in Protocol 4.10. Assume p, q, n are relatively prime to each other, $p \geq 2^{\kappa_\mathsf{s}+2}$, and $n \geq 2^{\kappa_\mathsf{s}+3} \cdot s_x \cdot s_a$, then Protocol 4.10 UC-realize OLE_q in the OLE_n-hybrid model, against semi-honest senders, with statistical security $2^{-\kappa_\mathsf{s}}$.*

Proof. Correctness is proved in Claim 4.12, security against semi-honest senders is proved in Claim 4.13, and security against malicious receivers is proved in Claim 4.15. □

In the following, we fix the parameters $\kappa_\mathsf{s}, q, p, n$ that match the requirements of the theorem, and let r_d, s_x, s_a, s_b be as derived in Protocol 4.10.

Claim 4.12 (Correctness)**.** *Protocol 4.10 is perfectly correct.*

Proof. In an honest execution, it holds that $a'x' + b' = y' \bmod n$, and $ax + b < n$. Thus, $a'x' + b' = y'$ (without the modular operation). It follows that

$$y = y' + \delta_b + \delta_a x = a'x' + b' + (b - b') + x' \cdot (a - a') = ax' + b = ax + b \bmod q.$$

□

4.3.1 Semi-Honest Senders

Claim 4.13 (Semi-honest senders)**.** *There exists a simulator in the OLE_q-hybrid model that* perfectly *emulates the execution of S in the OLE_n-hybrid model.*

Proof. The simulator Sim for the semi-honest sender S is defined as follows:

Algorithm 4.14 (Sim)**.**

Oracle: OLE_q.

Input: $(a, b) \in \mathbb{Z}_q^2$.

Operation:

1. Call OLE_q with input (a, b).
2. Start emulating S. Let (a', b') denote the input provided by S to OLE_n.
3. Send $y'_p \leftarrow b' \bmod p$ to S as the message of R in Step 4.

Fix the input (a, b) of Sim and the random input (a', b') S provides to OLE_n. In the real execution, $y'_p = b' \bmod p$, which is exactly its value y'_p in the simulated execution (given the same view a', b'). Thus, the two views are identically distributed. □

4.3.2 Malicious Receivers

Claim 4.15 (Security against malicious receivers)**.** *For any* PPT *receiver for Protocol 4.10 in the* OLE_n*-hybrid model, there exists a* PPT *non-rewinding simulator in the* OLE_q*-hybrid model, such that for any input of the honest sender, the statistical distance between the real and simulated executions is at most* $2^{-\kappa_s}$*.*

To prove Claim 4.15, we distinguish between two types of (malicious) receivers. Receivers of the first type use "small" inputs to the OLE_n-call (the exact definition of "small" will be provided soon). We show that while the honest sender might not catch such receivers, they do not pose a danger to the protocol's security. We complete the picture by proving that, with high probability, the sender aborts when interacting with receivers that use "large" inputs to the OLE_n-call.

The above is formalized by providing a simulator for the variant of Protocol 4.10 resulting from replacing the functionality OLE_n with its variant $\widetilde{\mathsf{OLE}}_n$ that aborts on large receiver's inputs, and leaks some additional information needed for the simulation in case it does not abort. Let $\widetilde{\gcd}$ be the poly-time computable function guaranteed by Lemma 3.8 (i.e., Thue's lemma).

Functionality 4.16 ($\widetilde{\mathsf{OLE}}_n$: Leaky OLE)**.**

Parameters: $n \in \mathbb{N}$. Let $r_c \leftarrow r_d \cdot n/s_a$.

S's input: $(a, b) \in \mathbb{Z}_n^2$.

R's input: $x \in \mathbb{Z}_n$.

Operation:

1. Let $(c, d) \leftarrow \widetilde{\gcd}(n, x, r_d, r_c)$.
2. Abort if $(c, d) = \perp$.
3. Let $a_d \leftarrow a \bmod d$ and $y \leftarrow c \cdot (a - a_d)/d + b$.

Output: Send (a_d, y) to R.

Definition 4.17 (Protocol $\widetilde{\Pi}$)**.** *Let* $\widetilde{\Pi} = (\widetilde{\mathsf{S}}, \widetilde{\mathsf{R}})$ *be the variant of Protocol 4.10 in which the call to* OLE_n *is replaced with a call to* $\widetilde{\mathsf{OLE}}_n$*.*[15]

In Claim 4.18, we show that the leakage of $\widetilde{\mathsf{OLE}}_n$ on x is not harmful, and the call to $\widetilde{\mathsf{OLE}}_n$ done in $\widetilde{\Pi}$ can be simulated using a call to OLE_q. In Claim 4.21, we complete the picture by showing that (for malicious receivers) the call to OLE_n done in Protocol 4.10 can be simulated using a call to $\widetilde{\mathsf{OLE}}_n$.

Emulating access to $\widetilde{\mathsf{OLE}}_n$ using OLE_q.

Claim 4.18 ($\widetilde{\mathsf{OLE}}_n$ to OLE_q)**.** *For any* PPT *receiver* $\widetilde{\mathsf{R}}^*$ *for* $\widetilde{\Pi}$ *in the* $\widetilde{\mathsf{OLE}}_n$*-hybrid model, there exists a* PPT *non-rewind simulator* Sim *in the* OLE_q*-hybrid model, such that the statistical distance between the real and simulated executions is at most* $2^{-\kappa_s - 1}$*.*

Proof. Fix a malicious and without loss of generality deterministic receiver $\widetilde{\mathsf{R}}^*$, and consider the following simulator Sim:

Algorithm 4.19 (Sim)**.**

Oracle: OLE_q.

Operation:

[15] For honest R, the protocol is not well defined (R expect a single output from its call to OLE_n). This is not an issue, however, since we only use this protocol with malicious receivers that are aware of the change in the hybrid model.

1. Emulate a random execution of $(\mathsf{S}(0,0), \widetilde{\mathsf{R}}^*)$ until the last step (including the call to $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$). Abort if the emulation does. Let x' be the input $\widetilde{\mathsf{R}}^*$ provided to $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$. Let (a'_d, y') denote the value $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$ sent back to $\widetilde{\mathsf{R}}^*$.
2. Let $(c,d) \leftarrow \widetilde{\gcd}(n, x', r_d, r_c)$.
3. Call $\mathsf{OLE}_{\mathsf{q}}$ with input $x \leftarrow c \cdot d^{-1} \bmod q$.[a] Let y denote the returned output.
4. Sample $\delta_a \xleftarrow{R} \mathbb{Z}_q$ and set $\delta_b \leftarrow y - y' + cd^{-1}(\delta_a + a'_d) \bmod q$.

   Send (δ_a, δ_b) to $\widetilde{\mathsf{R}}^*$ as the message S sends in the last step of the protocol.

[a] $d^{-1} \bmod q$ exists since $d < q$ (and thus $\gcd(d,q) = 1$).

In the following, we refer to the execution induced by $\widetilde{\mathsf{R}}^*$ in $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$-hybrid model as the *real execution*, and the execution induced by Sim in the $\mathsf{OLE}_{\mathsf{q}}$-hybrid model as the *simulated execution*. Fix the input (a,b) of S. Since $\widetilde{\mathsf{R}}^*$ is deterministic, the input x' it provides to $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$ is also fixed. In the following, we omit these fixed values from $\widetilde{\mathsf{R}}^*$'s view. We assume without loss of generality that the call to $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$ does not abort and thus $d \cdot a' = c \bmod n$, $d \in [r_d]$ and $|c| < r_d n / s_a$, for c, d being as computed by $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$.

By construction, the partial views (y', a'_d) of $\widetilde{\mathsf{R}}^*$ are identically distributed in both executions. We next argue that in the real execution, with high probability over (y', a'_d), the distribution of δ_a conditioned on (y', a'_d) is statistically close to uniform over $\mathbb{Z}_q$, which by construction corresponds to its distribution in the simulated execution. Let (A', B') be the input S used in the call to $\widetilde{\mathsf{OLE}}_{\mathsf{n}}$. Since, $\delta_a = A' - a \bmod q$, we prove this part by bounding the distance of $A' \bmod q$ from uniform, conditioned on (y', a'_d). In the following, we assume for concreteness that $c \neq 0$; the proof for $c \neq 0$ follows similar lines. Let $A_0 \leftarrow (A' - A'_d)/d$, and observe that conditioned on a'_d, the value of A_0 is $(d/s_a < 2^{-\kappa_s - 4})$-close to uniform over $[\lfloor s_a/d \rfloor]$. Since B' is uniform over (s_b) and since $y' = A'x' + B \bmod n$, the value of A_0 conditioned on v' is $2^{-\kappa_s - 4}$-close to uniform over

$$\mathcal{A} := \begin{cases} [\lceil (y' - s_b)/c \rceil, \lfloor y'/c \rfloor] \cap [\lfloor s_a/d \rfloor] & c > 0, \\ [\lceil y'/c \rceil, \lfloor (y' - s_b)/c \rfloor] \cap [\lfloor s_a/d \rfloor] & c < 0. \end{cases}$$

Note that $\mathcal{A}$ is a determined by (a'_d, y'). We use the following claim (proven below).

Claim 4.20. $\Pr_{(a'_d, y')}\left[|\mathcal{A}| < q \cdot 2^{\kappa_s + 4}\right] \leq 2^{-\kappa_s - 2}$.

In the following, we assume $\mathcal{A}$ is large, i.e., $|\mathcal{A}| \geq qp \cdot 2^{\kappa_s + 4}$, and take care of the complementary case later. By Claim 4.20 and Fact 3.3, the value of $A_0 \bmod q$ conditioned on (y', a'_d) is $q/|\mathcal{A}| + 2^{-\kappa_s - 4} \leq 2^{-\kappa_s - 3}$ close to uniform over $\mathbb{Z}_q$. Since conditioned on (y', a'_d), there is in injective mapping between $A_0 \bmod q$ and $A' \bmod q$, the value of $A' \bmod q$ conditioned on (y', a'_d) is (also) $2^{-\kappa_s - 3}$-close to uniform over $\mathbb{Z}_q$.

We next analyze the distribution of $A' \bmod q$ conditioned on (a'_d, y', y'_p, e) for $e \leftarrow y'_p \stackrel{?}{=} B' \bmod p$—$\mathsf{R}^*$'s full view without (δ_a, δ_b). We prove that there exists a function t such that

$$\Pr\left[e \neq t(a'_d, y', y'_p)\right] \leq 2^{-\kappa_s - 3},$$

where the probability is over (A', B') conditioned on (a'_d, y'). Indeed, if $x' = 0 \bmod p$, then $e = 1$ iff $y'_p = y' \bmod p$. If $x' \neq 0 \bmod p$ then, since $B' = y' - A' \cdot x' \bmod p$, it holds that $e = 1$ iff $A' = (x')^{-1}(y' - y'_p) \bmod p$. Applying the same argument for proving the closeness to uniformity of $A' \bmod q$, yields that $A' \bmod p$ is $2^{-\kappa_s - 3}$ close to uniform given (a'_d, y'). Thus, the test passes in this case with probability at most $2^{-\kappa_s - 3}$. Hence, a simply hybrid argument yields that $A' \bmod q$ is $(2 \cdot 2^{-\kappa_s - 3} = 2^{-\kappa_s - 2})$-close to uniform over $\mathbb{Z}_q$ conditioned on (a'_d, y', y'_p, e). Taking into account the probability that $\mathcal{A}$ is not large, we deduce that the value of $(a'_d, y', y'_p, e, \delta_a)$ in the real and simulated executions are $2^{-\kappa_s - 1}$-close.

We conclude the proof by showing that δ_b, the missing part of the view, is a deterministic function of (v', δ_a) (in both executions), and thus can be ignored. Indeed, in the real execution, recalling that

$a'_d = A \bmod d$, $y' = c(A' - a'_d)/d + B'$, and letting $y \leftarrow acd^{-1} + b \bmod q$, it holds that

$$\begin{aligned}
\delta_b &= b - B' \\
&= b - (y' + cd^{-1}(A' - a'_d)) \\
&= y - acd^{-1} - (y' + cd^{-1}(A' - a'_d)) \\
&= y - y' + cd^{-1}(a'_d + a - A') \\
&= y - y' + cd^{-1}(\delta_a + a'_d) \bmod q,
\end{aligned}$$

which is the value of δ_b in the simulated execution. □

Proving Claim 4.20.

Proof of Claim 4.20. We prove for $c > 0$; the other case is analogous. We prove that $\mathcal{A}$ is large with high probability over A', which yields the claim since $\mathcal{A}$ is determined by (a'_d, y'). Let $\ell \leftarrow q \cdot 2^{\kappa_s+4}$. The proof follows by taking a union bound on the following probabilities, depending on all possible values (for the left and right borders) of $\mathcal{A}$.

1. $\mathcal{A} = [\lfloor s_a/d \rfloor]$. Then, $|\mathcal{A}| \geq s_a/d - 1 \geq s_a/(2r_d) \geq \ell$.
2. $\mathcal{A} = [\lceil (y' - s_b)/c \rceil, \lfloor y'/c \rfloor]$. Then, $|\mathcal{A}| \geq s_b/c - 1 \geq s_b/r_c - 1 \geq (n - s_x s_a - 1)/r_c - 1 \geq n/(2r_c) \geq s_a/(2r_d) \geq \ell$.
3. $\mathcal{A} = [0, \lfloor y'/c \rfloor]$. Compute

$$\begin{aligned}
\Pr[|\mathcal{A}| < \ell] &\leq \Pr[y'/c < \ell] = \Pr[y' \leq c\ell] = \Pr[A_0 c + b' \leq c\ell] \\
&\leq \Pr[A_0 c \leq c\ell] \leq \Pr[A_0 \leq \ell] \leq \frac{\ell}{s_a/d - 1} + 2^{-\kappa_s - 4} \\
&\leq \frac{2\ell r_d}{s_a} + 2^{-\kappa_s - 4} < 2^{-\kappa_s - 2}.
\end{aligned}$$

4. $\mathcal{A} = [\lceil (y' - s_b)/c \rceil, \lfloor s_a/d \rfloor]$. Since $|\mathcal{A}| \geq s_a/d - 1 - ((y' - s_b)/c + 1) + 1 \geq s_a/r_d + (s_b - y')/c - 1$, it holds that

$$\begin{aligned}
&\Pr[|\mathcal{A}| \leq \ell] \\
&\leq \Pr[s_a/r_d + (s_b - y')/c - 1 \leq \ell] \\
&= \Pr[c(s_a/r_d - A_0) + s_b - b' \leq c \cdot (\ell + 1)] \\
&\leq \Pr[s_a/r_d - A_0 \leq \ell + 1] \\
&\leq \Pr[A_0 \geq s_a/r_d - \ell - 1] \\
&\leq \frac{\ell}{s_a/r_d - 1} + 2^{-\kappa_s - 4} \\
&\leq \frac{2\ell r_d}{s_a} + 2^{-\kappa_s - 4} < 2^{-\kappa_s - 2}.
\end{aligned}$$

□

Emulating access to OLE_n using $\widetilde{\mathsf{OLE}}_n$. To conclude the proof of the malicious receiver part, we show how to simulate the interaction of a malicious receiver R^* with access to OLE_n with S by a malicious receiver with access to $\widetilde{\mathsf{OLE}}_n$. Specifically, we prove the following result

Claim 4.21 (OLE_n to $\widetilde{\mathsf{OLE}}_n$)**.** *For any malicious receiver R^* for Protocol 4.10 in the OLE_n-hybrid model, there exists a malicious receiver $\widetilde{\mathsf{R}}^*$ for $\widetilde{\Pi}$ in the $\widetilde{\mathsf{OLE}}_n$-hybrid model, such that the statistical distance between the real and simulated executions is at most $2^{-\kappa_s - 1}$.*

Fix a malicious, without loss of generality deterministic, R^* for Protocol 4.10. We define a malicious receiver $\widetilde{\mathsf{R}}^*$ for interacting with the honest sender $\widetilde{\mathsf{S}}$ in $\widetilde{\Pi}$ as follows.

Algorithm 4.22 ($\widetilde{\mathsf{R}}^*$)**.**

Oracle: $\widetilde{\mathsf{OLE}}_n$.

Operation: Interact with $\widetilde{\mathsf{S}}$ as follows:

1. Start emulating R^*. Let $x' \in \{0, \ldots, n-1\}$ denote the input provided by R^* to OLE_n.
2. Call $\widetilde{\mathsf{OLE}}_n$ with input x'. Let (a'_d, y') be the returned output.
3. Forward $y' + a'_d \cdot x' \bmod n$ to R^* as the answer of the call to OLE_n.
4. Interact with the sender till the end of the protocol by forwarding its messages to R^* and sending R^*'s responses back to it.

The heart of the proof of Claim 4.21 is in the following lemma, proved in Section 6.

Definition 4.23. *For $n, \ell, s \in \mathbb{N}$, let $\mathcal{X}_{n,s,\ell}$ be defined as*

$$\{x \in \mathbb{N} \colon \exists c \in \mathbb{Z}, d \in [\ell] \;\; s.t. \;\; |c| \leq dn \cdot \ell/s \;\wedge\; d \cdot x = c \bmod n\}.$$

Lemma 4.24 (Unpredictability of modular multiplication)**.** *Let $\ell, n, s \in \mathbb{N}$, let $x \in \mathbb{N} \setminus \mathcal{X}_{n,s,\ell}$, let $p \in \mathcal{P} \cap (2\ell, \infty)$ be such that $p \nmid n$ and $s/p > 16\ell^2$, and let $V \xleftarrow{R} [s]$ and $W \xleftarrow{R} \mathbb{Z}_n$. Then,*

$$\mathrm{P}_\infty(W \bmod p \mid x \cdot V + W \bmod n) \leq 42 \cdot \ell^{-1} + \frac{p^2}{s}.$$

Proof of Claim 4.21. In the following, we refer to the execution induced by R^* in OLE_n-hybrid model as the real execution, and the execution induced by $\widetilde{\mathsf{R}}^*$ in the $\widetilde{\mathsf{OLE}}_n$-hybrid model as the simulated execution. Let x' be the value provided by R^* to OLE_n. By construction, $\widetilde{\mathsf{OLE}}_n$ aborts on receiver's input x' iff $x' \notin \mathcal{X}_{n,s_a,r_d}$.

Let $A' \xleftarrow{R} (s_a)$ and $B' \xleftarrow{R} (s_b)$ denote the sender input in the real execution, and let $Y' \leftarrow A' \cdot x' + B' \bmod n$ be the value R^* received from OLE_n. Note that the sender's test in Step 5a passes iff R^* sends y'_p such that $y'_p = B' \bmod p$. Given this notation, we have to upperbound $\alpha \leftarrow \mathrm{P}_\infty(B' \bmod p \mid Y')$. Assume B' is uniform over $\mathbb{Z}_n$ (and not over (s_b)), then by Lemma 4.24 (letting $s \leftarrow s_a$, $x \leftarrow x'$, $V \leftarrow A'$ and $W \leftarrow B'$),

$$\alpha' \leftarrow \mathrm{P}_\infty(B' \bmod p \mid Y') \leq 42 \cdot 2^{-r_d} + p^2/s_a \leq 2^{-\kappa_s - 2} + 2^{-\kappa_s - 3}.$$

When taking into account the real distribution of B', we deduce that the probability that the test in Step 5a passes is bounded by $\alpha \leq \alpha' + s_b/n \leq \alpha' + 2^{-\kappa_s - 3} \leq 2^{-\kappa_s - 1}$.[16]

We conclude the proof by showing that the executions are *identical* if $x' \in \mathcal{X}_{n,s_a,r_d}$. Since in the simulated execution, the call to $\widetilde{\mathsf{OLE}}_n$ does not abort on such x'; the only potential difference might be in the answer of the call to OLE_n. Fix the random (a', b') sampled by S. In the simulated execution, the output of OLE_n is set to $y' + a'_d \cdot x \bmod n$, for $a'_d \leftarrow a' \bmod d$ and $y' \leftarrow (a' - a'_d) \cdot c/d + b'$, for some $c, d \in \mathbb{Z}$ such that $d \cdot x' = c \bmod n$. Compute

$$\begin{aligned} y' + a'_d \cdot x' &= (a' - a'_d) \cdot c/d + b' + a'_d \cdot x' \\ &= (a' - a'_d) \cdot d \cdot x'/d + b' + a'_d \cdot x' \\ &= (a' - a'_d) \cdot x' + b' + a'_d \cdot x' \\ &= a'x' + b' \bmod n, \end{aligned}$$

which is the output of OLE_n in the real execution. Thus, the two executions are identical for such x'. □

[16] Indeed, let B'' be the uniform distribution over $\mathbb{Z}_n$ and $Y'' \leftarrow A'x' + B''$. The statistical distance between (B', Y') and (B'', Y'') is just the distance between B' and B'' (Y' and Y'' are the same random function of B' and B'', respectively), which is s_b/n. Since the success of predicting B' given Y' (by an arbitrary algorithm) is a randomized function of (B', Y'), by the data-processing property of statistical distance, one cannot do it much better (better than s_b/n) than in guessing B'' given Y''.

Putting it together.

Proof of Claim 4.15. Immediately follows from Claims 4.18 and 4.21. □

4.3.3 Round Reduction

Using a standard conditional-disclosure-of-secret approach, Protocol 4.10 can be modified into a single-round protocol, two rounds when taking into account the instantiation of the $\mathsf{OLE}_{\mathsf{n}}$ protocol. For simplicity, we describe the protocol in the random-oracle model, but it can also be instantiated (somewhat less efficiently) using information-theoretic tools.

Let $\mathsf{RO}_{a,b}$ be the functionality that on input in $\mathbb{Z}_a$, returns a uniform, consistent output in $\mathbb{Z}_b^2$ (i.e., random oracle).

Protocol 4.25 (Two-Round, CRT-based malicious receiver OLE)**.**

Parameters: Same as in Protocol 4.10, and in addition, computational security parameter $1^{\kappa_{\mathsf{c}}}$

Oracle: $\mathsf{OLE}_{\mathsf{n}}$ and $\mathsf{RO}_{p,q}$.

S's input: $a, b \in \mathbb{Z}_q$.

R's input: $x \in \mathbb{Z}_q$.

Operation:

1. S: Sample $a' \xleftarrow{R} (s_a)$ and $b' \xleftarrow{R} (s_b)$.
2. R: Let $x' \leftarrow \mathsf{crt}(\{x, q\}, \{0, p\})$.
3. The parties jointly call $\mathsf{OLE}_{\mathsf{n}}$, S using input (a', b') and R using input x'. Let y' denote the output of R for this call.
4. S (in parallel to the above): Send $z \leftarrow (a' - a, b' - b) + \mathsf{RO}_{p,q}(b' \bmod p) \bmod q$ to R.
5. R: let $(\delta_a, \delta_b) \leftarrow z - \mathsf{RO}_{p,q}(y' \bmod p)$.

Output: R outputs $y \leftarrow y' - x \cdot \delta_a - \delta_b \bmod q$.

Theorem 4.26 (Security of Protocol 4.25)**.** *Let $\kappa_{\mathsf{s}}, q, p, \{n_i\}_{i\in[t]}$ be as in Theorem 4.11, and let $\kappa_{\mathsf{c}} \in \mathbb{N}$, and in addition assume $p \geq 2^{\kappa_{\mathsf{c}}}$. Then Protocol 4.25 on UC-realize $\mathsf{OLE}_{\mathsf{q}}$ in the $\{\mathsf{OLE}_{\mathsf{n}}, \mathsf{RO}_{p,q}\}$-hybrid model, against semi-honest senders with statistical security $2^{-\kappa_{\mathsf{s}}}$ and computational security $2^{-\kappa_{\mathsf{c}}}$.*

Proof sketch. The only non-trivial part in the proof is arguing security against a malicious sender $\widehat{\mathsf{S}}$ that uses input $x' \notin \mathcal{X}_{n,s_a,r_d}$ to the $\mathsf{OLE}_{\mathsf{n}}$ (an input that in the Protocol 4.10 would cause R to abort). We claim that in this case, the simulator in the $\mathsf{OLE}_{\mathsf{q}}$-hybrid model can sample the simulated z uniformly in $\mathbb{Z}_q^2$, and emulate the $\mathsf{OLE}_{\mathsf{n}}$ using independent random values a', b. Indeed, the security proof of Protocol 4.10 yields that the value of $b' \bmod p$ is unpredictable from $\widehat{\mathsf{S}}$'s point of view (for such x). Taking p to be larger than $2^{\kappa_{\mathsf{c}}}$, yields that in the real execution, $\widehat{\mathsf{S}}$ is unlikely to query on $b \bmod p$. Hence, the above emulation goes through. □

4.3.4 Vector OLE

It is easy to extend the malicious-receiver OLE from Protocol 4.10 into a malicious-receiver $\mathsf{VOLE}_{\mathsf{q},\mathsf{m}}$, for any $q, m \in \mathbb{N}$, in the $\mathsf{VOLE}_{\mathsf{n},\mathsf{m}}$-hybrid model:

1. In Step 1, S samples $\mathbf{a}' \xleftarrow{R} (s_a)^m, \mathbf{b}' \xleftarrow{R} (s_b)^m$.
2. In Step 3, the parties cal $\mathsf{VOLE}_{\mathsf{n},\mathsf{m}}$, S using $(\mathbf{a}', \mathbf{b}')$ and R using x.

3. The protocol continues, in parallel, for each of the m coordinates of the output.[17]

Since $\mathsf{VOLE}_{\mathsf{n},\mathsf{m}}$ can be implemented using a straightforward variant of Protocol 4.1, with communication cost that is m times that of the $\mathsf{OLE}_{\mathsf{n}}$ protocol, the communication cost of the above VOLE protocol is m times that of Protocol 4.10. It is also easy to verify that the resulting protocol, with the same choice of parameters as in Theorem 4.11, is secure against a semi-honest sender with security $m \cdot 2^{-\kappa_{\mathsf{s}}}$.

5 Fully Secure OLE

In this section, we show how to immunize the malicious-receiver OLE protocol from Section 4.3 against malicious senders. Thus, a fully secure OLE protocol is obtained. This immunization is not without a cost, in terms of communication and round complexity. We start in Section 5.2 by showing that a simple variant of Gilboa's protocol retains some (weak) level of security even when facing a malicious sender. In Section 5.3, we utilize this weak multiplication protocol to obtain a fully malicious OLE protocol. The latter OLE protocol uses a variant of a so-called commit&modular-reveal functionality, for which we present two implementations in Section 7. Finally, in Section 5.4, we show how to extend our OLE protocol to vector OLE.

5.1 Additional Preliminaries

This section presents the (rather standard) commit&modular-reveal functionality used by the fully malicious protocol. We also provide a combinatorial observation about random variables' unpredictability, which is used in the protocol's security proof.

5.1.1 Commit and Modular Reveal

The commit&modular-reveal functionality accepts a bounded-size v and publishes v mod w for a random w.

Functionality 5.1 ($\mathsf{Cmt\&ModRvl}$: commit & modular reveal)**.**

Parties: S and R.

Parameters: $d_v, \mathsf{s} \in \mathbb{N}$, and a set $\mathcal{W} \subseteq \mathbb{N}$.

S's input: $v \in (d_v)$. A corrupt S can use $v \in \pm(\mathsf{s} \cdot d_v)$.

Corrupt R's input: $w \in \mathcal{W}$.

Operation:

1. If w is not set, sample $w \xleftarrow{R} \mathcal{W}$.
2. Send $(w, v \bmod w)$ to both parties.

Leaky variant. The lightweight integer-commitment-based implementation presented in Section 7.1 only realizes a leaky variant of the functionality where a malicious receiver might obtain v mod ℓ, for some not too large ℓ.

Definition 5.2 (Modular-output function)**.** *A randomized function f is (ℓ, γ)-*$\mathsf{modular}$ *if* $\mathrm{SD}(f(x), f(x')) \leq \gamma$ *for any $x \equiv x' \bmod \ell$.*

Definition 5.3 (Games)**.** *A* game *is a two-party protocol between a* $\mathsf{challenger}$ *and a* player*. A game has* value *α if no (even unbounded) player makes the challenger accept with probability greater than α.*

[17] A slight save can be achieved by batching the $\mathbf{y}'_p$'s into a single element.

Functionality 5.4 ($\widetilde{\text{Cmt\&ModRvl}}$: leaky commit & modular reveal)**.**

Parties: S and R.

Parameters: 1^{κ_s}, $d_v, \mathsf{s} \in \mathbb{N}$ and $\mathcal{W} \subseteq \mathbb{N}$.

S's input: $v \in (d_v)$. A corrupt S can use $v \in \pm(\mathsf{s} \cdot d_v)$.

Corrupt R's input: $w \in \mathcal{W}$, function f and game G, both described as circuits,[a] and a proof that for some $\ell \in \mathbb{N}$: f is $(\ell, 2^{-\kappa_\mathsf{s}})$-modular and the value of G is at most $1/\ell + 2^{-\kappa_\mathsf{s}}$.[b]

Operation:

1. If R is corrupt:
   (a) Interact with R in G, taking the role of the challenger. Abort if no win.
   (b) Send $f(v)$ to R.
2. If w is not set, sample $w \xleftarrow{\text{R}} \mathcal{W}$.
3. Send $(w, v \bmod w)$ to both parties.

[a] G is an interactive circuit, describing the challenger, with a fixed number of rounds and fixed message length.
[b] The proof is a mathematical proof that can be verified via standard proof-checking tools.

That is, a malicious receiver can ask for (a function of) $v \bmod \ell$, but at the price of getting caught with probability $1 - 1/\ell$.

5.1.2 Condensing Predictability

In our fully malicious protocol, see Section 5.3, a random variable X of min-entropy k is reduced modulo an ℓ-bit random prime p. We want to argue that for large enough ℓ, the expected predictability of $X \bmod p$, given p, is close to k. Namely, modulus is a good (strong) condenser. Assuming the support of X is not too large relative to the size of p, then it is easy to see that mod p is a universal hash function: for any $x \neq x'$ it holds that $\Pr[x = x' \bmod p]$, for a random p, is not much larger than $2^{-\ell}$. It follows that conditional collision probability of $X \bmod p$, given p, is essentially 2^{-k} for $\ell > k$. Since min-entropy is at least half the Rényi entropy, we can deduce that the expected predictability of $X \bmod p$ given p is at $2^{-k/2}$. The following lemma yields that this factor-two loss is unnecessary, and the expected predictability of $X \bmod p$ given p is close to 2^{-k} (for large enough ℓ).

Lemma 5.5 (Condensing predictability)**.** *Let X be a random variable, let $\mathcal{H}$ be a function family over $\mathcal{X} = \text{supp}(X)$, and let H be uniformly distributed over $\mathcal{H}$. Assume that $\Pr[H(x) = H(x')] \leq 2^{-\ell}$ for every $x \neq x' \in \mathcal{X}$, then*

$$\mathrm{P}_\infty(H(X) \mid H) \leq \mathrm{P}_\infty(X) + 2^{-\ell/2}.$$

By the linearity of expectation, Lemma 5.5 yields the following corollary for the conditional case.

Corollary 5.6 (Condensing conditional predictability)**.** *Let X, Y be jointly distributed random variables, and let $\mathcal{H}$ be as in Lemma 5.5, then*

$$\mathrm{P}_\infty(H(X) \mid H, Y) \leq \mathrm{P}_\infty(X|Y) + 2^{-\ell/2}.$$

In the proof of Lemma 5.5, we use the following immediate fact:

Fact 5.7. *Let A, B be events with $B \implies A$, then $\Pr[A \wedge \neg B] = \Pr[A] - \Pr[B]$.*

Proof. $\Pr[A] = \Pr[A \wedge B] + \Pr[A \wedge \neg B] = \Pr[B] + \Pr[A \wedge \neg B]$. □

Proof of Lemma 5.5. For a set $\mathcal{S}$, let $\mathrm{CP}^-(\mathcal{S}) := \sum_{x \neq x' \in \mathcal{S}} p_X(x) \cdot p_X(x') = \Pr_{x,x' \xleftarrow{R} X}[x, x' \in \mathcal{S} \wedge x \neq x']$, i.e., the probability of hitting $\mathcal{S}$ twice, without getting the same element twice, and let $\alpha_{\mathcal{S}} := \max_{x \in \mathcal{S}}\{\Pr[X = s]\}$. It is easy to verify that, for any set $\mathcal{S}$,

$$\mathrm{CP}^-(\mathcal{S}) \geq \Pr_X[\mathcal{S}] \cdot (\Pr_X[\mathcal{S}] - \alpha_{\mathcal{S}}) \geq (\Pr_X[\mathcal{S}] - \alpha_{\mathcal{S}})^2 \tag{1}$$

For $h \in \mathcal{H}$, let $\mathcal{S}_h := h^{-1}(y)$ for $y \leftarrow \mathrm{argmax}_{y'} \Pr[h(X) = y']$, the preimages of the heaviest image and let $c(h) := \mathrm{CP}^-(\mathcal{S}_h)$. Let $k \leftarrow \mathrm{H}_\infty(X)$. Since $\mathrm{P}_\infty(h(X)) = \Pr_X[\mathcal{S}_h]$, it holds that

$$\begin{aligned} \mathrm{E}_H\left[(\mathrm{P}_\infty(H(X)) - 2^{-k})^2\right] &\leq \mathrm{E}_H\left[(\mathrm{P}_\infty(H(X)) - \alpha_{\mathcal{S}_H})^2\right] \\ &= \mathrm{E}_H\left[(\Pr_X[\mathcal{S}_H] - \alpha_{\mathcal{S}_H})^2\right] \\ &\leq \mathrm{E}_H\left[\mathrm{CP}^-(\mathcal{S}_H)\right] \\ &= \mathrm{E}_H[c(H)]. \end{aligned} \tag{2}$$

The first inequality holds by Equation (1). Note that

$$c(h) \leq \Pr_{x,x' \xleftarrow{R} X}[h(x) = h(x') \wedge x \neq x'] = \Pr_{x,x' \xleftarrow{R} X}[h(x) = h(x')] - \Pr_{x,x' \xleftarrow{R} X}[x = x'],$$

The inequality holds since the right-hand term accounts also for $x, x' \notin \mathcal{S}_h$, and the equality holds since $\{x = x'\} \implies \{h(x) = h(x')\}$ (see Fact 5.7). Hence,

$$\begin{aligned} \mathrm{E}_H[c(H)] &\leq \mathrm{E}_H\left[\Pr_{x,x' \xleftarrow{R} X}[H(x) = H(x')]\right] - \Pr_{x,x' \xleftarrow{R} X}[x = x'] \\ &= \Pr_{x,x' \xleftarrow{R} X, h \xleftarrow{R} \mathcal{H}}[h(x) = h(x')] - \mathrm{CP}(X) \\ &\leq 2^{-\ell} + \mathrm{CP}(X) - \mathrm{CP}(X) = 2^{-\ell}, \end{aligned} \tag{3}$$

and we conclude that

$$\mathrm{E}_H\left[(\mathrm{P}_\infty(H(X)) - 2^{-k})^2\right] \leq 2^{-\ell} \tag{4}$$

By Jensen's inequality, $\mathrm{E}_H\left[\mathrm{P}_\infty(H(X)) - 2^{-k}\right] \leq 2^{-\ell/2}$, and thus $\mathrm{P}_\infty(H(X) \mid H) = \mathrm{E}_H\left[\mathrm{P}_\infty(H(X)) - 2^{-k}\right] \leq 2^{-k} + 2^{-\ell/2}$. □

Remark 5.8 (Relation to the Chinese Remaindering error correction code). When considering the function family $\mathcal{H} = \{h_p\}_{p \in \mathcal{P}}$ where the elements of $\mathcal{P}$ are relatively prime, and $h_p(x) := x \bmod p$ (which is the family considered in Section 5.3), Lemma 5.5 is qualitatively related to the list-decodability of the *Chinese Remaindering error correction code*, introduced by Goldreich et al. [GRS99], where the message x is encoded as $(x \bmod p)_{p \in \mathcal{P}}$. Assume Lemma 5.5 does not hold with respect to $\mathcal{H}$ for some random variable X. By an averaging argument, there exists a list $\{y_p \in \mathbb{Z}_p\}$ such that for a large fraction $\mathcal{X} \in \mathrm{Supp}(X)$ it holds that $\Pr_p[h_p(x) = y_p \bmod p] \gg \mathrm{P}_\infty(X) + 2^{-\ell/2}$ for any $x \in \mathcal{X}$. The list-decodability of the Chinese Remaindering code yields that $\mathcal{X}$ is not too large, in contradiction to the assumed high min-entropy of X. The parameters of this reduction, however, are inferior to those we get in Lemma 5.5

5.2 Weak Multiplication

This section defines the weak multiplication functionality and shows that a simple variant of Gilboa's protocol realizes it (over prime inputs).[18]

[18] We emphasize that this functionality is unrelated to the functionality with the same name considered by [HMRT22].

5.2.1 The Ideal Functionality

The weak multiplication Wmult_n defined below makes use of the following definition of unpredictable functions,

Definition 5.9 (Unpredictable functions)**.** *A function* $f\colon \mathbb{Z}_n \times \{0,1\}^{t_1} \times \{0,1\}^{t_2} \mapsto \mathbb{Z}_n$ *is* unpredictable*, if*

$$\mathrm{E}_{r^1 \xleftarrow{R} \{0,1\}^{t_1}}\left[\max_x \left\{\mathrm{P}_\infty(f_{r^1}(Y,R^2) - x \cdot Y \bmod n)\right\}\right] \leq 1/2 + \mu_n,$$

for $f_{r^1}(y,r^2) := f(y,r^1,r^2)$*,* $(Y,R^2) \xleftarrow{R} \mathbb{Z}_n \times \{0,1\}^{t_2}$ *and* $\mu_n := \max\left\{0, \frac{8}{15\cdot(\lceil \log n \rceil - 1)} - \frac{1}{10}\right\}$.

That is, on average, over the choice of r^1, the value of $f_{r^1}(Y,R^2)$ is far from any linear function of Y.[19]

Functionality 5.10 (Wmult: weak multiplication)**.**

Parties: S and R.

Parameters: $n \in \mathbb{N}$.

S's input: $a \in \mathbb{Z}_n$. Corrupt S can provide $o^{\mathsf{S}} \in \mathbb{Z}_n$, a function $f\colon \mathbb{Z}_n \times \{0,1\}^{t^1} \times \{0,1\}^{t^2} \mapsto \mathbb{Z}_n$, described as circuit, and a proof π that f is unpredictable.

R's input: $x \in \mathbb{Z}_n$. Corrupt R can provide $o^{\mathsf{R}} \in \mathbb{Z}_n$.

Operation: If $f = \perp$,

1. Let $z \leftarrow a \cdot x \bmod n$.
2. If neither parties are malicious, let $o^{\mathsf{S}} \xleftarrow{R} \mathbb{Z}_n$ and $o^{\mathsf{R}} \leftarrow z - o^{\mathsf{S}} \bmod n$.
   Else, letting P be the malicious party and $\widehat{\mathsf{P}}$ being the honest one, set $o^{\widehat{\mathsf{P}}} \leftarrow z - o^{\mathsf{P}} \bmod n$.
3. Send o^{P} to P (for both P's).

Else,

1. Verify the proof π.
2. Sample $(r^1,r^2) \xleftarrow{R} \{0,1\}^{t_1} \times \{0,1\}^{t_2}$.
3. Set $o^{\mathsf{R}} \leftarrow f(x,r^1,r^2)$.
4. Send r^1 to S and o^{R} to R.

That is, the malicious sender can instruct the functionality to set the receiver's output to an arbitrary function of its input x, as long as this function is not close to a linear function (i.e., $ax + b \bmod n$). Namely, the sender either acts honestly or is far from honest. This functionality captures the effect a cheating sender has on the receiver's output in the execution of (a variant of) the Gilboa's protocol we consider in Section 5.2.2: the receiver samples (r^1,r^2) and sends r_1 to the sender for enabling it to compute its output.[20]

5.2.2 Implementation over Primes

In this section, we present an OT-based implementation of Functionality 5.10 for prime numbers. Our protocol is a simple variant of Gilboa's protocol (Protocol 4.1). In the following, let $\mathbb{S}_{(\ell-1)}$ be the set of all permutations over $(n) := \{0,\ldots,\ell-1\}$.

[19] The exact definition of the "farness" parameter μ_n is what we were able to achieve in our construction, and can be replaced with any function of n that converges to 0 (of course, the choice of function will effect the performance of our final protocol).

[20] One can show that the vanilla Gilboa's protocol also realizes this functionality, but with respect to a simpler notion of unpredictable function (r^1 is removed from the input of f). Alas, the parameter μ_n that measures the unpredictability of f obtained by this protocol (at least the one we managed to prove) is not as good, and thus using it would yield a lesser OLE protocol.

Protocol 5.11 (Implementation of Wmult over primes)**.**

Parameters: Prime $p \in \mathrm{P}$. Let $\ell \leftarrow \lceil \log p \rceil$.

Oracle: $\mathsf{OT_p}$.

S's input: $a \in \mathbb{Z}_p$.

R's input: $x \in \mathbb{Z}_p$.

Operations:

1. R:
   (a) If $x + p \geq 2^\ell$, let $c \leftarrow 0$; otherwise, sample $c \xleftarrow{R} \{0,1\}$.
   (b) Sample a random permutation $\tau \xleftarrow{R} \mathbb{S}_{(\ell-1)}$.
   (c) Let $(\lambda_0, \ldots, \lambda_{\ell-1}) \in \{0,1\}^\ell$ be the values with $x + c \cdot p = \sum_{i=0}^{\ell-1} \lambda_i \cdot 2^{\tau(i)}$.
2. For all $i \in (\ell - 1)$ (in parallel):
   (a) S: Sample $\delta_i \xleftarrow{R} \mathbb{Z}_p$.
   (b) The parties jointly call $\mathsf{OT_p}((\delta_i, \delta_i + a \bmod p), \lambda_i)$. Let z_i denote R's output.
3. R: Send τ to S.

Output:

S: $-\sum_{i=0}^{\ell-1} 2^{\tau(i)} \cdot \delta_i \bmod p$.

R: $\sum_{i=0}^{\ell-1} 2^{\tau(i)} \cdot z_i \bmod p$.

The differences from Protocol 4.1 are that (1) the binary representation of x is randomly permuted, and (2) for small values of x, we uniformly choose between x and $x + p$. Both modifications aim to make R's output more unpredictable from a cheating S's point of view. The security of Protocol 5.11 is stated in the following theorem, proven in Appendix C.

Theorem 5.12. *For any prime $p > 3$, Protocol 5.11 with parameter p, UC-realizes $\mathsf{Wmult_p}$ in the $\mathsf{OT_p}$-hybrid model.*

COT implementation. Note that Protocol 5.11 can readily be implemented, without additional cost, in the COT-hybrid model.

5.3 Fully Secure OLE from Weak Multiplication

In this section, we realize fully secure OLE using (1) the weak multiplication functionality from Section 5.2, (2) the Commit&modular-reveal functionality Cmt&ModRvl Functionality 5.1 (in Section 5.3.3, we extend the protocol to use the leaky variant of this functionality), (3) and a non-interactive perfectly hiding commitment functionality Com (Com can be realized up to arbitrary small statistical distance using a random oracle, or an OT oracle.) In Section 5.3.5, we present a simpler and more efficient variant of the protocol that does not use Cmt&ModRvl, and is information-theoretically secure under a plausible number-theoretic assumption.

Notation 5.13 (Additional notation)**.** *Let $\mathcal{P}_t$ be the set of t-bit primes. We use the following definitions with respect to a set $\{n_i\}_{i \in [t]} \subseteq \mathbb{N}$, which we assume for ease of notation that it is ordered in increasing order (i.e., $i < j \implies n_i \leq n_j$): let $\mathsf{lrgst}(j, \{n_i\}) := \prod_{i=t-j+1}^{t} n_i$. Let $\mathsf{unp}(\alpha, \{n_i\})$ be the minimal $j \in [t]$ such that $\prod_{i=1}^{j}(1/2 + \mu_{n_i}) \leq \alpha$ for μ being according to Section 5.2.1, (it equals ∞ if no such j exists).*

Protocol 5.14 (Fully secure $\mathsf{OLE}_{\mathsf{q}}$)**.**

Parameters: $1^{\kappa_{\mathsf{s}}}$, $q, p, \mathsf{s} \in \mathbb{N}$ and set $\{n_i\}_{i\in[t]} \subset \mathbb{N}$.

Derived parameters: Let $n \leftarrow \prod_{i\in[t]} n_i$, let $s_r \leftarrow 2(\kappa_{\mathsf{s}}+5)+\lceil 5 \cdot \mathrm{loglog}(\mathsf{s} \cdot n)\rceil$, $\mathsf{unpMul} \leftarrow \mathsf{unp}(2^{-\kappa_{\mathsf{s}}-4}, \{n_i\})$, $\mathsf{leak} \leftarrow \mathsf{lrgst}(\mathsf{unpMul}, \{n_i\})$, $r_{d,x} \leftarrow 2^{\kappa_{\mathsf{s}}+13}$ and $s_x \leftarrow \mathsf{leak} \cdot \max\{3qp \cdot r_{d,x}^3, 2^{2s_r+\kappa_{\mathsf{s}}+7}\}$, $s_a \leftarrow \max\{\max\{q,p\} \cdot r_{d,a} \cdot 2^{2\kappa_{\mathsf{s}}+8} \cdot 2^{s_r}, p^2 \cdot 2^{\kappa_{\mathsf{s}}+2}\}$ and $s_b \leftarrow n - s_x s_a - 1$.

Oracles: $\{\mathsf{Wmult}_{\mathsf{n_i}}\}_{i\in[t]}$, $\mathsf{Cmt\&ModRvl} \leftarrow \mathsf{Cmt\&ModRvl}_{s_a,\mathsf{s},\mathcal{P}_{s_r}\setminus\{q\}}$, Com.

S's input: $a, b \in \mathbb{Z}_q$.

R's input: $x \in \mathbb{Z}_q$.

Operation:

1. S: Sample $a' \xleftarrow{R} (s_a)$.
2. R: Sample $x' \xleftarrow{R} (s_x)$.
3. For all $i \in [t]$ (in parallel): the parties jointly call $\mathsf{Wmult}_{\mathsf{n_i}}$, with S using input $a'_i \leftarrow a' \bmod n_i$ and R using input $x'_i \leftarrow x' \bmod n_i$. Let o_i^{P} denote P's output (for $\mathsf{P} \in \{\mathsf{S},\mathsf{R}\}$).

   Each $\mathsf{P} \in \{\mathsf{S},\mathsf{R}\}$: Set $o^{\mathsf{P}} \leftarrow \mathsf{crt}((o_i^{\mathsf{P}}, n_i)_{i\in[t]})$.
4. The jointly parties call $\mathsf{Cmt\&ModRvl}$ with S, acting as the sender, using private input a'. Let (r, a'_r) be the common output.
5. R: Set $(x'_p, o_p^{\mathsf{R}}) \leftarrow (x', o^{\mathsf{R}}) \bmod p$, and send $c \xleftarrow{R} \mathsf{Com}(x'_p, o_p^{\mathsf{R}})$ to S.[a]
6. S: Send $o_r^{\mathsf{S}} \leftarrow o^{\mathsf{S}} \bmod r$ to R.
7. R:
   (a) Verify $o^{\mathsf{R}} + o_r^{\mathsf{S}} - a'_r \cdot x' \in \{0, n\} \bmod r$.[b]
   (b) Set $f \leftarrow 0$ if $o^{\mathsf{R}} + o_r^{\mathsf{S}} - a'_r \cdot x' = 0 \bmod r$; else set $f \leftarrow n$.
   (c) Open the commitment c to (x'_p, o_p^{R}).
   (d) Send $\delta_x \leftarrow x - x' \bmod q$ to S.
8. S:
   (a) Verify $o_p^{\mathsf{R}} + o^{\mathsf{S}} - a' \cdot x'_p \in \{0, n\} \bmod p$.
   (b) Send $(\delta_a, \delta_b) \leftarrow (a - a', b + o^{\mathsf{S}} + a'\delta_x) \bmod q$ to R.

R's output: $y \leftarrow o^{\mathsf{R}} + \delta_b + \delta_a x - f \bmod q$.

[a] To keep the text simpler, we do not name the random coins used by the randomized Com.
[b] The check is set to also accept $o^{\mathsf{R}} + o_r^{\mathsf{S}} - a'_r \cdot x' = n \bmod r$, since it might be that $o^{\mathsf{R}} + o^{\mathsf{S}} = a' \cdot x' + n$.

That is, the above protocol follows the lines of Protocol 4.10, the malicious-receiver OLE presented in Section 4.3, with the following modifications/additions: first, since we do not have an efficient malicious-sender implementation of the $\mathsf{OLE}_{\mathsf{n}}$ functionality in the OT-hybrid model, we replace it with calls to the weak multiplication functionality Wmult. Second, in Step 7a, we add a consistency check for the sender's input/output of the Wmult, which is analogous to the consistency check for the receiver in Protocol 4.10. Finally, since a cheating sender has an additional attack vector compared to the malicious receiver in Protocol 4.10, i.e., corrupting the Wmult calls, we use $\mathsf{Cmt\&ModRvl}$ to bind it to the value of a' (before knowing r). As stated, Protocol 5.14 has six rounds (counting each function call as a single round). In Section 5.3.4, we present a four-round variant of this protocol.

Remark 5.15 (How to choose the parameters concretely). We propose the following method for setting the parameters: set $\widetilde{n}$ to be equal to the product of the first κ_{s} primes and define $\widetilde{s}_a, \widetilde{s}_x$ with respect to $\widetilde{n}$ the same way s_a, s_x are defined with respect to n. If $\widetilde{n} > 2^{\kappa_{\mathsf{s}}+3} \cdot \widetilde{s}_a \cdot \widetilde{s}_x$, then set $(n, s_a, s_x) \leftarrow (\widetilde{n}, \widetilde{s}_a, \widetilde{s}_x)$. Otherwise, update $\widetilde{n}$ by setting $\widetilde{n} \leftarrow \widetilde{n} \cdot \widetilde{p}$ where $\widetilde{p}$ is the smallest prime which is not a factor of $\widetilde{n}$, and repeat the process.

For convenience, in Table 5.16 we provide concrete values for the minimal bit length of n, s_a, s_x and the minimal value of s_r, depending on the bit length of q, for various choices of interest.

$\lvert q\rvert$	κ_{s}	s_r	$\lvert s_a\rvert$	$\lvert s_x\rvert$	$\lvert n\rvert$
32	40	143	373	858	1281
64	40	143	373	858	1281
128	40	143	409	861	1320
256	40	143	537	873	1460
384	40	145	667	1013	1724
512	40	146	796	1151	1994
32	80	227	657	1598	2344
64	80	227	657	1598	2344
128	80	227	657	1598	2344
256	80	227	741	1603	2430
384	80	228	870	1616	2570
512	80	228	998	1759	2843

Table 5.16: Minimal parameters.

Theorem 5.17 (Security of Protocol 5.14)**.** *Let $\kappa_{\mathsf{s}}, q, p, \mathsf{s}, \{n_i\}_{i\in[t]} \in \mathbb{N}$, and define n, s_x, s_a as in Protocol 5.14. Assume $q, p, \{n_i\}$ are relatively prime, $p \geq 2^{\kappa_{\mathsf{s}}+3}$, and $n \geq 2^{\kappa_{\mathsf{s}}+3} \cdot s_x \cdot s_a$. Then Protocol 5.14 with parameters $(1^{\kappa_{\mathsf{s}}}, q, p, \{n_i\})$ UC-realizes $\mathsf{OLE}_{\mathsf{q}}$ in the $\{\{\mathsf{Wmult}_{\mathsf{n_i}}\}_{i\in[t]}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model, with statistical security $2^{-\kappa_{\mathsf{s}}}$.*

Proof. Correctness is stated in Claim 5.18, security against malicious receiver in Claim 5.19, and security against malicious senders in Claim 5.20. □

In the following, we fix the parameters $\kappa_{\mathsf{s}}, q, p, \{n_i\}_{i\in[t]}$ that match the requirements of the theorem, and let $n, s_r, \mathsf{unpMul}, \mathsf{leak}, r_{d,x}, s_x, r_{d,a}, s_a, s_b$ be as derived in Protocol 5.14.

Claim 5.18 (Correctness)**.** *Protocol 5.14 is* perfectly *correct.*

Proof. By construction, in an all-honest execution, it holds that $o^{\mathsf{S}} + o^{\mathsf{R}} = a' \cdot x' \bmod n$. Furthermore, since $a' \cdot x' < n$, it holds that $o^{\mathsf{S}} + o^{\mathsf{R}} - a' \cdot x' \in \{0, n\}$, and since $\gcd(r, n) = 1$, it holds that $o^{\mathsf{S}} + o^{\mathsf{R}} - a' \cdot x' = f$. Let $y \leftarrow ax + b \bmod q$. Compute

$$\begin{aligned} y &= ax + b \\ &= (\delta_a + a')(\delta_x + x') + b \\ &= \delta_a\delta_x + \delta_a x' + a'\delta_x + a'x' + b \\ &= \delta_a x + a'\delta_x + (o^{\mathsf{S}} + o^{\mathsf{R}} - f) + b \\ &= \delta_b + \delta_a x + o^{\mathsf{R}} - f \bmod q, \end{aligned}$$

which equals R's output in the protocol. □

5.3.1 Malicious Receivers

Security against malicious receiver follows similar lines to that of Theorem 4.11.

Claim 5.19 (Security against malicious receivers)**.** *For any malicious receiver for Protocol 5.14 in the* $\{\{\mathsf{Wmult}_{\mathsf{n}_i}\}_{i\in[t]}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$*-hybrid model, there exists a simulator in the* $\mathsf{OLE}_{\mathsf{q}}$*-hybrid model, such that the statistical distance between the real and simulated executions is at most* $2^{-\kappa_{\mathsf{s}}}$*.*

Proof. Since the only vector attack a malicious receiver $\widehat{\mathsf{R}}$ has in the calls to $\{\mathsf{Wmult}_{\mathsf{n}_i}\}$ is determining its part of the output, it is easy to verify that these calls can be simulated using a *single* call to the following variant of $\mathsf{OLE}_{\mathsf{n}}$:

- S's input is $a' \in (s_a)$.
- $\widehat{\mathsf{R}}$'s input is $x', o^{\mathsf{R}} \in \mathbb{Z}_n$.
- The functionality sends $o^{\mathsf{S}} \leftarrow a' \cdot x' - o^{\mathsf{R}} \bmod n$ to S.

Hence, the proof of Claim 5.19 follows the same line as the proof of Claim 4.15. The only differences are:

1. By construction, $\widehat{\mathsf{R}}$ also learns $(a'_r, o^{\mathsf{S}}_r) \leftarrow (a', o^{\mathsf{S}}) \bmod r$. Since the domain of a' is increased by a factor of 2^{s_r}, this compensates for the additional leakage.
2. There are now two values in $\mathbb{Z}_p$ that pass S's test in Step 8a. This vulnerability is addressed by the slight increment in the size of p.

□

5.3.2 Malicious Senders

In this part, we state and prove the security of Protocol 5.14 against malicious senders.

Claim 5.20 (Security against malicious senders)**.** *For any malicious sender for Protocol 5.14 in the* $\{\{\mathsf{Wmult}_{\mathsf{n}_i}\}_{i\in[t]}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$*-hybrid model, there exists a simulator in the* $\mathsf{OLE}_{\mathsf{q}}$*-hybrid model, such that the statistical distance between the real and simulated executions is at most* $2^{-\kappa_{\mathsf{s}}}$*.*

To prove Claim 5.20, we differentiate between two types of, without loss of generality deterministic, malicious senders, according to the number of Wmult corrupt calls they make, i.e., the calls in which the sender acts dishonestly by providing an unpredictable function (we assume without loss of generality that in all calls it provides o_{S}). Let $\mathcal{I} = \{i \in [t]\colon$ the sender corrupts the call to $\mathsf{Wmult}_{\mathsf{n}_i}\}$. We start, Claims 5.23 and 5.27, by handling the case that $|\mathcal{I}| \leq \mathsf{unpMul}$, and then complete the picture, Claim 5.35, by showing that R aborts with all but negligible probability in the other case. Recall that $\{n_i\}_{i\in[t]}, r_{d,x}, \mathsf{unpMul}$ and s_x are as in Protocol 5.14.

Few Wmult corruptions. This part of the proof is similar to security proof against malicious receivers done in Section 4.3 (proof of Claim 4.15). We distinguish between two types of malicious senders, depending on whether their inputs to the non-corrupt Wmult-calls correspond to a small or large input a'. For the "small a' senders, we show that while the receiver might not catch their misbehavior, they do not impose danger for the protocol security. We complete the picture by proving that the receiver aborts with high probability when interacting with "large-a senders".

Let $\widetilde{\gcd}$ be the poly-time function guaranteed by Lemma 3.8 (i.e., Thue's lemma), and let $\ell \leftarrow r_{d,x} \cdot pq$.

Functionality 5.21 ($\widetilde{\mathsf{Mult}}_{\mathsf{n}}$: leaky multiplication)**.**

S's input: $a \in \mathbb{Z}_n$, and a set $\mathcal{I} \subseteq [t]$ with $|\mathcal{I}| \leq \mathsf{unpMul}$.

R's input: $x \in \mathbb{Z}_n$.

Operation:

1. Let $n' \leftarrow \prod_{i\in[t]\setminus\mathcal{I}} n_i$, $r_c \leftarrow r_{d,x} \cdot n/s_x$, and $a' \leftarrow a \bmod n'$.

2. Let $(c', d') \leftarrow \widetilde{\gcd}(n', a', r_{d,x}, r_c)$.
3. Abort if $(c', d') = \perp$.
4. Send $y \leftarrow a \cdot x \bmod n$ to R.
5. Send $(x_{d'}, \mathcal{X} \leftarrow \{x_{n_i} \leftarrow x \bmod n_i\}_{i\in\mathcal{I}}, x_M \leftarrow x - (x \bmod \ell))$ to S.

That is, $\widetilde{\mathsf{Mult}}_\mathsf{n}$ leaks $x \bmod n_i$ for each corrupt $i \in \mathcal{I}$. In addition, it leaks $x_{d'}$ and x_M, the most significant bits of x.[21] We first prove, Claim 5.23, that the variant of Protocol 5.14 with access to $\widetilde{\mathsf{Mult}}_\mathsf{n}$ instead of $\{\mathsf{Wmult}_{\mathsf{n}_i}\}$, can be emulated using access to OLE_q, i.e., this variant security realizes OLE_q against malicious senders (this part corresponds to the "small-a senders"). We complete the picture, Claim 5.27, by showing that access to $\{\mathsf{Wmult}_{\mathsf{n}_i}\}$ can be emulated using access to $\widetilde{\mathsf{Mult}}_\mathsf{n}$ (this part corresponds to the "large-a senders").

Definition 5.22 (Protocol $\widetilde{\Pi}$)**.** *Let $\widetilde{\Pi} = (\widetilde{\mathsf{S}}.\widetilde{\mathsf{R}})$ be the variant of Protocol 5.14 in which the calls to $\{\mathsf{Wmult}_{\mathsf{n}_i}\}_{i\in[t]}$ are replaced by a single call to $\widetilde{\mathsf{Mult}}_\mathsf{n}$; $\widetilde{\mathsf{S}}$ using input a' and $\widetilde{\mathsf{R}}$ using input x'.*

Claim 5.23. *For any malicious sender $\widetilde{\mathsf{S}}^*$ for $\widetilde{\Pi}$ in the $\{\widetilde{\mathsf{Mult}}_\mathsf{n}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model, there exists a malicious receiver Sim in the OLE_q-hybrid model, such that the statistical distance between the real and simulated executions is at most $2^{-\kappa_\mathsf{s}-2}$.*

Proof. The proof follows similar lines to that of the malicious receiver case, though with some complications. For notation convenient, we assume $\widetilde{\mathsf{Mult}}_\mathsf{n}$ also outputs $(c, d) \leftarrow (n/n') \cdot (c', d')$. We also assume that rather than outputting $(x_{d'}, \mathcal{X}, x_M)$, it outputs $(x_d \leftarrow x \bmod d, x_M)$ (it is easy to verify that the original output can be computed from the new one). Fix a deterministic malicious sender $\widetilde{\mathsf{S}}^*$ for $\widetilde{\Pi}$, and consider the following simulator.

Algorithm 5.24 (Sim)**.**
Oracle: OLE_q.
Operation:

1. Emulate a random execution of $(\widetilde{\mathsf{S}}^*, \mathsf{R}(0))$ till its end. Abort if the emulation does.
   (a) Let $(a', \cdot)$ be the input provided by $\widetilde{\mathsf{S}}^*$ to the $\widetilde{\mathsf{Mult}}_\mathsf{n}$-call.
   (b) Let (x'_d, x'_M, c, d) be the output $\widetilde{\mathsf{S}}^*$ got back from this call.
   (c) Let $\delta_x, \delta_a, \delta_b, o_r^\mathsf{S}$ be the values R and $\widetilde{\mathsf{S}}^*$ sent, and let (r, a_r) be the output of $\mathsf{Cmt\&ModRvl}$.
2. Let $f^S \leftarrow 0$ if $o_r^\mathsf{S} = \lfloor (c \cdot d^{-1} \cdot (x'_M - x'_d) + a' x_d)/n \rfloor \cdot n \bmod r$; else set $f^S \leftarrow n$.
3. Let $a \leftarrow c \cdot d^{-1} + \delta_a \bmod q$ and $b \leftarrow -c \cdot d^{-1} \cdot (\delta_x + x'_d) + a' x'_d - \lfloor (a' x'_d + c x'_M / d)/n \rfloor \cdot n + \delta_b - f^S \bmod q$.
4. Call OLE_q with input (a, b),

Fix the input x of R, and note that since $\widetilde{\mathsf{S}}^*$ is deterministic, the value of $(a', \mathcal{I})$ it sends to $\widetilde{\mathsf{Mult}}_\mathsf{n}$ is also fixed. We assume without loss of generality that $(a', \mathcal{I})$ does not cause $\widetilde{\mathsf{Mult}}_\mathsf{n}$ to abort. In the following, we refer to the execution of $(\widetilde{\mathsf{S}}^*.\widetilde{\mathsf{R}})$ (with access to $\widetilde{\mathsf{Mult}}_\mathsf{n}$) as the *real execution* and the execution of $(\mathsf{Sim}, \mathsf{R})$ (with access to OLE_q) as the *simulated execution*. The heart of the proof lies in the following sub-claim:

Claim 5.25. *The values (x'_d, x'_M, δ_x) in $\widetilde{\mathsf{S}}^*$'s view in the real and the simulated executions are $2^{-\kappa_\mathsf{s}-3}$-close.*

[21] An alternative definition for Functionality 5.21 is to replace the leakage x_M with information that is tailored for the value a' used, i.e., in the spirit of Functionality 4.16. While such a definition yields a somewhat simpler proof, it particularly avoids the tedious proof of Claim 5.26, it is less suitable for our vector OLE protocol, as seen in Section 5.4.

Proof. By construction, the values of (x'_d, x'_M) are identically distributed in the both executions. We prove that in the real execution, with high probability over x'_M, the distribution of δ_x conditioned on (x'_d, x'_M) is statistically close to uniform over $\mathbb{Z}_q$, which by construction corresponds to its distribution in the simulated execution. Let X' denote R's input in the real execution. Since, $\delta_x = X' - x \bmod q$, we prove the claim by bounding the distance of $X' \bmod q$ from uniform, conditioned on (x'_d, x'_M). Since X' is uniform over (s_x), the value of X' conditioned on x'_M is uniform over

$$\mathcal{A} := \{x'_M, x'_M + 1, \ldots, \max\{x'_M + \ell - 1, s_x - 1\}\}$$

By Fact 3.3, $X' \bmod p$ conditioned on (x'_M, x'_d, x'_p) is $\frac{dpq-1}{|\mathcal{A}|}$-close to uniform over $\mathbb{Z}_q$, and thus it is $2^{-\kappa_s-4}$-close to uniform if $x_M \leq s_x - \ell$. Since $\Pr[x'_M > s_x - \ell] \leq (\ell - 1)/s_x \leq 2^{-\kappa_s-4}$, we deduce that the value of $v' \leftarrow (x'_d, x'_M, x'_d, \delta_x)$ in the real and simulated executions are $2^{-\kappa_s-3}$ close. □

Our next and final step is proving that if the value of (x'_d, x'_M, δ_x) in both executions is set to the same "typical" value, then the full views of $\widetilde{\mathsf{S}}^*$ and the output of R in the real and emulated executions are the same. The proof will then follow by Claim 5.25. Towards this end, we use the following sub-claim, proven below.

Claim 5.26. *There exists a set $\mathcal{B}ad$ such that the following holds:*

1. *For every $x' \in (s_x)$ with $(x'_d, x'_M) \notin \mathcal{B}ad$:*

$$\lfloor (c \cdot (x' - x'_d)/d + a' x'_d)/n \rfloor = \lfloor (c(x'_M - x'_d)/d + a' x'_d)/n \rfloor.$$

2. $\Pr_{X' \xleftarrow{R} (s_x)}[(X'_d, X'_M) \in \mathcal{B}ad] \leq 2^{-\kappa_s-3}$.

We next show that if the value of (x'_d, x'_M, δ_x) in both executions is set to the same value with $(x'_d, x'_M) \notin \mathcal{B}ad$, then the full views of $\widetilde{\mathsf{S}}^*$ and the output of R in the real and emulated executions are the same. By Claims 5.25 and 5.26, it follows that the overall distance between the real and emulated executions is bounded by $2^{-\kappa_s-2}$, concluding the proof.

Fix the value of (x'_d, x'_M, δ_x) so that $(x'_d, x'_M) \notin \mathcal{B}ad$, and consider a real execution in which R uses input x', consistent with (x'_d, x'_M, δ_x), in the call to $\widetilde{\mathsf{Mult}_\mathsf{n}}$. Starting with the full view of $\widetilde{\mathsf{S}}^*$, we note that the only part that is not (at least not syntactically) a deterministic function of (x'_d, x'_M, δ_x) is the output of the test $e := o^{\mathsf{R}} + o^{\mathsf{S}}_r - a'_r \cdot x' \stackrel{?}{\in} \{0, n\} \bmod r$ (done by R). Since $o^{\mathsf{R}} = a' \cdot x' \bmod n$ and $(x'_d, x'_M) \notin \mathcal{B}ad$,

$$\begin{aligned} e = 1 &\iff o^{\mathsf{S}}_r + (a' \cdot x' \bmod n) - a' \cdot x' \in \{0, n\} \bmod r \\ &\iff o^{\mathsf{S}}_r - \lfloor a' \cdot x'/n \rfloor \cdot n \in \{0, n\} \bmod r \\ &\iff o^{\mathsf{S}}_r - \lfloor (c \cdot (x' - x'_d) + a' \cdot x'_d)/n \rfloor \cdot n \in \{0, n\} \bmod r \\ &\iff o^{\mathsf{S}}_r - \lfloor (c(x'_M - x'_d)/d + a' x'_d)/n \rfloor \cdot n \in \{0, n\} \bmod r. \end{aligned} \tag{5}$$

The last transition holds since $(x'_d, x'_M) \notin \mathcal{B}ad$. Hence, e, for $(x'_d, x'_M) \notin \mathcal{B}ad$, is a deterministic function of the rest of the view, and thus takes the same value as in the simulated execution.

Moving to R's output, let $v \leftarrow (x'_d, x'_M, \delta_x, \delta_a, \delta_b, \ldots,)$ denote $\widetilde{\mathsf{S}}^*$'s view determined by (x'_d, x'_M, δ_x) in the real execution, and assume without loss of generality that v is a non-aborting view. Let f be the value computed by R in the real execution, as determined by x' and v. In the real execution, R's output is

$$\begin{aligned} y^R &\leftarrow (a' \cdot x' \bmod n) + \delta_b + \delta_a x + f \\ &= (c \cdot (x' - x'_d)/d + a' x'_d \bmod n) + \delta_b + \delta_a x + f \\ &= c \cdot d^{-1} \cdot (x' - x'_d) + a' x'_d - \lfloor (c \cdot (x' - x'_d)/d + a' x'_d)/n \rfloor \cdot n + \delta_b + \delta_a x + f \\ &= c \cdot d^{-1} \cdot (x' - x'_d) + a' x'_d - \lfloor (c(x'_M - x'_d)/d + a' x'_d)/n \rfloor \cdot n + \delta_b + \delta_a x + f \\ &= c \cdot d^{-1} \cdot (x - \delta_x - x'_d) + a' x'_d - \lfloor (c(x'_M - x'_d)/d + a' x'_d)/n \rfloor n + \delta_b + \delta_a x + f \\ &= (c \cdot d^{-1} + \delta_a) x - c \cdot d^{-1} \cdot (\delta_x + x'_d) + a' x'_d - \lfloor (c(x'_M - x'_d)/d + a' x'_d)/n \rfloor n + \delta_b + f \bmod q. \end{aligned}$$

The third equality holds since $(x'_d, x'_M) \notin \mathcal{B}ad$. Note that since v is non-aborting,

$$f = 0 \iff o_r^{\mathsf{S}} - \lfloor a' \cdot x'/n \rfloor \cdot n = 0 \bmod r \iff o_r^{\mathsf{S}} - \lfloor a' \cdot x'_M/n \rfloor \cdot n = 0 \bmod r$$

and otherwise $f = n$. In the simulated execution, letting (a, b, f^S) be the values computed by Sim, R's output is

$$\begin{aligned} y^S \leftarrow ax + b &= (c \cdot d^{-1} + \delta_a)x - c \cdot d^{-1} \cdot (\delta_x + x'_d) + a'x'_d - \lfloor (a'x'_d + cx'_M/d)/n \rfloor \cdot n + \delta_b - f^S \\ &= (c \cdot d^{-1} + \delta_a)x - c \cdot d^{-1} \cdot (\delta_x + x'_d) + a'x'_d - \lfloor (a'x'_d + cx'_M/d)/n \rfloor \cdot n + \delta_b - f \\ &= y^R \bmod q. \end{aligned}$$

The first equality holds by the definition of f^S. This concludes the proof of Claim 5.23. □

Proof of Claim 5.26. We assume $c > 0$; the case $c \leq 0$ follows similar lines. Fix (x'_d, x'_M), let $x' \in (s_x)$ be consistent with (x'_d, x'_M) and let $x'_L \leftarrow x' - (x' \bmod \ell)$. We write

$$\lfloor (c \cdot (x' - x'_d)/d + a'x'_d)/n \rfloor = \left\lfloor (\frac{c}{d} \cdot x'_M + a'x'_d + \frac{c}{d} \cdot (x'_L - x'_d))/n \right\rfloor \tag{6}$$

Hence, we need to find a set $\mathcal{B}ad$ such that if $(x'_d, x'_M) \notin \mathcal{B}ad$ then removing $\frac{c}{d} \cdot (x'_L - x'_d)/n$ from the above right-hand-side $\lfloor \cdot \rfloor$ does not change its value. Let $\mathsf{gap} := n/(3r_{d,x})$. Since $c \leq r_{d,x}n/s_x \leq 1/(3\ell r_{d,x})$,

$$\frac{c}{d} \cdot (x'_L - x'_d) \leq c \cdot x'_L \leq n/(3_{d,x}) = \mathsf{gap}.$$

Hence, Equation (6) holds for any $(x'_d, x'_M) \notin \mathcal{B}ad$ for the set $\mathcal{B}ad$ defined by

$$\mathcal{B}ad := \{(x'_d, x'_M) \colon \frac{c}{d} \cdot x'_M + a'x'_d \in \bigcup_{z \in \mathbb{Z}} [zn - \mathsf{gap}, zn - 1]\}.$$

So in the following we need to bound $\Pr_{X' \xleftarrow{R} (s_x)}[(X'_d, X'_M) \in \mathcal{B}ad]$. Let $v \in \mathbb{Z}$ be such that $da' = c + vn$ (recall that $da' = c \bmod n$). Thus, $a'x'_d = \frac{1}{d}(c + vn)x'_d = \frac{c}{d}x'_d + \frac{vn}{d}x'_d$. Therefore,

$$a'x'_d \bmod n \leq c + n(1 - 1/d) \leq n + n/(3r_{d,x}) - n/r_{d,x} = n - 2\mathsf{gap}.$$

Hence,

$$\Pr_{X' \xleftarrow{R} (s_x)}[(X'_d, X'_M) \in \mathcal{B}ad] \leq \max_{y \in (n - 2\mathsf{gap})} \left\{ \Pr_{X' \xleftarrow{R} (s_x)}[\mathcal{B}ad_y] \right\} \tag{7}$$

for $\mathcal{B}ad_y := \left\{x'_m \colon y + \frac{c}{d}x'_m \in \bigcup_{z \in \mathbb{Z}} [zn - \mathsf{gap}, zn - 1]\right\}$. We conclude the proof by showing that for any $y \in (n - 2\mathsf{gap})$:

$$\Pr_{X' \xleftarrow{R} (s_x)}[\mathcal{B}ad_y] \leq 2^{-\kappa_s - 3} \tag{8}$$

Fix y and let $\alpha \leftarrow c/d$. Without loss of generality, $\alpha s_x \geq \mathsf{gap}$; otherwise, Equation (8) holds trivially. Note that X'_M is (ℓ/s_x)-close to uniform over the set $\mathcal{C} = \{0, \ell, 2\ell, \ldots, \lceil s_x/\ell \rceil - \ell\}$. Also note that for any $\lceil n/(\alpha\ell) \rceil$ consecutive values in $\mathcal{C}$, at most $t \leftarrow 1 + n/(\alpha\ell\mathsf{gap}) < 2n/(\alpha\ell\mathsf{gap})$ of them are in $\mathcal{B}ad_y$, and thus at most $(2/\mathsf{gap} + t/|\mathcal{C}| \leq 3/\mathsf{gap})$ fraction of $\mathcal{C}$ is in $\mathcal{B}ad_y$. We conclude that $\Pr[X'_M \in \mathcal{B}ad_y] \leq \ell/s_x + 3/\mathsf{gap} < 2^{-\kappa_s - 3}$. □

Emulating access to $\{\mathsf{Wmult}_{\mathsf{n}_i}\}$ using $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$.

Claim 5.27 ($\{\mathsf{Wmult}_{\mathsf{n}_i}\}$ to $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$). *For any* PPT *malicious sender* S^* *for Protocol 5.14 in the* $\{\{\mathsf{Wmult}_{\mathsf{n}_i}\}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$*-hybrid model with* $|\mathcal{I}| \leq \mathsf{unpMul}$*, there exist a malicious* PPT *sender* $\widetilde{\mathsf{S}}^*$ *for* $\widetilde{\Pi}$ *in the* $(\widetilde{\mathsf{Mult}}_{\mathsf{n}}, \mathsf{Cmt\&ModRvl}, \mathsf{Com})$*-hybrid model, such that the statistical distance between the real and simulated executions is at most* $2^{-\kappa_s - 2}$.

To prove Claim 5.27, we show that the calls to Wmult can be simulated using calls to the following simpler-to-understand functionality.

Functionality 5.28 ($\widehat{\mathsf{Wmult}}$: leaky variant of Wmult)**.**

Parameters: $t \in \mathbb{N}$.

S's input: $a \in \mathbb{Z}_t$ and $\mathsf{leak} \in \{0,1\}$.

R's input: $x \in \mathbb{Z}_t$.

Operation:

1. Send $a \cdot x \bmod t$ to R
2. If $\mathsf{leak} = 1$, send x to S.

Namely, rather than adding noise to the output of R, the sender can request R's input.

Definition 5.29 (Protocol $\widehat{\Pi}$)**.** *Let $\widehat{\Pi} = (\widehat{\mathsf{S}}, \widehat{\mathsf{R}})$ be the variant of Protocol 5.14 in which the calls to $\{\mathsf{Wmult}\}$ are replaced with calls to $\{\widehat{\mathsf{Wmult}}\}$.*

Fix a malicious S^* for Protocol 5.14, and consider the following malicious sender $\widehat{\mathsf{S}}^*$ for $\widehat{\Pi}$.

Algorithm 5.30 ($\widehat{\mathsf{S}}^*$)**.**

Oracles: $\{\widehat{\mathsf{Wmult}}_{\mathsf{n_i}}\}_{i\in[t]}$,$\mathsf{Cmt\&ModRvl}$,$\mathsf{Com}$.

Operation: Interact with $\widehat{\mathsf{R}}$ in $\widehat{\Pi}$ as follows:

1. Emulate the execution of S^* until Step 3. Let $\{(a'_i, o_i^{\mathsf{S}}, f_i)\}_{i\in[t]}$ be the inputs it provided to the calls to $\{\mathsf{Wmult}_{\mathsf{n_i}}\}$, and let $\mathcal{I} \leftarrow \{i \in [t] \colon f_i \neq \perp\}$.
2. For each $i \in \mathcal{I}$: Sample $(r_i^1, r_i^2) \xleftarrow{R} \{0,1\}^{t^1} \times \{0,1\}^{t^2}$, for t^1, t^2 being the domains sizes of f_i (see Functionality 5.10).

   Return $\left\{t_i^1\right\}_{i\in\mathcal{I}}$ to S^* as the output of the $\{\mathsf{Wmult}_{\mathsf{n_i}}\}_{i\in\mathcal{I}}$ calls.
3. For each each $i \in [t]$:

   If $i \notin \mathcal{I}$, call $\widehat{\mathsf{Wmult}}_{\mathsf{n_i}}$ with input $(a'_i, 0)$. Set $o_i \leftarrow -o_i^{\mathsf{S}}$.

   Else, call $\widehat{\mathsf{Wmult}}_{\mathsf{n_i}}$ with input $(0,1)$. Let x'_i be the returned output. Set $o_i \leftarrow f_i(x'_i; t_i^1, t_i^2)$.

   Let $o \leftarrow \mathsf{crt}((o_i, n_i)_{i\in[t]})$.
4. Continue the interaction with R till its end by forwarding the messages from/to S^* and R, with the following adjustments:
   (a) In Step 6: replace the value o_r^{S} sent by S^*, with $o_r^{\mathsf{S}} - o \bmod r$.
   (b) In Step 7c: replace the value o_p^{R} opened with Com by $o_p^{\mathsf{R}} + o \bmod p$.
   (c) In Step 8b: replace the value δ_b sent by S^* with $\delta_b - o \bmod q$.

Claim 5.31 ($\{\mathsf{Wmult}_{\mathsf{n_i}}\}$ to $\{\widehat{\mathsf{Wmult}}_{\mathsf{n_i}}\}$)**.** *The simulation induced by $(\widehat{\mathsf{S}}^*, \widehat{\mathsf{R}})$ in the $\{\{\widehat{\mathsf{Wmult}}_{\mathsf{n_i}}\}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model perfectly emulates the simulation of $(\mathsf{S}^*, \mathsf{R})$ in the $\{\{\mathsf{Wmult}_{\mathsf{n_i}}\}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model.*

Proof sketch. Fix the input and randomness of the honest R. After Step 3, the calls to $\{\mathsf{Wmult}_{\mathsf{n_i}}\}$ and $\{\widehat{\mathsf{Wmult}}_{\mathsf{n_i}}\}$, the difference between the real and simulated executions is that the value of o^{R} computed by R in the real execution is larger by o from its value in the simulated execution. Hence, the fixes taken in Step 4 make sure that S^*'s view and R's output in the real and emulated execution are the same. □

Fix a malicious, without loss of generality deterministic, $\widehat{\mathsf{S}}^*$ for $\widehat{\Pi}$, and consider the following malicious sender $\widetilde{\mathsf{S}}^*$ for interacting with the honest receiver in $\widetilde{\Pi}$.

Algorithm 5.32 ($\widetilde{\mathsf{S}}^*$)**.**

Oracles: $\widetilde{\mathsf{Mult}_\mathsf{n}}$,Cmt&ModRvl,Com.

Operation: Interact with the $\widetilde{\mathsf{R}}$ in $\widetilde{\Pi}$ as follows:

1. Emulate the execution of $\widehat{\mathsf{S}}^*$ until Step 3. Let $\{(a'_i, \mathsf{leak}_i)\}_{i\in[t]}$ be the inputs it provided to the $\{\widehat{\mathsf{Wmult}_{\mathsf{n}_i}}\}$ calls.
2. Let $a' \leftarrow \mathsf{crt}((a'_i, n_i)_{i\in[t]})$ and $\mathcal{I} \leftarrow \{i \in [t] \colon \mathsf{leak}_i = 1\}$.
3. Call $\widetilde{\mathsf{Mult}_\mathsf{n}}$ with input $(a', \mathcal{I})$. Let $(\cdot, \mathcal{X} = \{x'_i \colon i \in \mathcal{I}\}, \cdot)$ be the returned output.
   If the call aborts:
   (a) Continue the emulation until Step 7 by providing uniform values to the outputs of the $\widehat{\mathsf{Wmult}_{\mathsf{n}_i}}$'s calls and the message o^{S}_r sent by R.
   (b) In Step 7, send an abort message on behalf of R, and halt.
4. For each $i \in \mathcal{I}$, forward x'_i to $\widehat{\mathsf{S}}^*$ as the output of $\widehat{\mathsf{Wmult}_{\mathsf{n}_i}}$.
5. Continue the interaction with the receiver till its end by forwarding its messages to S^* and sending $\widehat{\mathsf{S}}^*$'s responses back to it.

The heart of the proof of Claim 5.27 is in the following lemma, proved in Section 6.

Definition 5.33 (Restatement of Definition 4.23)**.** *For $n, \ell, s \in \mathbb{N}$, let $\mathcal{X}_{n,s,\ell}$ be defined as*

$$\{x \in \mathbb{N} \colon \exists c \in \mathbb{Z}, d \in [\ell] \;\; s.t. \;\; |c| \leq dn \cdot \ell/s \;\wedge\; d \cdot x = c \bmod n\}.$$

Lemma 5.34 (Unpredictability of modular multiplication, the with leakage case)**.** *Let $\ell, n, s, u \in \mathbb{N}$, be such that $u \,|\, n$. Let $x \in \mathbb{N}$ be such that $(x \bmod n/u) \notin \mathcal{X}_{n/u,s/u,\ell}$, let $p \in \mathcal{P} \cap (2\ell, \infty)$ be such that $p \not| \, n$ and $\lfloor s/u \rfloor / p > 16 \cdot \ell^2$, and let $V \xleftarrow{R} [s]$. Then for any $e \in (p-1)$:*

$$\mathrm{P}_\infty(e \cdot V - (x \cdot V \bmod n) \bmod p \mid V \bmod u) \leq 210 \cdot \ell^{-1} + 6 \cdot \frac{p^2}{\lfloor s/u \rfloor} \;\; .$$

Proof of Claim 5.27. By Claim 5.31, we need to prove that the real execution: $\widehat{\mathsf{S}}^*$ interacting in $\widehat{\Pi}$ (in $\{\{\widehat{\mathsf{Wmult}_{\mathsf{n}_i}}\}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model) is close to the simulated execution: $\widetilde{\mathsf{S}}^*$ interacting in $\widetilde{\Pi}$ (in the $\{\widetilde{\mathsf{Mult}_\mathsf{n}}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model). Fix $\widehat{\mathsf{S}}^*$'s coins in the real and simulated executions to the same value. Let $(\{a_i, \mathsf{leak}_i\})$ be the inputs provided by $\widehat{\mathsf{S}}^*$ to the $\{\widehat{\mathsf{Wmult}_{\mathsf{n}_i}}\}$ calls, as determined by its coins, and let a' and $\mathcal{I}$ be as computed by $\widetilde{\mathsf{S}}^*$. By the assumption on $\widehat{\mathsf{S}}^*$, it holds that $|\mathcal{I}| \leq \mathsf{unpMul}$. Let $u \leftarrow \prod_{i\in\mathcal{I}} n_i$, $n' \leftarrow n/u$, and $a'' \leftarrow a' \bmod n'$. Since the call to $\widetilde{\mathsf{Mult}_\mathsf{n}}$ done in $\widetilde{\Pi}$ aborts iff $a'' \notin \mathcal{X}_{n',s_x/u,r_{d,x}}$, it is easy to verify that the two executions, including the receiver's output, are identically distributed if $a'' \in \mathcal{X}_{n',s_x/u,r_{d,x}}$.

We conclude the proof showing that if $a'' \notin \mathcal{X}_{n',s_x/u,r_{d,x}}$, then $\widehat{\mathsf{R}}$ aborts in Step 7 with all but negligible probability. Let $X' \xleftarrow{\mathrm{R}} (s_x)$ and let $Z \leftarrow a' \cdot X' \bmod n$. We note the $\widehat{\mathsf{R}}$'s test in Step 7a passes iff $\widehat{\mathsf{S}}^*$ sends (a'_r, o^{S}_r) such that $Z + o^{\mathsf{S}}_r - a'_r \cdot X' \in \{0, n\} \bmod r$. Since the only information about X' leaked to $\widehat{\mathsf{S}}^*$ before the test is $\{X' \bmod n_i\}_{i\in\mathcal{I}}$, to conclude the proof, we need to bound $\alpha \leftarrow \mathrm{P}_\infty(a'_r \cdot X' - Z \bmod r \mid X' \bmod u)$. By Lemma 5.34, letting $V \leftarrow X'$, $p \leftarrow r$, and $s \leftarrow s_x$, we get that

$$\alpha \leq 5 \cdot \left(42 \cdot 2^{-r_{d,x}} + \frac{r^2}{\lfloor s_x/u \rfloor}\right) + \frac{r}{\lfloor s_x/u \rfloor} \leq 2^{-\kappa_s-4} + 2^{-\kappa_s-4} \leq 2^{-\kappa_s-3}.$$

Hence, the test passes with probability at most $2 \cdot \alpha \leq 2^{-\kappa_s-2}$, concluding the proof. □

Many Wmult corruptions. Recall that $\mathcal{I} \subseteq [t]$ is the indices of the calls to Wmult in which the malicious sender provides unpredictable functions.

Claim 5.35 (Many corruptions)**.** *In a random execution of Protocol 5.14 between a malicious sender and the honest receiver in which* $|\mathcal{I}| \geq \mathsf{unpMul}$*, the honest receiver aborts with probability at least* $1 - 2^{-\kappa_s-2}$*.*

Proof. The following random variables are defined with respect to random execution of Protocol 5.14 between S^* and R. Let $X' \in (s_x)$ denote the value of x' sampled by R in Step 2. For each $i \in [t]$, let $X'_i \leftarrow X' \bmod n_i$ and let O_i^{R} be the output R received from the call to $\mathsf{Wmult}_{\mathsf{n_i}}$. For each $i \in \mathcal{I}$, let f_i be the unpredictable function S^* provides for the $\mathsf{Wmult}_{\mathsf{n_i}}$ call, and let $\{R_i = (R_i^1, R_i^2)\}$ be the randomness sampled by the functionality in this call. Hence,

$$O_i^{\mathsf{R}} = f_i(X'_i; R_i^1, R_i^2)$$

Let a be the value S^* sent to Cmt&ModRvl. Since each f_i is an unpredictable function, for each $i \in \mathcal{I}$:

$$\mathrm{P}_\infty(O_i^{\mathsf{R}} - a \cdot X'_i \bmod n_i \mid R_i^1) \leq 1/2 + \mu_{n_i} \tag{9}$$

Let $u \leftarrow \prod_{i\in\mathcal{I}} n_i$. Since X' is uniform over s_x, it holds that $X' \bmod u$ is $u/s_x < 2^{-\kappa_s-3}$ close to be uniform over $\mathbb{Z}_u$. In the following, we assume it is truly uniform, and we will account for this minor variation later. Under this assumption, the elements of $\{R_i, X'_i\}_{i\in\mathcal{I}'}$ are independent, and thus

$$\mathrm{P}_\infty(\{O_i^{\mathsf{R}} - a \cdot X' \bmod n_i\}_{i\in\mathcal{I}} \mid \{R_i^1\}_{i\in\mathcal{I}'}) \leq \prod_{i\in\mathcal{I}'} (1/2 + \mu_{n_i}) \leq 2^{-\kappa-4} \tag{10}$$

Let O^{R} be the value o^{R} computed by R in Step 3 (i.e., $O^{\mathsf{R}} \leftarrow \mathsf{crt}(\{O_i^{\mathsf{R}}\}_{i\in[t]}, \{n_i\}_{i\in[t]})$) and let $Z \leftarrow O^{\mathsf{R}} - a \cdot X'$. Since crt is an injective function of $\{O_i^{\mathsf{R}}\}_{i\in[t]}$, by the above $\mathrm{P}_\infty(Z \bmod n \mid R^1) \leq 2^{-\kappa_s-4}$.Thus,

$$\mathrm{P}_\infty(Z \mid R^1) \leq 2^{-\kappa_s-4} \tag{11}$$

Since $O^{\mathsf{R}} \in (n)$ and $a \cdot X' \in [\mathsf{s} \cdot n]$, there are at most $2\log(\mathsf{s} \cdot n)$ primes that divided two different elements in $\mathrm{Supp}(Z)$. Let $\mathcal{H} := \{h_r : r \in \mathcal{P}_{s_r}\}$ for $h_r(z) := z \bmod r$. By Proposition 3.5, $|\mathcal{P}_{s_r}| \geq 2^{s_r}/(5s_r)$. Hence, for any $z \neq z' \in \mathrm{Supp}(Z)$:

$$\Pr_{h \xleftarrow{R} \mathcal{H}}[h(z) = h(z')] \leq 5\log(\mathsf{s} \cdot n)/2^{s_r} < 2^{-2(\kappa_s+4)} \tag{12}$$

Let $H \xleftarrow{R} \mathcal{H}$. By Equations (11) and (12) and Corollary 5.6,

$$\varepsilon \leftarrow \mathrm{P}_\infty(H(Z) \mid H, R^1) \leq 2^{-\kappa_s-4} + 2^{-\kappa_s-4} = 2^{-\kappa_s-3} \tag{13}$$

Considering also the statistical distance of $X' \bmod u$ from uniform, which we omitted above, we get that $\varepsilon \leq 2^{-\kappa_s-2}$. This concludes the proof since, by construction, R aborts with probability at least $1 - \varepsilon$. □

Putting it together.

Proof of Claim 5.20. Follows by combining Claims 5.23, 5.27 and 5.35. □

5.3.3 Using Leaky Commit&Modular-Reveal

In this part, we address the security of the variant of Protocol 5.14 in which the commit&modular-reveal functionality Cmt&ModRvl is replaced by its leaky variant $\widetilde{\text{Cmt\&ModRvl}}$. When proving the security of this variant, we have to address the additional concern that in the $\widetilde{\text{Cmt\&ModRvl}}$ call, a malicious receiver can obtain $a' \bmod \ell$ for $\ell \in [2^{-\kappa_s}]$. This additional ability is easily addressed by multiplying s_a (and thus n) in the parameters of the protocol by 2^{κ_s}. The security proof of Protocol 5.14, which refers to Claim 4.18, already handles modular leakage about a'. It is easy to see that the above increase in parameters takes care of the additional leakage for the proof to go through.

5.3.4 Round Reduction

Consider the following variant of Protocol 5.14:

1. Rather the sending (x'_p, o^{R}_p) in Step 5, R reveals x'_p right in the beginning, and S sends the conditional disclosure of (δ_a, δ_b) in parallel to Step 4.

   To maintain the security of the protocol, the size of s_x, and thus n, is increased by a factor of 2^{s_r}.

2. R sends the conditional disclosure of δ_x right after Step 4.

The resulting protocol has four rounds, and thus, using a non-interactive Cmt&ModRvl and the two-round implementation of Wmult, the number of rounds grows to five.

5.3.5 OLE Without Commit&Prove?

Protocol 5.14 uses the commit&prove functionality whose implementation is rather costly or requires additional hardness assumptions. In this section, we prove that, assuming a plausible number-theoretic conjecture, the very same protocol remains secure without using the commit-and-prove functionality.

For two vectors $x, y \in \{0,1\}^t$, let $\mathrm{Ham}(x,y)$ denote their relative Hamming distance, i.e., $\mathrm{Ham}(x,y) := |\{i \in [t] \colon x_i \neq y_i\}|/t$. For a randomized function f, randomness r and a set $\mathcal{S}$, let $f(\mathcal{S}; r)$ denote the ordered vector $(f(x;r))_{x \in \mathcal{S}}$ (the enumeration is with respect to some fixed order of $\mathcal{S}$), and let $f_p(x;r) := f(x;r) \bmod p$. Finally, for $a, b \in \mathbb{N}$ let $g^{a,b}(x) := ax + b$. The following quantities measure how far a function is from being linear.

Definition 5.36 (Distance from linear functions)**.** *For $\kappa, \ell \in \mathbb{N}$, $\mathcal{S} \subseteq (n)$ and randomized function $f \colon (n) \mapsto (n)$, let $\alpha := \min_{a,b}\{\mathrm{E}_r[\mathrm{Ham}(f(\mathcal{S};r), g^{a,b}(\mathcal{S}))]\}$, and let $\widehat{\alpha}_\kappa := \mathrm{E}_{p \xleftarrow{R} \mathcal{P}_\kappa}[\min_{a,b}\{\mathrm{E}_r[\mathrm{Ham}(f_p(\mathcal{S};r), g^{a,b}_p(\mathcal{S}))]\}]$. Finally, let $\delta_{\kappa,\ell} := \max_{\mathcal{S},f}\{\widehat{\alpha}_\kappa - \alpha\}$.*

Namely, α measures how far f is, in expectation, from being a linear function over $\mathcal{S}$, where $\widehat{\alpha}_\kappa$ measures how far it is from being a linear function modulo a uniform κ-bit prime p, and $\delta_{\kappa,n}$ is the distance between the two (on worst case $f, \mathcal{S}$). The following conjecture states that the distance is exponentially small (in κ).

Conjecture 5.37 (Distance from lines is resilient to modulo)**.** *There exist $c_\delta, c_n > 0$ such that $\delta_{\kappa,n} \leq 2^{-c_\delta \cdot \kappa}$ for all $\kappa \in \mathbb{N}$ and $n \leq 2^{c_n \cdot \kappa}$.*

We note that the deterministic variant of Conjecture 5.37, where the max in the definition of $\delta_{\kappa,\ell}$ is only taken over deterministic f, which might be easier to prove, is also useful (see Remark 5.40).

Assuming Conjecture 5.37 holds, one can prove the security of the variant Protocol 5.14 without the integer commitment parts is secure.

Protocol 5.38 (Π': OLE w/o commit&prove)**.** Protocol Π' is the following variant of Protocol 5.14:

1. Step 4 and Step 6 are removed.
2. In Step 5, R sends $r \xleftarrow{R} (\mathcal{P}_{s_r} \setminus \{q\})$.

3. In Step 6, S sends $a'_r \leftarrow a \bmod r$.

Namely, protocol Π' does not pay for the communication and additional round incurred by the implementation of Cmt&ModRvl.

Theorem 5.39. *Let $\kappa_\mathsf{s}, q, p, \{n_i\}_{i\in[t]}, n$ be as in Theorem 5.17. Then Protocol 5.38 with parameters $(1^{\kappa_\mathsf{s}}, q, p, \{n_i\}_{i=1}^t)$ UC-realizes OLE_q in the $\{\{\mathsf{Wmult}_{\mathsf{n}_\mathsf{i}}\}_{i\in[t]}\}$-hybrid model, with statistical security $2^{-\kappa_\mathsf{s}} + \delta_{2\kappa_\mathsf{s},n}$.*

Namely, the protocol is secure if $\delta_{2\kappa_\mathsf{s},n}$ is small Specifically, if Conjecture 5.37 holds for $c_\delta = 1/2$ and $c_n = \log n/\kappa_\mathsf{s} \leq 4\log q + 25\kappa_\mathsf{s}$, see Appendix A for the upper-bound on n, protocol Π' is as secure as Protocol 5.14 (ignoring $2^{-\kappa_\mathsf{s}}$ difference).

Proof. The only part we should take care of is reproving Claim 5.35. Since in the modified protocol there is no committed $a \in \mathbb{Z}_n$, we let $a \leftarrow \mathsf{crt}_{\{n_i\}}(\{a_r\})$, for a_r being the value of a_r sent in Step 6 (setting $a_r \leftarrow 0$ if no respond is sent). Note that with respect to such large a, Equation (12) is no longer guaranteed to hold (and thus we cannot use Corollary 5.6 to deduce Equation (13)). Rather, in the following we prove that the following variant of Equation (13) does hold

$$\mathrm{P}_\infty(H(Z) \mid H, R^1) \leq 2^{-\kappa_\mathsf{s}} + \delta_{2\kappa_\mathsf{s},n}, \tag{14}$$

which concludes the proof. To prove Equation (14), we note that the argument used to prove Equation (11) actually yields

$$\mathrm{P}_\infty(O^\mathsf{R} - a \cdot X' \mid R^1) \leq 2^{-\kappa_\mathsf{s}-4} \tag{15}$$

for *any* $a \in \mathbb{N}$ (and not only for the committed a).

Let $f_{r_1^1,\ldots,r_t^1}(x; \{r_i^2\}_{i\in[t]}) := \mathsf{crt}(\{f_i(x \bmod n_i; (r_i^1, r_i^2)\}_{i\in[t]}, \{n_i\}_{i\in[t]})$, where f_i is the unpredictable functions provided by the cheating sender to the calls to $\mathsf{Wmult}_{\mathsf{n}_\mathsf{i}}$ (set to the 0 function if no such function is provided). By Equation (15),

$$\mathrm{P}_\infty(f_{R^1}(X'; R^2) - a \cdot X' \mid R^1) \leq 2^{-\kappa_\mathsf{s}-4}$$

for $R^2 \xleftarrow{\mathrm{R}} (\{0,1\}^{t_2})^t$. The above is equivalent to

$$\mathrm{E}_{r_1}\left[\min_{a,b}\left\{\mathrm{E}_{r_2}\left[\mathrm{Ham}(f_{r_1}(\mathcal{S}; r_2), g^{a,b}(\mathcal{S})\right]\right\}\right] \geq 2^{-\kappa_\mathsf{s}-4} \tag{16}$$

for any a, b, and thus by the definition of $\delta_{2\kappa_\mathsf{s},n}$:

$$\mathrm{E}_{r_1}\left[\mathrm{E}_{p\xleftarrow{\mathrm{R}}\mathcal{P}_{2\kappa}}\left[\min_{a,b}\left\{\mathrm{E}_{r_2}\left[\mathrm{Ham}(f_{r_1}(\mathcal{S}; r_2), h^{a,b}(\mathcal{S})\right]\right\}\right]\right] \geq 2^{-\kappa_\mathsf{s}-4} + \delta_{2\kappa_\mathsf{s},n} \tag{17}$$

We conclude that

$$\mathrm{P}_\infty(f_{R^1}(X'; R^2) - a \cdot X' \bmod P \mid P, R^1) \leq 2^{-\kappa_\mathsf{s}-4} + \delta_{2\kappa_\mathsf{s},n}$$

for $P \xleftarrow{\mathrm{R}} \mathcal{P}_{2\kappa}$, and thus $\mathrm{P}_\infty(H(Z) \mid H, R^1) \leq 2^{-\kappa_\mathsf{s}-4} + \delta_{2\kappa_\mathsf{s},n}$. □

Remark 5.40 (Using the deterministic variant of Conjecture 5.37). The deterministic variant of Conjecture 5.37, where the max in the definition of $\delta_{\kappa,\ell}$ is only taken over deterministic f, also yields a protocol of interest. While using it yields a slightly less efficient variant of Protocol 5.38, more "small primes" will be needed; this variant is still efficient than Protocol 5.14 for not too large q's.

Further improvements. An even more drastic improvement in the protocol's effectiveness, regardless of whether Conjecture 5.37 holds or not, might come from a better analysis of cheating calls to Wmult. We proved that if S^* cheats in a call, it adds a bit of min-entropy to the R's output (which is rather tight for our implementation). If many cheats occur, we abort the simulation. But when only a few such cheats happened, we leak the R's input in this call to S^*. Namely, we lose $\log\log q$ bits versus gaining a single bit. The reason is that we only know how to analyze a leakage of *all* of R's input to a call (using CRT), and we do not know how to gain further from the fact that, say, only the first bit of R's input to this call has leaked.

5.4 Vector OLE

In this section, we show how to modify Protocol 5.14 to realize the vector OLE (VOLE) functionality. For technical reasons, the performance of the resulting protocol is not optimal: for an m-size vector, it takes more than m times the cost of Protocol 5.14 (communication-wise). Yet, for small m, the resulting protocol outperforms the generic reduction presented in Appendix B (that costs $m+1$ times the cost of a single OLE). See Appendix A.2.

The VOLE protocol uses the natural generalization of the weak multiplication functionality Wmult to the vector case.

Functionality 5.41 ($\mathsf{vWmult}_{m,p}$: vector weak multiplication)**.**

Parties: S and R.

Parameters: $m, n \in \mathbb{N}$.

S's input: $\mathbf{a} \in \mathbb{Z}_n^m$.

R's input: $x \in \mathbb{Z}_n$.

Corrupt party's input: Corrupt S can provide a polynomial-verifiable functions $f_1, \ldots, f_m$, represented as a circuits.

Operation: For each $i \in [m]$: act according to $\mathsf{Wmult}_{\mathsf{n}}((\mathbf{a}_i, f_i), x)$.

Implementation of $\mathsf{vWmult}_{m,n}$ for prime p, can be done using the variant Protocol 5.11 in which Step 2 is replaced with

- For all $i \in (\ell - 1)$ (in parallel):
  1. S: Sample $\delta_i \xleftarrow{R} \mathbb{Z}_p^m$.
  2. The parties jointly call $\mathsf{OT}_{\mathsf{m \cdot n}}((\delta_i, \delta_i + \mathbf{a} \bmod p), \lambda_i)$.[22] Let $\mathbf{z}_i$ denote R's output.

and the rest of the protocol is adjusted accordingly. Given $\{\mathsf{vWmult}_{m,n_i}\}$, the $\mathsf{VOLE}_{\mathsf{q}}$ protocol is defined as follows:

Protocol 5.42 (Fully secure $\mathsf{VOLE}_{\mathsf{q}}$)**.**

Parties: S and R.

Parameters: $1^{\kappa_{\mathsf{s}}}$, 1^m, $q, p, \mathsf{s} \in \mathbb{N}$ and set $\{n_i\}_{i\in[t]} \subset \mathbb{N}$.

Derived parameters: Same as in Protocol 5.14 (as function of the parameters). Let $vs_x \leftarrow s_x \cdot (210)^m$ and $vs_a \leftarrow s_a + \lceil \log m \rceil$.

Oracles: $\{\mathsf{vWmult}_{m,n_i}\}_{i\in[t]}$, Cmt&ModRvl, Com.

S's input: $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_q^m$.

R's input: $x \in \mathbb{Z}_q$.

[22] We abuse notation and view $\mathsf{OT}_{\mathsf{m \cdot n}}$ as an OT for strings in $\mathbb{Z}_p^m$, and not for strings in $\mathbb{Z}_{pm}$. The cost of these two variants, however, is essentially the same.

Operation: The parties interact in m parallel and independent executions of Protocol 5.14 according to the inputs, i.e., in the i^{th} execution: R is using input x, and S is using $(\mathbf{a}_i, \mathbf{b}_i)$, but with the following exceptions.

1. In all executions, R is using the same value of $x' \xleftarrow{R} (vs_x)$.
2. In the i^{th} execution, S samples (an independent) $a'_i \xleftarrow{R} [vs_a]$.
3. Instead of m independent executions of Step 3, the parties do as follows: let $\mathbf{a}' = (a'_1, \ldots, a'_t)$. For all $i \in [t]$ (in parallel): the parties jointly call vWmult_{m,n_i}, with S using input $\mathbf{a}'$ mod n_i and R using input x' mod n_i. Let $\mathbf{o}_i^{\mathsf{P}}$ denote P's output (for $\mathsf{P} \in \{\mathsf{S}, \mathsf{R}\}$).

   Each $\mathsf{P} \in \{\mathsf{S}, \mathsf{R}\}$: for each $j \in [m]$, set $o_j^{\mathsf{P}} \leftarrow \mathsf{crt}((\mathbf{o}_{i,j}^{\mathsf{P}}, n_i)_{i \in [m]})$ (o_j^{P} takes the role of o in the j^{th} execution).

Theorem 5.43. *Let $m, \kappa_{\mathsf{s}}, q, p, \{n_i\}_{i\in[t]}, n$ be as in Theorem 5.17. Assume $q, p, \{n_i\}$ are relatively prime, $p \geq 2^{\kappa_{\mathsf{s}}+3}$, and $n \geq 2^{\kappa_{\mathsf{s}}+3} \cdot s_x \cdot s_a$. Then Protocol 5.42 with parameters $(1^{\kappa_{\mathsf{s}}}, 1^m, q, p, \{n_i\})$ UC-realizes $\mathsf{VOLE}_{\mathsf{q}}$ in the $\{\{\mathsf{vWmult}_{m,n_i}\}_{i\in[t]}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model, with statistical security $2^{-\kappa_{\mathsf{s}}}$.*

Proof. Correctness follows immediately from that of Protocol 5.14. Security against malicious receivers follows rather immediately from that of Protocol 5.14: since S uses independent a' for each of the Protocol 5.14 invocations, the simulation against malicious receivers straightforwardly combines the simulation against malicious receivers done in Theorem 5.17 (the theorem stating the security of Protocol 5.14). The $\log m$ increase in the domain of a' absorbs the union bound over the m invocations of Theorem 5.17. Security against malicious senders, however, has to be significantly adjusted. Since R uses the *same* x' in all Protocol 5.14 invocations, a malicious sender has m attacking opportunities against the *same* value of x', whose combined effect has to be carefully analyzed, a task that we do below.

As in the proof of Theorem 5.17, we still categorize malicious senders according to their number Wmult corruptions. But we count the overall number of $i \in [t]$ for which the sender has corrupted a call to Wmult. That is, we let $\mathcal{I} = \{i \in [t]$: the sender corrupts a call to $\mathsf{Wmult}_{\mathsf{n}_i}$ in any of the m executions of Protocol 5.14$\}$. Very similar lines to those used in the proof of Claim 5.35 yield security against malicious senders with $|\mathcal{I}| > \mathsf{unpMul}$. The more challenging part, and the one that causes the slight increment in the size x', is the case that $|\mathcal{I}| \leq \mathsf{unpMul}$. While most of the proof follows the very same line as in the proof of Theorem 5.17, the proof of the following claim, which is the analogue of Claim 5.23, needs to be adjusted.

Definition 5.44 (Protocol $\widetilde{\Pi}$)**.** *Let $\widetilde{\Pi} = (\widetilde{\mathsf{S}}, \widetilde{\mathsf{R}})$ be the variant of Protocol 5.42 in which the calls to $\{\mathsf{Wmult}_{\mathsf{n}_i}\}_{i\in[t]}$ in the j^{th} execution of Protocol 5.14 are replaced by a single call to $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$; $\widetilde{\mathsf{S}}$ using input a'_j and $\widetilde{\mathsf{R}}$ using input x'.*

Claim 5.45. *For any malicious sender $\widetilde{\mathsf{S}}^*$ for $\widetilde{\Pi}$ in the $\{\widetilde{\mathsf{Mult}}_{\mathsf{n}}, \mathsf{Cmt\&ModRvl}, \mathsf{Com}\}$-hybrid model, there exists a malicious receiver Sim in the $\mathsf{VOLE}_{\mathsf{q}}$-hybrid model, such that the statistical distance between the real and simulated executions is at most $2^{-\kappa_{\mathsf{s}}-2}$.*

In the proof of Protocol 5.14, we showed that the call to $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$ does not leak too much information about x', and thus the execution of $\widetilde{\Pi}$ can be emulated in the $\mathsf{OLE}_{\mathsf{q}}$-hybrid model. This is no longer true, however, when considering the leakage on x' induced by the m calls to $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$. Yet, we prove that if the m-calls to $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$ leak too much information about x', then with high probability the receiver aborts in Step 7a, of one of the m executions, and thus the execution can still be simulated by instructing the simulator to abort in this case.

Proof of Claim 5.45. We assume without loss of generality that in all of the m calls to $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$ the sender $\widetilde{\mathsf{S}}^*$ uses the same set $\mathcal{I}$. Let n' be as computed by $\widetilde{\mathsf{Mult}}_{\mathsf{n}}$ and let $u \leftarrow n/n'$. For notation convenient, we assume that rather than outputting $(x_{d'_i}, \mathcal{X}, x_M)$ in the i^{th} call, it outputs $(x_{d_i} \leftarrow x \bmod d_i, x_M, d_i, c_i)$ for $(c_i, d_i) \leftarrow u \cdot (c'_i, d'_i)$ ((c'_i, d'_i) are the values it computed in this call). It is easy to verify that the original

output can be computed from the new one. The simulator for $\widetilde{\mathsf{S}}^*$ given below follows the very same lines as those used in the proof of Theorem 5.17.

Algorithm 5.46 (Sim)**.**

Oracle: $\mathsf{VOLE}_{\mathsf{q}}$.

Operation:

1. Emulate a random execution of $(\widetilde{\mathsf{S}}^*, \mathsf{R}(0))$ till its end. Abort if the emulation does.
   (a) Let $(a'_i, \cdot)$ be the input provided by $\widetilde{\mathsf{S}}^*$ to the i^{th} $\widetilde{\mathsf{Mult}_{\mathsf{n}}}$-call.
   (b) Let $(x'_{d_i}, x'_M, c_i, d_i)$ be the output $\widetilde{\mathsf{S}}^*$ got back from this call.
   (c) Let $\delta_x, \left\{\delta_{a,i}, \delta_{b,i}, o^{\mathsf{S}}_{r_i,i}\right\}_{i\in[m]}$ be the values R and $\widetilde{\mathsf{S}}^*$ sent in the m executions of Protocol 5.14, and let $(r_i, a_{r,i})$ be the output of the m calls to Cmt&ModRvl done in the simulated execution.
2. For each $i \in [m]$: compute (a_i, b_i) as a function of $(x'_{d_i}, x'_M, c_i, d_i, \delta_{a,i}, \delta_{b,i})$ as done in Algorithm 5.24 Step 3.
3. Call $\mathsf{VOLE}_{\mathsf{q}}$ with input $(\{a_i\}_{i\in[m]}, \{b_i\}_{i\in[m]})$.

We complete the proof by showing that if $\prod d'_i \le \prod d'_i \le (210)^m \cdot r_{d_x}$, then the real and simulated executions are statistically close, and otherwise, both executions abort with very high probability.

Case $\prod d'_i \le (210)^m \cdot r_{d_x}$. This case follows very similar lines to the proof of Claim 5.23. Note that the information that leaked to $\widetilde{\mathsf{S}}^*$ about X' by the $\widetilde{\mathsf{Mult}_{\mathsf{n}}}$-calls can be computed from $X' \bmod d$ for $d \leftarrow u \cdot \prod d'_i \le \mathsf{leak} \cdot r_{d_x} \cdot (210)^m$ (compare to $X' \bmod d$, for $d \le \mathsf{leak} \cdot r_{d,x}$, in proof of Claim 5.23). Since $vs_x = (210)^m s_x$, it still holds that the value of $(x'_d, x'_M, x'_d, \delta_x)$ in the real and simulated executions are $2^{-\kappa_{\mathsf{s}}-3}$ close. The proof is concluded by the following variant of Claim 5.26.

Claim 5.47. *There exists a set $\mathcal{B}ad$ such that the following holds:*

1. *For every $x' \in (vs_x)$ with $(\{x'_{d_i}\}, x'_M) \notin \mathcal{B}ad$, and every $i \in [m]$:*

$$\left\lfloor (c_i \cdot (x' - x'_{d_i})/d_i + a'_i x'_{d_i})/n \right\rfloor = \left\lfloor (c_i (x'_M - x'_{d_i})/d_i + a'_i x'_{d_i})/n \right\rfloor$$

   .

2. $\Pr_{X' \xleftarrow{R} (vs_x)}\left[(\{X'_{d'_i}\}, X'_M) \in \mathcal{B}ad\right] \le 2^{-\kappa_{\mathsf{s}}-3}$.

The proof of Claim 5.47 follows by a union bound from Claim 5.26, where we do not have to pay a factor of m since the domain of x' in the former is increased.

Case $\prod d'_i \le (210)^m \cdot r_{d_x}$. The very same line of the proof of Claim 5.27 yields that in the i^{th} execution of Protocol 5.14, R accepts in the check done in Step 7a, with probability at most

$$\alpha_i \le 5 \cdot \left(\frac{42}{d'_i} + \frac{r_i^2}{\lfloor vs_x/u \rfloor}\right) + \frac{r_i}{\lfloor vs_x/u \rfloor} \le \frac{210}{d'_i} + 2^{-\kappa_{\mathsf{s}}-4}/(210^m)$$

Since the above hold also for unbounded $\widetilde{\mathsf{S}}^*$, the probability of R accepting in all m checks decreases multiplicatively, and thus it is at most

$$\prod \alpha_i \le 2 \cdot 210^m \prod p_i^{-1} \le 2/r_{d,x} \le 2^{-\kappa_{\mathsf{s}}-4}$$

Hence, in this case, both the real and emulated execution abort with probability at least $1 - 2^{-\kappa_{\mathsf{s}}-4}$, and thus they are $2^{-\kappa_{\mathsf{s}}-3}$ close, concluding the proof. □

□

6 Unpredictability of Modular Multiplication

In this section, we prove Lemmas 4.24 and 5.34. Recall the definition of $\mathcal{X}_{n,s,\ell}$, the set of simulateable cheating inputs.

Definition 6.1. *For $n, \ell, s \in \mathbb{N}$, let $\mathcal{X}_{n,s,\ell}$ be defined as*

$$\{x \in \mathbb{N} \colon \exists c \in \mathbb{Z}, d \in [\ell] \;\; s.t. \;\; |c| \leq dn \cdot \ell/s \;\wedge\; d \cdot x = c \bmod n\}.$$

6.1 Additional Preliminaries

We use the following simple facts.

Fact 6.2. *Let U and V be arbitrary random variables. For any $p, w \in \mathbb{N}$, it holds that*

$$\mathrm{P}_\infty(Uw + V \bmod p) \leq p \cdot \mathrm{P}_\infty((U, V) \bmod p).$$

Proof. Let z_0 be such that $\mathrm{P}_\infty(Uw + V \bmod p) = \Pr[Uw + V = z_0 \bmod p]$. Then,

$$\begin{aligned}\Pr[Uw + V = z_0 \bmod p] &= \sum_{u \in \mathbb{Z}_p} \Pr[(Uw + V, U) = (z_0, u) \bmod p] \\ &= \sum_{u} \Pr[(V, U) = (z_0 - wu, u) \bmod p] \\ &\leq p \cdot \mathrm{P}_\infty((U, V) \bmod p).\end{aligned}$$

□

Fact 6.3. *For any $a, d, p \in \mathbb{Z}$ with $\gcd(p, d) = 1$, it holds that $\lfloor a/d \rfloor = (a - [a]_d) \cdot d^{-1} \bmod p$.*

Proof. Let $a = d \cdot b + r$, where b and r denote the quotient and the remainder of a by d. Clearly, $r = [a]_d$, $b = \lfloor a/d \rfloor$, and $a = db + r \bmod p$ implies $\lfloor a/d \rfloor = d^{-1}(a - [a]_d]) \bmod p$, since p and d are coprime. □

Fact 6.4. *Let $u, p \in \mathbb{N}$ and $\mu \in \mathbb{R}^+$. Then for $U \xleftarrow{R} (u)$, it holds that*

$$\mathrm{P}_\infty(\lfloor \mu p U \rfloor \bmod p) \leq \frac{1}{p} + \mu + \frac{1}{u}\left(1 + \frac{1}{\mu p}\right).$$

Proof. Let $\lambda_0 = \lfloor \lfloor \mu p u \rfloor / p \rfloor$. Fix an arbitrary j, and notice that

$$\Pr[\lfloor \mu p U \rfloor = j \bmod p] = \Pr\left[\bigvee_{\lambda=0}^{\lambda_0} \mu p U \in [j + \lambda p, j + 1 + \lambda p)\right] \tag{18}$$

For any interval of size a, it is easy to see that there are at most $\lceil a/\mu p \rceil$ possible values of $Z = \mu p U$. Thus, for any λ, there are at most $\left\lceil \frac{1}{\mu p} \right\rceil$ possible values of $\mu p U$ in the interval $[j + \lambda p, j + 1 + \lambda p)$. Therefore, by Equation (18),

$$\Pr[\lfloor \mu p U \rfloor = j \bmod p] \leq (\lambda_0 + 1)\left\lceil \frac{1}{\mu p} \right\rceil \cdot \frac{1}{u} \tag{19}$$

Since $\lambda_0 \leq u\mu$ and $\left\lceil \frac{1}{\mu p} \right\rceil \leq \frac{1}{\mu \cdot p} + 1$, we deduce that

$$\Pr[\lfloor \mu p U \rfloor = j \bmod p] \leq (u\mu + 1)\left(\frac{1}{\mu p} + 1\right) \cdot \frac{1}{u} = \frac{1}{p} + \mu + \frac{1}{u}\left(1 + \frac{1}{\mu p}\right).$$

□

6.2 Proving Lemma 4.24

Lemma 6.5 (Restatement of Lemma 4.24)**.** *Let $\ell, n, s \in \mathbb{N}$, let $x \in \mathbb{N} \setminus \mathcal{X}_{n,s,\ell}$, let $p \in \mathcal{P} \cap (2\ell, \infty)$ be such that $p \nmid n$ and $s/p > 16\ell^2$, and let $V \xleftarrow{R} [s]$ and $W \xleftarrow{R} \mathbb{Z}_n$. Then,*

$$\mathrm{P}_\infty(W \bmod p \mid x \cdot V + W \bmod n) \leq 42 \cdot \ell^{-1} + \frac{p^2}{s}.$$

We prove Lemma 6.5 using the following lemma, proven in Section 6.2.1.

Lemma 6.6. *Let $\ell, n, s \in \mathbb{N}$, let $x \in \mathbb{Z}_n \setminus \mathcal{X}_{n,s,\ell}$ and let $p \in \mathcal{P} \cap (2\ell, \infty)$ be such that $p \nmid n$ and $s/p > 16 \cdot \ell^2$. Then, for $V \xleftarrow{R} [s]$ it holds that*

$$\mathrm{P}_\infty((V, \lfloor Vx/n \rfloor) \bmod p) \leq 42 \cdot \frac{1}{\ell p} + \frac{p}{s}.$$

Proof of Lemma 6.5. Let $Y \leftarrow xV + W \bmod n$ and note that $Y = xV + W - \lfloor xV/n \rfloor \cdot n$. Thus,

$$\begin{aligned}
\mathrm{P}_\infty(W \bmod p \mid Y) &= \mathrm{P}_\infty(Y + \lfloor xV/n \rfloor \cdot n - xV \bmod p \mid Y) \\
&= \mathrm{P}_\infty(-xV + Y + \lfloor xV/n \rfloor \cdot n \bmod p \mid Y) \\
&= \mathrm{P}_\infty(-xV + \lfloor xV/n \rfloor \cdot n \bmod p) \qquad (20) \\
&= \mathrm{P}_\infty(-n^{-1}xV + \lfloor xV/n \rfloor \bmod p). \qquad (21)
\end{aligned}$$

Equation (20) follows from the fact that V and Y are independent (since W is an independently-sampled uniform value in $\mathbb{Z}_n$). Equation (21) holds since $\gcd(n, p) = 1$. Given Equation (21), Lemma 4.24 follows from Lemma 6.6 and Fact 6.2.

□

6.2.1 Proving Lemma 6.6

Lemma 6.6 easily follows from the following proposition, proven below.

Proposition 6.7. *Let $\ell, n, s, s' \in \mathbb{N}$, let $x \in (n-1) \setminus \mathcal{X}_{n,s,\ell}$, and let $p \in \mathcal{P}$ such that $p \nmid n$. Then for $V \xleftarrow{R} [s']$, it holds that*

$$\mathrm{P}_\infty(\lfloor p \cdot V/n \rfloor \bmod p) \leq 7 \cdot \left(\max\left\{ \frac{1}{\ell}, \left(\frac{1}{p} + \frac{1}{2\ell} + \frac{1}{s'/2\ell - 1} + \frac{1}{\ell p} \cdot \frac{s}{s' - 2\ell} \right) \right\} + \frac{2\ell}{s'} \right).$$

Proof of Lemma 6.6. Let $(V_0, V_1) \xleftarrow{R} ((p-1), (\lfloor s/p \rfloor))$. Observe that $\mathrm{SD}(V, V_0 + pV_1) \leq p/s$, and thus $\mathrm{P}_\infty((V, \lfloor Vx/n \rfloor) \bmod p) \leq \mathrm{P}_\infty((V_0 + pV_1, \lfloor (V_0 + pV_1)x/n \rfloor) \bmod p) + p/s$. It remains to bound $\mathrm{P}_\infty((V_0 + pV_1, \lfloor (V_0 + pV_1)x/n \rfloor) \bmod p)$. Compute

$$\begin{aligned}
\mathrm{P}_\infty((V_0 + pV_1, \lfloor (V_0 + pV_1)x/n \rfloor \bmod p)) &= \mathrm{P}_\infty((V_0, \lfloor (V_0 + pV_1)x/n \rfloor \bmod p)) \qquad (22) \\
&\leq 2 \cdot \mathrm{P}_\infty((V_0, \lfloor (V_0 x/n \rfloor + \lfloor pV_1 x/n \rfloor \bmod p)) \\
&\leq 2 \cdot \mathrm{P}_\infty(\lfloor pV_1 x/n \rfloor \bmod p)/p.
\end{aligned}$$

Applying Proposition 6.7 with $V = V_1$, $s' = \lfloor s/p \rfloor$ and under the constraints that $s/p > 16 \cdot \ell^2$ and $p > 2\ell$, we deduce that

$$\begin{aligned}
&\frac{1}{p} + \frac{1}{2\ell} + \frac{1}{s'/2\ell - 1} + \frac{1}{\ell p} \cdot \frac{s}{s' - 2\ell} + \frac{2\ell}{s'} \\
&\qquad \leq \frac{1}{p} + \frac{1}{2\ell} + 2 \cdot \frac{p \cdot 2\ell}{s} + \frac{3}{2} \cdot \frac{1}{\ell p} \cdot \frac{s}{s/p} + \frac{2p \cdot 2\ell}{s} \\
&\qquad \leq 4 \cdot \frac{p \cdot 2\ell}{s} + \frac{5}{2} \cdot \frac{1}{\ell} \leq \frac{3}{\ell}.
\end{aligned}$$

Thus, $\mathrm{P}_\infty((V_0 + pV_1, \lfloor (V_0 + pV_1)x/n \rfloor \bmod p)) \leq 42 \cdot \ell^{-1}/p$. □

Proving Proposition 6.7.

Proof of Proposition 6.7. By Lemma 3.7 (Thue's lemma), $dx = c \bmod n$ for some $d \in [2\ell-1]$ and $c \in \{-t, t\}$, for $t \leftarrow \lfloor n/2\ell \rfloor$. Fix such c and d. By definition, $d \cdot x - c = \lambda n$ for some $\lambda \in \mathbb{N}$ with $\lambda < d$ and $\gcd(\lambda, d) = 1$. So, $x = \frac{\lambda n + c}{d}$. Therefore,

$$\lfloor pVx/n \rfloor = \lfloor pV(n \cdot \lambda + c)/dn \rfloor = \lfloor p\lambda V/d \rfloor + \lfloor pVc/dn \rfloor \pm 1 \tag{23}$$

Let $V' \leftarrow V_0 + dV_1$ for $(V_0, V_1) \xleftarrow{R} ((d), (\lceil s'/d \rceil))$. It holds that

$$\begin{aligned} \lfloor p\lambda V'/d \rfloor + \lfloor pV'c/dn \rfloor &= \lfloor p\lambda(V_0 + dV_1)/d \rfloor + \lfloor p(V_0 + dV_1)c/dn \rfloor \\ &= \lfloor p\lambda V_0/d \rfloor + \lfloor p\lambda V_1 \rfloor + \lfloor pV_0c/dn \rfloor + \lfloor pcV_1/n \rfloor \pm 2 \\ &= \underbrace{\lfloor p\lambda V_0/d \rfloor + \lfloor pV_0c/dn \rfloor}_{A} + \underbrace{\lfloor pcV_1/n \rfloor}_{B} \pm 2 \bmod p. \end{aligned} \tag{24}$$

Since, by assumption, $x \notin \mathcal{X}_{n,s,\ell}$, at least one of the following holds.

1. $|d| > \ell$ and $c \leq d \cdot \ell \cdot n/s$, or
2. $c > d \cdot \ell \cdot n/s$.

We continue by case analysis depending on which of the above conditions holds, showing that either $A \bmod p$ or $B \bmod p$ are unpredictable, and therefore, since A and B are independent, $A + B \bmod p$ is unpredictable.

Case 1: $|d| > \ell$ and $c \leq d \cdot \ell \cdot n/s$. We prove that A is unpredictable in this case by showing that (1) $\lfloor pV_0c/dn \rfloor = 0$ and (2) $\lfloor p\lambda V_0/d \rfloor$ is unpredictable. Since c is small and $V_0 \leq d$,

$$pV_0c/dn \leq \frac{pd}{dn} \cdot \frac{d \cdot \ell \cdot n}{s} \leq \frac{pd\ell}{s} < 1$$

and thus $\lfloor pV_0c/dn \rfloor = 0$. By Fact 6.3, $\lfloor p\lambda V_0/d \rfloor = (p\lambda V_0 - [p\lambda V_0]_d) \cdot d^{-1} = [p\lambda V_0]_d \cdot d^{-1} \bmod p$. Since p and λ are relatively prime to d, it holds that $[\lambda pV_0]_d$ is a uniform element in $[d]$. Thus, $\mathrm{P}_\infty(\lfloor p\lambda V_0/d \rfloor \bmod p) \leq \frac{1}{d} \leq \frac{1}{\ell}$. It follows that

$$\alpha_1 := \mathrm{P}_\infty(A \bmod p) \leq \frac{1}{\ell} \tag{25}$$

Case 2: $c > d \cdot \ell \cdot n/s$. We prove that $B \bmod p$ is unpredictable in this case. Applying Fact 6.4 with $U = V_1$, $u = \lfloor s'/d \rfloor$, and $\mu = c/n$, yields that

$$\mathrm{P}_\infty(B \bmod p) \leq \frac{1}{p} + \mu + \frac{1}{u}\left(1 + \frac{1}{\mu p}\right) \tag{26}$$

Since in this regime of parameters $\mu \in (\ell/\lfloor s/d \rfloor, 1/2\ell)$, we deduce that

$$\alpha_2 := \mathrm{P}_\infty(B \bmod p) \leq \frac{1}{p} + \frac{1}{2\ell} + \frac{1}{\lfloor s'/d \rfloor}\left(1 + \frac{\lfloor s/d \rfloor}{\ell p}\right) \tag{27}$$

Since $\mathrm{P}_\infty(\lfloor p\lambda V/d \rfloor + \lfloor pVc/dn \rfloor \bmod p) \leq \mathrm{P}_\infty(A + B \bmod p) + \mathrm{SD}(V, V')$ and since $\mathrm{SD}(V, V') \leq d/s' \leq 2\ell/s'$, we deduce that $\mathrm{P}_\infty(\lfloor pVx/n \rfloor \bmod p) \leq 7 \cdot (\max\{\alpha_1, \alpha_2\} + 2\ell/s')$. □

6.3 Proving Lemma 5.34

Lemma 6.8 (Restatement of Lemma 5.34)**.** *Let $\ell, n, s, u \in \mathbb{N}$, be such that $u \mid n$. Let $x \in \mathbb{N}$ be such that $(x \bmod n/u) \notin \mathcal{X}_{n/u,s/u,\ell}$, let $p \in \mathcal{P} \cap (2\ell, \infty)$ be such that $p \nmid n$ and $\lfloor s/u \rfloor / p > 16 \cdot \ell^2$, and let $V \xleftarrow{R} [s]$. Then for any $e \in (p-1)$:*

$$\mathrm{P}_\infty(e \cdot V - (x \cdot V \bmod n) \bmod p \mid V \bmod u) \leq 210 \cdot \ell^{-1} + 6 \cdot \frac{p^2}{\lfloor s/u \rfloor} \quad .$$

We prove Lemma 6.8 using the following additional lemma, proven in Section 6.3.1.

Lemma 6.9. *Let $\ell, n, s \in \mathbb{N}$ let u, h such that $u \mid n$ and $h \in \mathbb{Z}_u$ and $\lfloor s/u \rfloor / p > 16 \cdot \ell^2$. Let $x \in (n-1)$ such that $(x \bmod n/u) \notin \mathcal{X}_{n/u,s/u,\ell}$. Let $p \in \mathcal{P} \cap (2\ell, \infty)$ such that $p \nmid n$ and define $V \xleftarrow{R} [s] \mid V = h \bmod u$. Then,*

$$\mathrm{P}_\infty((V, \lfloor Vx/n \rfloor) \bmod p) \leq 5 \cdot \left(42 \cdot \frac{1}{\ell p} + \frac{p}{\lfloor s/u \rfloor}\right) + \frac{1}{\lfloor s/u \rfloor}$$

Proof of Lemma 6.8. Notice that $(x \cdot V \bmod n) = x \cdot V - \lfloor x \cdot V/n \rfloor \cdot n$ and thus

$$\begin{aligned}
&\mathrm{P}_\infty(e \cdot V - (x \cdot V \bmod n) \bmod p \mid V \bmod u) \\
&\qquad = \mathrm{P}_\infty(e \cdot V - (x \cdot V \bmod n) \bmod p \mid V \bmod u) \\
&\qquad \leq \mathrm{P}_\infty((e-x) \cdot V + \lfloor x \cdot V/n \rfloor \cdot n \bmod p \mid V \bmod u) \\
&\qquad \leq \mathrm{P}_\infty(n^{-1} \cdot (e-x) \cdot V + \lfloor x \cdot V/n \rfloor \bmod p \mid V \bmod u)
\end{aligned}$$

Fix an arbitrary $h \in \mathbb{Z}_u$ and define $V_h \xleftarrow{R} [s] \mid V_h = h \bmod u$. Notice that by Lemma 6.9 and Fact 6.2:

$$\mathrm{P}_\infty(n^{-1} \cdot (e-x) \cdot V_h + \lfloor x \cdot V_h/n \rfloor \bmod p) \leq 5 \cdot \left(42 \cdot \ell + \frac{p^2}{\lfloor s/u \rfloor}\right) + \frac{p}{\lfloor s/u \rfloor + 1}$$

Since the above holds for any fixed h, this concludes the proof. □

6.3.1 Proving Lemma 6.9

Proof of Lemma 6.9. Define $V_u \xleftarrow{R} \{0, \ldots, \lfloor s/u \rfloor\}$, and observe that $\mathrm{SD}(V, h + u \cdot V_u) \leq 1/(\lfloor s/u \rfloor + 1)$. Next, let $n' = n/u$ and recall that $x_0 = x \bmod n'$. Observe that

$$\begin{aligned}
\lfloor (h + uV_u)x/n \rfloor &= \lfloor hx/n + V_u x/n' \rfloor \\
&= \lfloor hx/n + V_u \lfloor x/n' \rfloor + V_u x_0/n' \rfloor \\
&= \lfloor hx/n \rfloor + V_u \lfloor x/n' \rfloor + \lfloor V_u x_0/n' \rfloor \pm 2.
\end{aligned}$$

Consequently,

$$\mathrm{P}_\infty((h + uV_u, \lfloor (h + uV_u)x/n \rfloor) \bmod p) \leq 5 \cdot \mathrm{P}_\infty((V_u, \lfloor V_u x_0/n' \rfloor) \bmod p).$$

In conclusion, we apply Lemma 6.6 with V_u and x_0 and deduce that

$$\mathrm{P}_\infty((h + uV_u, \lfloor (h + uV_u)x/n \rfloor) \bmod p) \leq 5 \cdot \left(42 \cdot \frac{1}{\ell p} + \frac{p}{\lfloor s/u \rfloor}\right).$$

and the lemma follows when accounting for the error in statistical distance. □

7 Implementing Commit&Modular-Reveal

In this section, we present two implementation of th commit and modular reveal functionality. In Section 7.1, we present an implementation of its *leaky* variant, Functionality 5.4, using *integer commitments* (security holds assuming strong RSA). In Section 7.2, we give an OT-based implementation of its non-leaky variant, Functionality 5.1. The OT-base construction is less communication-wise efficient that the integer commitments based variant, and it only works for restricted choice of the set $\mathcal{W}$ that parameterized the functionality (but suffices for our needs in Section 5), but the implementation is not leaky (which simplifies its use), and does not assumes additional hardness assumptions (it is information-theoretic secure in the OT-hybrid model).

7.1 Using Integer Commitments

In this section, we give an implementation of the leaky commit&modular-reveal functionality $\widetilde{\mathsf{Cmt\&ModRvl}}$, using *integer commitments* [FO97]. We start with relevant preliminaries about integer commitments. In Section 7.1.2, we present a protocol that uses the so-called integer commitments range and equality proofs and an ideal functionality $\mathsf{IntCmtGen}$ for honestly sampling the integer commitment parameters to realize the non-leaky commit&modular-reveal functionality $\mathsf{Cmt\&ModRvl}$. In Section 7.1.3, we show that by replacing the call to $\mathsf{IntCmtGen}$ with the recent light-setup protocol of [HLM25], we get a protocol that realizes $\widetilde{\mathsf{Cmt\&ModRvl}}$. Our implementations work for any *efficient set* $\mathcal{W}$.

Definition 7.1 (Efficient sets)**.** *A finite set $\mathcal{W} \subset \mathbb{N}$ is* $\mathsf{efficient}$ *if sampling and membership queries can be performed in polynomial time (in the description of $\mathcal{W}$).*

Notation 7.2 ($g_{\mathcal{W}}$)**.** *Given a s et $\mathcal{W}$, let $g_{\mathcal{W}} \colon \mathbb{N} \mapsto \mathcal{W}$ be the function that on input $w \in \mathbb{N}$ returns w if $w \in \mathcal{W}$ and an arbitrary (fix) element of $\mathcal{W}$ otherwise. Clearly, $g_{\mathcal{W}}$ runs in polynomial time for any efficient $\mathcal{W}$.*

7.1.1 Integer Commitments

We use the syntax of the recent survey of Haitner et al. [HLM25]. For $n, g, h \in \mathbb{N}$, let $\mathsf{intcmt}_{n,g,h}(a;\rho) = g^a \cdot h^\rho \bmod n$. Let $\mathsf{IntCmtGen}$ be the parameter-generation algorithm that takes as input computational security parameters 1^{κ_c} and minimum modulus length parameter s and outputs the commitment parameters $\mathsf{pp} = (n, g, h) \in \mathbb{N}^3$. Let $\mathsf{IntCmtGenEx}$ be its variant that also output $\alpha \in [n-1]$ such that $g = h^\alpha \bmod n$.

Integer commitments are known for their efficient *range* and *equality* proofs. Fix a cyclic group $\mathbb{G}$ with generator G.

Range proof. Consider the following promise problem:

$$\Pi^{\mathsf{RP}}_{\mathsf{s},d_v,d_r,\mathsf{pp}} \begin{cases} \text{Yes}: & \left\{(\widehat{V},(v,\rho))\colon \mathsf{intcmt}_{\mathsf{pp}}(v;\rho) = \widehat{V} \wedge v \in [d_v], \rho \in [d_r]\right\} \\ \text{Slack}: & \left\{(\widehat{V},(v,\rho))\colon \mathsf{intcmt}_{\mathsf{pp}}(v;\rho) = \pm\widehat{V} \wedge v \in [\pm \mathsf{s} \cdot d_v]\right\} \end{cases}$$

The Yes instances are integer commitments $\widehat{V}$ for $v \in [d_v]$. The Slack instances allow v to be somewhat larger than $[d_v]$ (and, for technical reasons, also allow negative a's).

Theorem 7.3 (Thm. 4.2, [HLM25])**.** *The exists a Sigma protocol $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$ such the following holds:*

Correctness. *For every $1^{\kappa_s}, 1^{\kappa_c}$, $d_v, d_r \in \mathbb{N}$ and $\mathsf{pp} = (n, g, h) \in \mathbb{N}^3$ with $n > 2 \cdot 2^{\kappa_s + \kappa_c} \cdot \max\{d_v, d_r\}$ given as parameters, $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$ is correct for any $(\widehat{V},(v,\rho)) \in \mathrm{Yes}^{\mathsf{RP}}_{\cdot,d_v,d_r,\mathsf{pp}}$.*

Zero knowldge. *For every $1^{\kappa_s}, 1^{\kappa_c}$, $d_v, d_r \in \mathbb{N}$ and $\mathsf{pp} = (n, g, h) \in \mathbb{N}^3$ given as parameters, $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$ is $2^{-\kappa_s}$-honest-verifier zero-knowledge for every $(\widehat{V},(v,\rho)) \in \mathrm{Yes}^{\mathsf{RP}}_{\cdot,d_v,d_r,\mathsf{pp}}$.*

Proof of knowledge. *Assuming the strong-RSA assumption,*[23] *for any efficient prover* $\mathsf{P}^*_{\mathsf{RP}}$ *there exists an efficient extractor* Ext *such that for any* $\kappa_{\mathsf{s}}, \kappa_{\mathsf{c}}, d_v, d_r \in \mathbb{N}$ *and* $\mathsf{s} \leftarrow 2^{2\kappa_{\mathsf{s}}}$*, the following happens with only negligible probability (in* κ_{c}*) over* $\mathsf{pp} \xleftarrow{R} \mathsf{IntCmtGen}(1^{\kappa_{\mathsf{s}}}, 1^{\kappa_{\mathsf{c}}}, \cdot)$*:* V_{RP} *accepts in an interaction with* $\mathsf{P}^*_{\mathsf{RP}}$ *with parameters* $c \leftarrow (1^{\kappa_{\mathsf{s}}}, 1^{\kappa_{\mathsf{c}}}, \mathsf{pp}, d_v, d_r)$ *and common input* $\widehat{V}$*, and* $\mathsf{Ext}(c, \widehat{V})$ *fails to output* $(\widehat{V}, (v, \rho)) \in \mathrm{Slack}^{\mathsf{RP}}_{\mathsf{s},d_v,d_r,\mathsf{pp}}$.

That is correctness and zero-knowledge hold for any YES instance and choice of pp. The knowledge extractor can output values in SLACK and might fail with negligible probability over the choice of pp (assuming strong RSA).

Equality proof. Consider the following promise problem:

$$\Pi^{\mathsf{Eq}}_{\mathsf{s},d_v,d_r,\mathsf{pp}} := \begin{cases} \text{Yes}: & \left\{((\widehat{V}, V), (v, \rho)) \colon \mathsf{intcmt}_{\mathsf{pp}}(v; \rho) = \widehat{V} \ \wedge \ v \cdot G = V \wedge v \in [d_v], \rho \in [d_r]\right\} \\ \text{Slack}: & \left\{((\widehat{V}, V), (v, \rho)) \colon \mathsf{intcmt}_{\mathsf{pp}}(v; \rho) = \pm\widehat{V} \ \wedge \ v \cdot G = V \wedge v \in [\pm\mathsf{s} \cdot d_v]\right\} \end{cases}$$

That is, the Yes instances are pairs $(\widehat{V}, V)$ of commitments for the same value $v \in [d_v]$. The commitment $\widehat{V}$ is just an integer commitment, and V is an "exponent commitment" in $\mathbb{G}$. The Slack instances allow a to be somewhat larger than $[d_v]$.

Theorem 7.4 (Thm. 5.3, [HLM25])**.** *The exists a Sigma protocol* $(\mathsf{P}_{\mathsf{Eq}}, \mathsf{V}_{\mathsf{Eq}})$ *such the following holds:*

Correctness. *For every* $1^{\kappa_{\mathsf{s}}}, 1^{\kappa_{\mathsf{c}}}$*,* $d_v, d_r \in \mathbb{N}$ *and* $\mathsf{pp} = (n, g, h) \in \mathbb{N}^3$ *with* $n > 2 \cdot 2^{\kappa_{\mathsf{s}}+\kappa_{\mathsf{c}}} \cdot \max\{d_v, d_r\}$ *given as parameters,* $(\mathsf{P}_{\mathsf{Eq}}, \mathsf{V}_{\mathsf{Eq}})$ *is correct for any* $((\widehat{V}, V), (v, \rho)) \in \mathrm{Yes}^{\mathsf{Eq}}_{\cdot,d_v,d_r,\mathsf{pp}}$.

Zero knowldge. *For every* $1^{\kappa_{\mathsf{s}}}, 1^{\kappa_{\mathsf{c}}}$*,* $d_v, d_r \in \mathbb{N}$ *and* $\mathsf{pp} = (n, g, h) \in \mathbb{N}^3$ *given as parameters,* $(\mathsf{P}_{\mathsf{Eq}}, \mathsf{V}_{\mathsf{Eq}})$ *is* $2^{-\kappa_{\mathsf{s}}}$*-honest-verifier zero-knowledge for every* $((\widehat{V}, V), (v, \rho)) \in \mathrm{Yes}^{\mathsf{Eq}}_{\cdot,d_v,d_r,\mathsf{pp}}$.

Proof of knowledge. *Assuming the strong-RSA assumption, for any efficient prover* $\mathsf{P}^*_{\mathsf{Eq}}$ *there exists an efficient extractor* Ext *such that* $\kappa_{\mathsf{s}}, \kappa_{\mathsf{c}}, d_v, d_r \in \mathbb{N}$ *and for* $\mathsf{s} \leftarrow 2^{2\kappa_{\mathsf{s}}}$*, the following happens with only negligible probability (in* κ_{c}*) over* $\mathsf{pp} \xleftarrow{R} \mathsf{IntCmtGen}(1^{\kappa_{\mathsf{s}}}, 1^{\kappa_{\mathsf{c}}}, \cdot)$*:* V_{Eq} *accepts in an interaction with* $\mathsf{P}^*_{\mathsf{Eq}}$ *with parameters* $c \leftarrow (1^{\kappa_{\mathsf{s}}}, 1^{\kappa_{\mathsf{c}}}, \mathsf{pp}, d_v, d_r)$ *and common input* $(\widehat{V}, V)$*, and* $\mathsf{Ext}(c, \widehat{V}, V)$ *fails to output* $((\widehat{V}, V), (v, \rho)) \in \mathrm{Slack}^{\mathsf{Eq}}_{\mathsf{s},d_v,d_r,\mathsf{pp}}$.

7.1.2 Non-leaky Commit&Partial-Reveal in the IntCmtGen-hybrid model

We abuse notation view $\mathsf{IntCmtGen}$ as the functionality that returns the output of the parameter-generation algorithm $\mathsf{IntCmtGen}$ (on uniform coins). We also use an ideal functionality Com.

Protocol 7.5. [Commit&modular-reveal]

Parties: S and R.

Paramters: $1^{\kappa_{\mathsf{s}}}, 1^{\kappa_{\mathsf{c}}}$, $d_v \in \mathbb{N}$ and $\mathcal{W} \subset \mathbb{N}$.

Oracle: $\mathsf{IntCmtGen}$, Com.

S's input: $v \in \mathbb{N}$.

1. R: Sample $w \xleftarrow{R} \mathcal{W}$ and send $c \xleftarrow{R} \mathsf{Com}(w)$ to S.
2. The parties jointly call $\mathsf{IntCmtGen}(1^{\kappa_{\mathsf{c}}}, d_v \cdot 2^{\kappa_{\mathsf{s}}})$, let $\mathsf{pp} \leftarrow (n, g, h)$ be the common output and let $d_r \leftarrow n \cdot 2^{\kappa_{\mathsf{s}}}$.

[23] A somewhat more complicated protocol exists under standard RSA.

3. S: Send $\widehat{V} \leftarrow \mathsf{intcmt}_{\mathsf{pp}}(v;\rho)$, for $\rho \xleftarrow{R} [d_r]$, to R.

4. The parties interact in $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$, on parameters $(1^{\kappa_s}, 1^{\kappa_c}, d_v \cdot 2^{2\kappa_s}, d_r, \mathsf{pp})$, common input $\widehat{V}$, and S uses private input (v, ρ).

5. R opens c to reveal w.

6. S sets $w \leftarrow g_{\mathcal{W}}(w)$ and sends $v_w \leftarrow v \bmod w$ to R.

7. The parties interact in $(\mathsf{P}_{\mathsf{Eq}}, \mathsf{V}_{\mathsf{Eq}})$, with $\mathbb{Z}_w$ as the group $\mathbb{G}$, 1 is the generator G, parameters $(1^{\kappa_s}, 1^{\kappa_c}, d_v \cdot 2^{2\kappa_s}, d_r, \mathsf{pp})$, common input $(\widehat{V}, v_w)$ and S using private input (v, ρ).

As is, Protocol 7.5 does not UC-realize Cmt&ModRvl; a security simulator will require rewinding of a cheating sender to extract the input it "uses" in $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$. Fortunately, in the random-oracle model, we get the desired protocol by using Fischlin's transform [Fis05] on the sender's challenge. Let Π' be the variant of Protocol 7.5 in which the challenge sent by V_{RP} is sampled by P_{RP} by applying Fischlin's sampling method on a random oracle.

Theorem 7.6 (Security of Protocol 7.5)**.** *Assuming strong-RSA holds, then for any efficient set $\mathcal{W}$, protocol Π' UC-realizes* Cmt&ModRvl *for slackness parameter $\mathsf{s} \leftarrow 2^{2\kappa_s}$, in the $\{\mathsf{IntCmtGen}, \mathsf{Com}\}$-hybrid model.*

Proof. The simulation of a cheating receiver immediately follows from the (unconditional) hiding of the integer commitment and the hone-verifier zero-knowledge property of $(\mathsf{P}_{\mathsf{Eq}}, \mathsf{V}_{\mathsf{Eq}})$. It extracts w from the receiver, sends $g_{\mathcal{W}}(w)$ to the functionality, and forwards the result to the receiver. It emulates the sender's messages by committing to garbage and using the zero-knowledge simulator to simulate the proofs.

The simulation of a cheating sender extracts the input v from the sender (since we use Fischlin for $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$, this can be done without rewinding) and sends it to the functionality. The simulation then continues via standard means □

Remark 7.7 (W/o Fischlin and Com). Since the UC-simulation done in the security proof of Protocol 5.14 does not use the values provided by the parties in commit phase, the following variant Π'' of Protocol 7.5 suffices for the security proof of Protocol 5.14. Let RO be the random oracle functionality, mapping string of arbitrary length to $\{0,1\}^{\kappa_c}$.

1. Instead of calling Com in the first round, R sends $c \leftarrow \mathsf{RO}(w;r)$ to S for $r \xleftarrow{R} \{0,1\}^{\kappa_s}$. In the reveal phase, R sends (w, r) tp S, who verifies $c = \mathsf{RO}(w;r)$.

2. The interaction in $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$ is removed. (The call to $(\mathsf{P}_{\mathsf{Eq}}, \mathsf{V}_{\mathsf{Eq}})$, will make sure v is in the right range.)

3. The challenge sent by V_{Eq} is sampled by P_{Eq} by applying a *single* call to RO, on the transcript so far.

4. The challenge e in protocols $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$ and $(\mathsf{P}_{\mathsf{Eq}}, \mathsf{V}_{\mathsf{Eq}})$, is sampled from $[2^{\kappa_c}]$.

5. Parameter d_v in the execution of Π_{RP} is changes to $d_v \cdot 2^{\kappa_s + \kappa_c}$

Clearly, the security proof for the variant of Protocol 5.14 that uses protocol Π'' can no longer be proven using the UC compossibility framework. Yet, a direct argument yields that this variant is UC-secure, for $\mathsf{s} \leftarrow 2^{\kappa_s + \kappa_c}$.

7.1.3 Leaky Commit&Partial-Reveal

Since we have no efficient implementation for $\mathsf{IntCmtGen}$,[24] we use the recent light-setup parameters well-formedness protocol of Haitner et al. [HLM25] to implement a leaky-variant of the $\mathsf{IntCmtGen}$. This comes with a price, instantiating Protocol 7.5 with [HLM25]'s protocol only realizes the leaky Commit&modular-reveal functionality $\widetilde{\text{Cmt\&ModRvl}}$ Let $(\mathsf{P}_{\mathsf{WF}}, \mathsf{V}_{\mathsf{WF}})$ be according to [HLM25, Protocol 6.1].

[24] Known implementation require the verifier to prove that $g \in \langle h \rangle$, and the communication complexity of known protocols for this task grow linearly with the security parameter.

Protocol 7.8. [Leaky commit&modular-reveal]

Parties: S and R.

Paramters: $1^{\kappa_s}, 1^{\kappa_c}$, $d_v \in \mathbb{N}$ and $\mathcal{W} \subseteq \mathbb{N}$.

Oracle: Com.

S's input: $v \in \mathbb{N}$.

1. R: Sample $w \xleftarrow{R} \mathcal{W}$ and send $c \xleftarrow{R} \mathsf{Com}(w)$ to S.
2. R: Sample $(\mathsf{pp}, \alpha) \leftarrow \mathsf{IntCmtGenEx}(1^{\kappa_c}, d_a \cdot 2^{\kappa_s})$ and send pp to S.
3. The parties interact in $(\mathsf{P}_{\mathsf{WF}}, \mathsf{V}_{\mathsf{WF}})$, on parameter 1^{κ_s}, common input pp, and R, acting P_{WF}, using private input α.
4. Continue as in Protocol 7.5, after the $\mathsf{IntCmtGen}$ call.

Let Π' be the variant of Protocol 7.8 in which the challenge sent by V_{Eq} is sampled by P_{Eq} by applying Fischlin method on a random oracle.

Theorem 7.9 (Security of protocol Π')**.** *Assuming strong-RSA holds, then for any efficient set $\mathcal{W}$ protocol Π' UC-realizes $\widetilde{\mathsf{Cmt\&ModRvl}}$ for slackness parameter $\mathsf{s} \leftarrow 2^{2\kappa_s}$, in the Com-hybrid model.*

As in the case of Protocol 7.5, see Remark 7.7, in Protocol 5.14 it is OK to remove the interaction of $(\mathsf{P}_{\mathsf{RP}}, \mathsf{V}_{\mathsf{RP}})$, and that V_{Eq}'s challenge is sampled by P_{Eq} using a a single call to the random oracle.

Proof. We focus on simulating a malicious receiver $\widehat{\mathsf{R}}$. For a fixed first message pp sent by $\widehat{\mathsf{R}}$, let $f_{\mathsf{pp}}(x; \rho) := \mathsf{intcmt}_{\mathsf{pp}}(x; \rho)$ and let the game G_{pp} be protocol $(\mathsf{P}_{\mathsf{WF}}, \mathsf{V}_{\mathsf{WF}})$ on parameter 1^{κ_s}, and common input pp, in which the player is the prover. By [HLM25, Claims 6.4, 6.5], exists $\ell = \ell(\mathsf{pp}) \in \mathbb{N}$ such that

1. The value of G_{pp} is at most $1/\ell + 2^{-\kappa_s}$.
2. $\mathrm{SD}(f_{\mathsf{pp}}(x), f_{\mathsf{pp}}(x')) \leq 2^{-\kappa_s}$ for any $x \equiv x' \bmod \ell$.

Furthermore, [HLM25] contains a (non-interactive) proof π for the above. Given the above, the simulator Sim for $\widehat{\mathsf{R}}$ is defined as follows:

Algorithm 7.10 (Sim)**.**

Oracle: $\mathsf{Cmt\&ModRvl}$.

Operation:

1. Emulate $\widehat{\mathsf{R}}$ until is sends its first message pp. Let w be the input it used in the call to Com.
2. Send $(g_{\mathcal{W}}(w), f_{\mathsf{pp}}, \mathsf{G}_{\mathsf{pp}}, \pi)$ to $\mathsf{Cmt\&ModRvl}$.
3. Interact in G_{pp} with $\mathsf{Cmt\&ModRvl}$, by forwarding $\widehat{\mathsf{R}}$'s message to/from $\mathsf{Cmt\&ModRvl}$. Abort if $\mathsf{Cmt\&ModRvl}$ does.
4. Continue as in the proof of Theorem 7.6.

□

7.2 Using Oblivious Transfer

In this section we present a protocol that realizes the *non-leaky* Commit & Modular Reveal functionality Cmt&ModRvl with information-theoretic security in the COT-hybrid model. Unlike the protocols we presented in Section 7.1, the protocol we introduce here does not work for arbitrary efficient sets (Definition 7.1), but only for sufficiently large sets of prime numbers of a minimum length. In other words, it works for precisely the sets required by Protocol 5.14. It also enjoys slightly less soundness slack than the protocols in Section 7.1.

Our OT-based protocol is fundamentally based upon cut-and-choose techniques and follows from a simple intuition. Consider the following toy protocol:

1. The sender encodes its input v as the additive correlation between two OT messages (each of which is nearly uniformly distributed when considered alone).
2. The receiver chooses one of the OT messages at random, and then transmits random prime modulus w (much smaller than the message length) to the sender.
3. The sender replies by communicating the residues of the two OT messages with respect to w.
4. The reciever (who knows one OT message) is able to check one of the residues, and aborts if the one it checks is not correct. If it does not abort, then it outputs the difference between the residues, modulo w.

In this protocol, the sender can cheat only by sending an incorrect residue (such that the difference between the transmitted residues is not congruent to the difference between the OT inputs). If it does so, it will be caught with probability at least $1/2$. We can amplify the soundness of this protocol through parallel repetition, and optimize it by transmitting one of the two residues in each instance, along with the single (consistent) residual difference.

There are two important nuances when repeating the protocol above. First, we must bound the probability that the adversary chooses to encode multiple values of v in different instances, and they *happen* to be congruent modulo w (thus allowing the prover to get away with cheating). We do this by enforcing a minimum size s_w for w and by insisting that the set of primes from which the receiver samples w is exponentially large relative to s_w.

Second, since we desire simulation security, we must ensure that the simulator can extract an input v whenever there is a noticeable chance that the protocol does not abort. Consider two parallel repetitions of the above protocol cut-and-choose protocol: if the adversary encodes a different value of v in each, it avoids an abort with probability $\approx 1/4$, yet the simulator has no way of knowing which value of v to send to to the functionality. We solve this problem by insisting that there are enough repetitions that if there is not a *majority* value of v, then the protocol (and simulator) abort with probability $2^{-\kappa_s}$.

We now give a formal specification of the protocol, followed by a security theorem and proof, and finally we discuss optimizations and give an analysis of concrete costs.

Protocol 7.11 (OT-based Commit&modular-reveal)**.**

Parties: Sender S and a Receiver R.

Parameters: A statistical parameter $1^{\kappa_s} \in \mathbb{N}$, an input bound $d_v \in \mathbb{N}$, and a prime length $s_w \in \mathbb{N}$.

Derived parameters: Let $\kappa_s{}' = 2\kappa_s + 2$, $d_v{}' = 2^{\kappa_s + \log_2 \kappa_s{}'} \cdot d_v$.

Oracles: $\mathsf{COT}_{d_v{}'+1}$

S's input: $v \in (d_v)$

Operation:

1. R: Sample $\mathbf{b} \xleftarrow{R} \{0,1\}^{\kappa_s{}'}$.

2. For $i \in [\kappa_{\mathsf{s}}']$ (in parallel) the parties invoke $\mathsf{COT}_{d_v'+1}(v, b_i)$. The output of S is ρ_i. The output of R is ρ_i if $b_i = 0$ or $\hat{v}_i$ if $b_i = 1$.

3. R: Sample $w \xleftarrow{\mathrm{R}} \mathcal{P}_{s_w}$ and send w to S.

4. S: Compute

$$\hat{\rho}_i \leftarrow \rho_i \bmod w \quad \text{for } i \in [\kappa_{\mathsf{s}}']$$
$$v_w \leftarrow v \bmod w$$

and send $(v_w, \hat{\boldsymbol{\rho}})$ to R.

5. R: Compute

$$\hat{\rho}_i' \leftarrow \rho_i \bmod w \quad \text{for } i \in [\kappa_{\mathsf{s}}'] \text{ s.t. } b_i = 0$$
$$\hat{\rho}_i' \leftarrow (\hat{v}_i - v_w) \bmod w \quad \text{for } i \in [\kappa_{\mathsf{s}}'] \text{ s.t. } b_i = 1$$

and abort if there is any $i \in [\kappa_{\mathsf{s}}']$ such that $\hat{\rho}_i \neq \hat{\rho}_i'$.

Output: S and R both output (w, v_w)

Theorem 7.12. *If* $|d_v| \geq s_w \geq \max(6, \kappa_{\mathsf{s}} + \log_2(20) + \log_2(\kappa_{\mathsf{s}}) + \log_2(\log_2 d_v + \kappa_{\mathsf{s}} + \log_2(2\kappa_{\mathsf{s}} + 2)))$ *then Protocol 7.11 statistically UC-realizes* $\mathsf{Cmt\&ModRvl}_{d_v, 2^{s_w}, \mathcal{P}_{s_w}, \mathsf{s}}$ *with slackness parameter* $\mathsf{s} = 2^{\kappa_{\mathsf{s}} + \log_2(2\kappa_{\mathsf{s}}+2)}$ *and simulation error* $2^{-\kappa_{\mathsf{s}}}$ *in the* $\mathsf{COT}_{d_v'+1}$*-hybrid model against a malicious adversary corrupting either* S *or* R.

Proof. Simulation against a corrupt R is trivial: the simulator learns $\mathbf{b}$, for $i \in [\kappa_{\mathsf{s}}']$ samples either ρ_i (if $b_i = 0$) or $\hat{v}_i$ (if $b_i = 1$) uniformly, and computes accepting values of $\hat{\boldsymbol{\rho}}$ (as defined by the equations in step 5 of the protocol) once it receives w from R and v_w from $\mathsf{Cmt\&ModRvl}$. This simulation can be distinguished from the real world experiment only in the case that there exists some $i \in [\kappa_{\mathsf{s}}']$ such that $v + \rho_i > d_v'$, in which case the equations in step 5 of the protocol hold in the ideal world, but *do not hold* in the real world.[25] This occurs with probability at most $\kappa_{\mathsf{s}}' \cdot d_v / d_v' \leq 2^{-\kappa_{\mathsf{s}}}$.

For the remainder of the proof, we focus on the case the S is corrupted. We begin this case by specifying a simulator:

Algorithm 7.13 (Simulator for Protocol 7.11 and a corrupt prover)**.**

1. For each $i \in [\kappa_{\mathsf{s}}']$: upon receiving input v_i' from S on behalf of $\mathsf{COT}_{\mathsf{d_v'+1}}$, send $\rho_i \xleftarrow{\mathrm{R}} (d_v' + 1)$ to S.

2. Let v' be the most frequent value of v_i' over all $i \in [\kappa_{\mathsf{s}}']$. Send v' to $\mathsf{Cmt\&ModRvl}$. Let w be its reply.

3. Send w to S.

4. Upon receiving v_w and $\hat{\boldsymbol{\rho}}$ from S,

   (a) Let

$$\mathbf{b} \xleftarrow{\mathrm{R}} \{0,1\}^{\kappa_{\mathsf{s}}'}$$
$$\hat{\rho}_i' \leftarrow \rho_i \bmod w \quad \text{for } i \in [\kappa_{\mathsf{s}}'] \text{ s.t. } b_i = 0$$
$$\hat{\rho}_i' \leftarrow (\hat{v}_i - v_w) \bmod w \quad \text{for } i \in [\kappa_{\mathsf{s}}'] \text{ s.t. } b_i = 1.$$

[25] In this circumstance, a "false positive" abort occurs in the real world even though S is honest.

(b) If there is any $i \in [\kappa_s{}']$ such that $\hat{\rho}_i \neq \hat{\rho}'_i$, instruct Cmt&ModRvl to abort. Otherwise, instruct Cmt&ModRvl to release its output to R.

Next, we will prove that for any (potentially unbounded) and adversary that corrupts S, any κ_s, and any d_v and s_w such that $|d_v| \geq s_w \geq \max(6, \kappa_s + \log_2(20) + \log_2(\kappa_s) + \log_2(\log_2 d_v + \kappa_s + \log_2(2\kappa_s + 2)))$, the statistical distance between the distribution produced by an adversary's real world interaction in Protocol 7.11 and the simulator's ideal-world interaction with Cmt&ModRvl (with slackness parameter $\mathsf{s} = 2^{\kappa_s + \log_2(2\kappa_s+2)}$) is no greater than $2^{-\kappa_s}$.

We observe first of all that the conditions under which the simulator causes the ideal-world experiment to abort are exactly the same as those under which the real-world experiment aborts. Furthermore, w is identically distributed in the two experiments. The only distinction between the two is that if the experiments do not abort, then the functionality always outputs v'_w to R in the ideal world, which is the modular residue of the *most frequent* value of v'_i over all i,[26] whereas the real-world R outputs the value v_w that is transmitted by S. The adversary can therefore distinguish the two worlds only by contriving to send $v_w \neq v'_w$ without triggering an abort. It remains only to upper-bound the probability that it does so.

We can break our proof into two cases:

1. There exist indices i and j such that $v'_i \neq v'_j$ but $a'_i \bmod w = a'_j \bmod w = v_w$. In other words, $\mathbf{a'}$ contains more than one distinct value congruent to v_w modulo w. Let us fix the first such value as v'_i. There are at most $\kappa_s{}' - 2 = 2\kappa_s$ other indices which could also be congruent to v_w (recall that at least one index is congruent to v'_w). Per Proposition 3.6, the probability that any of them are congruent to v_w is no greater than
$$2\kappa_s \cdot \frac{5 \log_2 d_v{}'}{2^{s_w}}.$$

2. For every i and j such that $v'_i \bmod w = v'_j \bmod w = v_w$, it also holds that $v'_i = v'_j$. In other words, $\mathbf{v'}$ contains at most one distinct value congruent to v_w modulo w. Let that value (if it exists) be v'_i. Because v'_i is *not* the most frequent value, it follows that the number of indices i such that $v'_i \bmod w \neq v_w$ is *at least* $\kappa_s{}'/2$. The probability that the equations in step 5 of the protocol (and step 4 of the simulator) hold for *all* of these indices is therefore no greater than $2^{-\kappa_s{}'/2}$ in this case.

The theorem also specifies that $s_w \geq \kappa_s + \log_2(20) + \log_2(\kappa_s) + \log_2 \log_2 d_v{}'$, and so combining our cases we have a total distinguishability no greater than

$$2\kappa_s \cdot \frac{5 \log_2 d_v{}'}{2^{s_w}} + 2^{-\kappa_s - 1} \leq 2\kappa_s \cdot \frac{5 \log_2 d_v{}'}{2^{\kappa_s + \log_2(20) + \log_2(\kappa_s) + \log_2 \log_2 d_v{}'}} + 2^{-\kappa_s - 1} = 2^{-\kappa_s}$$

which concludes the proof. □

Using Protocol 7.11 in Protocol 5.14. Protocol 7.11 as specified above samples w from $\mathcal{P}_{s_w}$, the set of all primes of length s_w. On the other hand, Protocol 5.14 requires the sample set to be $\mathcal{P}_{s_w} \setminus \{q\}$ for some fixed prime q. We observe that this is not a problem: for most parameter regimes (see Table 5.16), $|q| \neq w$. On the other hand, if $|q| = s_w$, the proof goes through essentially unchanged if R samples $w \xleftarrow{R} \mathcal{P}_{s_w} \setminus \{q\}$ in step 3 of the protocol.

7.2.1 Optimizations and Cost Analysis

Eliminating Interaction via Fiat-Shamir. We note that the receiver in Protocol 7.11 is only responsible for the public-coin sampling and transmission of w, and for selecting random choice bits. We can apply the Fiat-Shamir heuristic to reduce the protocol to a single message from S to R in the COT-hybrid model. Since a corrupt sender can rejection sample the output of the random oracle, we must replace the statistical parameter with the computational security parameter in all places where soundness depends upon it. Assuming random

[26] Note that this isn't necessarily the same thing as the most frequent modular residue!

OT correlations are known to the parties, COT with a random choice bit can be implemented using a single message from sender to receiver. It is also possible to acheive a single message *without* pre-existing random OT correlations using the VOLE-in-the-head compiler [BBSGKORS23].

Bandwidth Reduction. In step 5 of Protocol 7.11, the receiver performs equality checks on $\kappa_{\mathsf{s}}' = 2\kappa_{\mathsf{s}} + 2$ individual values that were transmitted by the sender. In the computational setting, the sender can instead transmit the hash of these values, and the receiver can check equality of hashes. This reduces the bandwidth consumption of the protocol by $\kappa_{\mathsf{s}}' \cdot s_w - 2\kappa_{\mathsf{c}}$ bits.

Parameterization and Cost Analysis. Assuming the above bandwidth reduction optimization is employed, but Fiat-Shamir is *not* applied, and we measure costs in the Random OT model, the interactive version of the protocol involves transmitting $|d_v'| \cdot \kappa_{\mathsf{s}}' + 2\kappa_{\mathsf{c}} \leq (|d_v| + \kappa_{\mathsf{s}} + \log_2(\kappa_{\mathsf{s}} + 1) + 1) \cdot (2\kappa_{\mathsf{s}} + 2) + 2\kappa_{\mathsf{c}}$ bits from S to R, and s_w bits in the opposite direction. This excludes the cost of generating $2\kappa_{\mathsf{s}} + 2$ ROT correlations of size $|d_v'|$. In the following table we report computed bandwidth costs for several parameter regimes, along with the minimum values of s_w implied by the bound in Theorem 7.12.

$\lvert d_v \rvert$	373	409	537	667	797	657	741	870	998
κ_{s}	40	40	40	40	40	80	80	80	80
KB Sent	4.33	4.70	6.01	7.34	8.68	15.10	16.81	19.42	22.01
Min s_w	56	56	56	56	56	98	98	98	98

Table 7.14: Bandwidth Cost and Parameterization for Protocol 7.11. In all cases we set $\kappa_{\mathsf{c}} = 128$, and omit the cost of communicating w, which is s_w. Values of d_v correspond to useful/common values of $|q|$, as determined by the parameterization of Protocol 5.14. See also Table 5.16.

8 Applications

OLE is a general-purpose tool for arithmetic secure computation [IPS09]. Here we describe several concrete applications that use a *small number* of OLE or short VOLE instances, in which case our CRT-based approach substantially outperforms prior approaches in terms of communication cost. For our fully malicious protocols, this comes at the price of a higher round complexity. The relatively high round complexity of our protocols does not seem inherent, and improving it is left for future work.

8.1 Power-Residue Based Post-Quantum OPRF

We start with an application that only requires a *single* OLE with security only against a malicious receiver (and semi-honest sender).

Oblivious pseudorandom functions. An *oblivious pseudorandom function* (OPRF) [NR04; FIPR05] is a secure two-party protocol for a functionality defined by a pseudorandom function (PRF) $F(k, x)$. Concretely, in the beginning of the protocol a server holds a secret key k and a client holds a secret evaluation point x. In the end of the protocol, the client should only learn the PRF output $F(k, x)$ and the server should learn nothing about x.

Post-quantum OPRFs. OPRFs are finding a growing number of applications, including wide real-world deployment in the context of password-based protocols used by Whatsapp, PrivacyPass and more; see [BDFH24; CHL22] for a survey. However, while there is a simple and efficient protocol based on discrete-log style assumptions [JKK14], OPRF protocols that offer plausible post-quantum security lag far behind. Despite a significant body of recent works (see, e.g., [GRRSS16; BIPSW18; BKW20; ADDS21; DGHIKSZ21; Bas23; ADDG24; APRR24; BDFH24; KCM24] and references therein), all protocols that offer some kind

of security against malicious parties are quite inefficient. For example, the recent fully malicious protocol from [BDFH24] requires 1MB of communication for an OPRF with a 1-bit output, over 3 orders of magnitude more than the simple discrete-log based protocol from [JKK14] that has a 128-bit output. Assuming both parties to be semi-honest, there are plausibly post-quantum protocols with a much better communication cost [DGHIKSZ21; APRR24]; however, this level of security does not suffice for most realistic use cases.

Malicious-client OPRFs. In several use cases of OPRFs, it is realistic to assume the server to be semi-honest, whereas it is unrealistic to make the same assumption for the (often numerous) clients interacting with the server. In this setting, it was recently shown in [YBHKR25] that a well-studied post-quantum PRF candidate based on the power residue function (a higher-residue variant of the Legendre function) gives rise to an OPRF that only requires a *single* OLE call, where the latter only needs to be secure against malicious clients. Thus, combined with our malicious-receiver OLE from Section 4, we get a malicious-client OPRF that only requires 1.14 KB of communication in the OT-hybrid model for the default choice of parameters suggested in [YBHKR25], roughly 16x better than a similar protocol based on Gilboa's OLE. for self-containment, we give the details of this application below.

Power-residue PRF. Let p, q be primes and $m \geq 2$ an integer such that $p = mq + 1$. For $w \in \mathbb{Z}_p$, the (m^{th} residue) Legendre symbol of a is defined by

$$\left(\frac{w}{p}\right)_m = w^{\frac{p-1}{m}} \bmod p.$$

For $m = 2$, this is just the standard Legendre symbol that outputs 0 if $a = 0$, 1 if a is a nonzero square, and -1 otherwise. For $k, x \in \mathbb{Z}_p$ the power-residue PRF candidate [Dam88] is defined by

$$F_{\mathsf{LPRF}}(k, x) = \left(\frac{k+x}{p}\right)_m.$$

Note that the output domain is of size m, namely the subgroup of the m^{th} roots of unity in $\mathbb{Z}_p$; if $m \geq 2^k$, a pseudorandom k-bit output can be obtained by applying a random oracle $R : \{0,1\}^{\lceil \log p \rceil} \to \{0,1\}^k$ to the PRF output.

Parameters. The power-residue PRF has been proposed as a basis for post-quantum cryptography[27] and was subject to a considerable amount of analysis; see [YBHKR25] and references therein. To get 128-bit security against known quantum attacks with a 128-bit output, the parameter q also needs to satisfy $q \approx 2^{256}$. In this case, one should choose $p \approx 2^{384}$ and $x \in [0, 2^{256})$.

Power-residue OPRF. Using a standard randomized encoding technique [FKN94; GRRSS16; BHILM20], one can securely compute $F_{\mathsf{LPRF}}(k, x)$ by computing the randomized functionality

$$F'_{\mathsf{LPRF}}(k, x; r) = (k + x) \cdot r^m,$$

with randomness $r \in_R \mathbb{Z}_p$, where the output y of F can be efficiently decoded from the output y' of F' by letting $y = \left(\frac{y'}{p}\right)_m$. Writing the output of F' as $ax + b$, for $a = r^m$ and $b = k \cdot r^m$, one obtains the following reduction of F_{LPRF} to $\mathsf{OLE}_{\mathsf{p}}$:

- Inputs: The server has $k \in \mathbb{Z}_p$ and the client $x \in \mathbb{Z}_p$.
- The server picks $r \in_R \mathbb{Z}_p$ and computes $a = r^m$ and $b = k \cdot r^m$.

[27] As defined above, F_{LPRF} can be broken by a quantum algorithm given *quantum oracle access* to the function [DH00]. However, as discussed in [BSG20; YBHKR25], this is not relevant to use cases of a post-quantum OPRF.

- The server and the client invoke the $\mathsf{OLE_p}$ oracle with server input (a, b) and client input x. Denote the client's output by y'.
- The client outputs $y = \left(\frac{y'}{p}\right)_m$.

Theorem 8.1. *The above protocol realizes F_{LPRF} with $1/p$-security against a malicious client and a semi-honest server in the $\mathsf{OLE_p}$-hybrid model.*

Proof. Since the server only acts as OLE sender and does not receive any messages, security against semi-honest server is trivial. The client's output is correct except when $r = 0$, which happens with $1/p$ probability. To simulate a malicious client that feeds x to the $\mathsf{OLE_p}$ oracle, the simulator invokes the F_{LPRF} functionality with input x, obtaining an output y, and then simulates the oracle output y' by picking a random $r \in_R \mathbb{Z}_p$ and letting $y' = y \cdot r^m$. The simulation is perfect: since r^m is uniformly random in the subgroup of $\mathbb{Z}_p$ elements with Legendre symbol 1, the simulated y' is uniformly distributed over the coset of y. This is also the case in the real protocol, where $y' = ax + b = (k + x) \cdot r^m$ is uniformly distributed over the coset of $F_{\mathsf{LPRF}}(k, x)$. □

Realizing the $\mathsf{OLE_p}$ oracle using the malicious-receiver OLE from Section 4, we get a malicious-client OPRF with significantly better communication cost than a similar protocol based on Gilboa's OLE. With $p \approx 2^{384}$ for a 128-bit OPRF output (resp., $p \approx 2^{512}$ for a 256-bit output), the protocol requires ≈ 1.14 KB (resp., ≈ 1.47 KB) of communication in the ROT-hybrid model, roughly 16x (resp., 22x) better than a similar OPRF protocol based on Gilboa's OLE. Note that one can get an additional efficiency gain by restricting the honest client's input to come from an interval $x \in [0, 2^{256}) \subset \mathbb{Z}_p$ (while still allowing a malicious client to choose an input from the entire PRF domain).

8.2 Threshold Nonlinear Signatures

Recent years have seen a surge of interest in threshold signing protocols for common signatures, which has driven advancements in the techniques and building blocks that underlie such protocols, and also yielded a large number of concretely efficient and actually-deployed schemes. Of particular interest to us are signature schemes with signing equations that combine secret values *nonlinearly* because distributing such signatures inherently requires implementing the nonlinear operation in a distributed fashion, and this is often achieved via OLE.

ECDSA. The most prominent example is surely the Elliptic Curve Digital Signature Algorithm—ECDSA—which is relatively old, standardized by the United States government, and essentially ubiquitous in the domain of internet infrastructure. It is also commonly used by modern blockchain protocols, including Bitcoin. In many contexts, but particularly the context of cryptocurrency custody, individual signatures or small numbers of signatures must be generated with low latency, and potentially on relatively-underpowered hardware such as mobile phones. In such contexts it is advantageous to realize with a protocol that is concretely efficient in the *non-amortized* setting, such as the one we present.

Given a secret key sk, message m, and ephemeral key k, an ECDSA signer must first compute the public nonce $R \leftarrow k \cdot G$ where G is the elliptic curve generator, and then $s \leftarrow (\mathsf{SHA2}(m) + \mathsf{sk} \cdot r_{\mathsf{x}})/r$,[28] where r_{x} is the x-coordinate of R. Many threshold ECDSA protocols generate linear secret shares of sk and k, and then use the inversion trick of Bar-Ilan and Beaver [BB89] to rewrite the term sk/k as a sequence of OLE operations.[29] Specifically, under the Bar-Ilan and Beaver technique, the parties sample linear shares of an additional random value ϕ, use (Vector) OLE (applied to pairs of individual shares) to compute linear shares of $\alpha = \mathsf{sk} \cdot \phi$ and $\beta = k \cdot \phi$, and then publish β. Once β is public, linear shares of $\mathsf{sk}/k = \alpha/\beta$ can be computed via linear operations on the shares of α. The remainder of the problem consists of ensuring that

[28] We write elliptic curve operations additively.

[29] Although, for various technical reasons, many threshold ECDSA protocols do not use a clean OLE abstraction.

k is sampled without bias and that the various shared values are used in a consistent way between the OLE instances and the other parts of the signing equation.

Although a long line of work [CMI93; GJKR96; MR01; GGN16; Lin17; BGG17; DKLs18; GG18; LN18; DKLs19; ST19; CGGMP20; DJNPØ20; DOKSS20; CCLST20; GS22; ANOSS22; CCLST23; HLNR23][30] has generated and optimized many approaches to threshold ECDSA signing, we will focus on the recent protocol of Doerner et al. [DKLs24] for two related reasons: first, their protocol is based upon a standard Vector OLE functionality with a vector length of two, which is instantiated twice per pair of parties; second, their protocol is simple enough that its cost is dominated in all metrics by the cost of these Vector OLE instances.[31] Apart from Vector OLE costs, only a handful of elliptic curve operations are performed and only a few curve points are transmitted, over one additional round of communication.

Doerner et al. propose their own protocol to realize VOLE against malicious adversaries in the OT-hybrid random oracle model, which is based on a hardening of the classic Gilboa OLE protocol. We briefly discussed their approach in Section 1 and discussed the relationship between our protocol and theirs in terms of performance. It remains to observe that the performance improvements (and degradations) that our protocol attains relative to theirs carry over essentially directly to their ECDSA protocol, when their VOLE protocol is replaced by ours. In the interest of concreteness, we compute the concrete costs for signing, using both the VOLE protocol of Doerner et al. and our own VOLE protocol (in both variants). For each protocol combination, we furthermore give costs for two different OT Extension protocols: the SoftSpoken OT protocol of Roy [Roy22] (with tuning parameter $k = 4$), and the Silent OT protocol [BCGIKS19], which achieves nearly zero marginal bandwidth cost per OT instance.[32] These costs are reported in Table 8.2. We do not report the setup costs for OT Extension.

VOLE Protocol	OT Extension Protocol	KB Sent Per Signer	Rounds
DKLs24	SoftSpoken	$(N-1) \cdot 45.6$	3
DKLs24	Silent	$(N-1) \cdot 37.4$	3
Ours (Pure OT)	SoftSpoken	$(N-1) \cdot 31.9$	11
Ours (Pure OT)	Silent	$(N-1) \cdot 18.7$	11
Ours (RSA)	SoftSpoken	$(N-1) \cdot 21.0$	11
Ours (RSA)	Silent	$(N-1) \cdot 8.19$	11

Table 8.2: Bandwidth and Round Counts for an instance of the ECDSA signing protocol of Doerner et al. [DKLs24] involving N signers, with $\kappa_{\mathsf{s}} = 40$, $\kappa_{\mathsf{c}} = 128$, and a 256-bit elliptic curve.

BBS+. The BBS+ signature scheme (named in honor of Boneh, Boyen, and Shacham, but actually introduced by Au, Susilo, and Mu [ASM06]) accommodates concretely efficient proofs of possession, which makes it useful for the construction of anonymous credentials. Given the secret key sk, message vector $\mathbf{m}$, and nonces s and e, a BBS+ signature is (A, e, s) where

$$A \leftarrow \frac{G + s \cdot H_1 + \sum_i m_i \cdot H_{i+1}}{\mathsf{sk} + e}.$$

In order to create a threshold signature, the signers must produce shares of $1/(\mathsf{sk} + e)$ given linear shares of sk and e, and like in the case of threshold ECDSA signing, this can be done by using the Bar-Ilan and Beaver technique to rewrite the inversion as a sequence of OLE invocations. Indeed, another protocol of Doerner et al. [DKLsT23] proposes exactly this, and proves that in order to achieve security against malicious

[30] This list of citations is not exhaustive. See [DKLs24] for an overview of the history of the problem and a comparison of the various approaches.

[31] By comparison, most other approaches involve other substantive costs due to the use of zero-knowledge techniques, homomorphic encryption, etc.

[32] The original Silent OT protocol and its early successors required an expensive setup protocol, limiting their usefulness in the non-amortized setting, but recent advancements in public-key pseudorandom correlation functions [BCMPR24] have effectively mitigated this cost.

adversaries it is necessary only to use a malicious-secure VOLE and verify the signature afterward. As in the case of the threshold ECDSA protocol described above, VOLE invocation dominates the costs of this protocol, but because BBS+ requires a pairing curve, and pairing curves are typically defined over somewhat larger fields relative to non-pairing curves, given a particular security level, we achieve a noticeably greater bandwidth savings using our protocol than in the ECDSA case. We computed concrete costs using the same protocol combinations as we did for ECDSA, and reported them in Table 8.3.

VOLE Protocol	OT Extension Protocol	KB Sent Per Signer	Rounds
DKLs24	SoftSpoken	$(N-1)\cdot 54.0$	2
DKLs24	Silent	$(N-1)\cdot 49.6$	2
Ours (Pure OT)	SoftSpoken	$(N-1)\cdot 23.9$	10
Ours (Pure OT)	Silent	$(N-1)\cdot 9.6$	10
Ours (RSA)	SoftSpoken	$(N-1)\cdot 17.0$	10
Ours (RSA)	Silent	$(N-1)\cdot 3.1$	10

Table 8.3: Signer-to-Signer Bandwidth and Round Counts for an instance of the BBS+ signing protocol of Doerner et al. [DKLsT23] involving N signers, with $\kappa_{\mathsf{s}} = 40$, $\kappa_{\mathsf{c}} = 128$, and a 384-bit elliptic curve. Note that we exclude communication costs between the signers and the client, which do not depend upon the specifics of VOLE.

8.3 Authenticated Beaver Triples

Generic arithmetic multiparty computation can be achieved information-theoretically among N parties given a specific form of correlated randomness known as Beaver Triples [Bea91]. Each triple is a trio of linearly-secret-shared values $(a, b, c,)$ such that $c = a \cdot b$. Later, when two linearly-shared values x and y are supplied, each i^{th} party, in possession of the shares a_i, b_i, c_i, x_i, y_i computes $a'_i \leftarrow x_i - a_i$ and $b'_i \leftarrow y_i - b_i$ and broadcasts these two values, whereupon it becomes possible to compute $z_i \leftarrow c_i + a_i \sum_{j\in[N]} b'_j + b_i \sum_{j\in[N]} a'_j$. It is easy to see that z is a linearly shared product of x and y, and thus the parties have computed a multiplication gate. To achieve security against malicious adversaries, Beaver triples can be *authenticated* using a distributed information-theoretic MAC, as first proposed by Bendlin et al. [BDOZ11] and optimized shortly thereafter by Damgard et al. [DPSZ12]. Given such techniques, it remains only to sample the necessary correlations.

Keller, Orsini, and Scholl [KOS16] proposed the MASCOT protocol for sampling authenticated Beaver triples in the OT-hybrid model. Their protocol is formulated *directly* in the OT-hybrid model, without making use of an OLE abstraction, but it is easy to reformulate it in the OLE-hybrid model in retrospect, and indeed this was recently done by Cohen et al. [CDKs24] (in the context of achieving enhanced security guarantees). According to their formulation, the parties must evaluate a single VOLE instance with a vector length of five times the number of triples to be generated, and additionally they must evaluate one OLE instance per triple. These (V)OLE instances dominate the overall cost, and therefore our protocol yields a significant performance improvement over the original MASCOT protocol and over the reformulation of Cohen et al.

Acknowledgments. We thank Nico Döttling for helpful discussions on general constructions of VOLE from OLE, Hugo Krawczyk and Yibin Yang for helpful discussions on the OPRF application, Amir Shpilka for helpful discussions on Conjecture 5.37, and the anonymous referees for their suggestions. We also thank Michael Adjedj for running experimental benchmarks and for discussions on concrete performance. I. Haitner was supported by ISF grant 836/23. Y. Ishai was supported by ISF grant 3527/24, BSF grant 2022370, and ISF-NSFC grant 3127/23. J. Doerner was supported by the Azrieli Foundation and the Brown University Data Science Institute.

References

[ABCJM24] Michael Adjedj et al. "Two-Round 2PC ECDSA at the Cost of 1 OLE". In: *IACR Cryptol. ePrint Arch.* (2024), p. 1950. URL: https://eprint.iacr.org/2024/1950 (cit. on pp. 4, 5).

[ADDG24] Martin R. Albrecht et al. "Crypto Dark Matter on the Torus - Oblivious PRFs from Shallow PRFs and TFHE". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*. 2024, pp. 447–476 (cit. on p. 55).

[ADDS21] Martin R. Albrecht et al. "Round-Optimal Verifiable Oblivious Pseudorandom Functions from Ideal Lattices". In: *IACR International Conference on Practice and Theory of Public-Key Cryptography (PKC)*. 2021, pp. 261–289 (cit. on p. 55).

[ALSZ15] Gilad Asharov et al. "More Efficient Oblivious Transfer Extensions with Security for Malicious Adversaries". In: *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*. 2015 (cit. on pp. 1, 5).

[ANOSS22] Damiano Abram et al. "Low-Bandwidth Threshold ECDSA via Pseudorandom Correlation Generators". In: *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*. 2022 (cit. on p. 58).

[APRR24] Navid Alamati et al. "Improved Alternating-Moduli PRFs and Post-quantum Signatures". In: *Annual International Cryptology Conference (CRYPTO)*. 2024, pp. 274–308 (cit. on pp. 55, 56).

[ASM06] Man Ho Au et al. "Constant-Size Dynamic k-TAA". In: *Proceedings of the 5th Conference on Security and Cryptography for Networks (SCN)*. Ed. by Roberto De Prisco and Moti Yung. 2006, pp. 111–125 (cit. on p. 58).

[Bas23] Andrea Basso. "A Post-Quantum Round-Optimal Oblivious PRF from Isogenies". In: *SAC*. 2023, pp. 147–168 (cit. on p. 55).

[BB89] Judit Bar-Ilan and Donald Beaver. "Non-cryptographic fault-tolerant computing in constant number of rounds of interaction". In: *Proceedings of the 8th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. 1989 (cit. on p. 57).

[BBCCDS24] Maxime Bombar et al. "FOLEAGE: $\mathbb {F}_{\scriptstyle 4}$OLE-Based Multiparty Computation for Boolean Circuits". In: *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VI*. Ed. by Kai-Min Chung and Yu Sasaki. Vol. 15489. Lecture Notes in Computer Science. Springer, 2024, pp. 69–101. DOI: 10.1007/978-981-96-0938-3_3. URL: https://doi.org/10.1007/978-981-96-0938-3_3 (cit. on pp. 1, 5).

[BBSGKORS23] Carsten Baum et al. "Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head". In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14085. Lecture Notes in Computer Science. Springer, 2023, pp. 581–615. DOI: 10.1007/978-3-031-38554-4_19. URL: https://doi.org/10.1007/978-3-031-38554-4_19 (cit. on p. 55).

[BCGI18] Elette Boyle et al. "Compressing Vector OLE". In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2018, pp. 896–912 (cit. on p. 69).

[BCGIKS19] Elette Boyle et al. "Efficient Pseudorandom Correlation Generators: Silent OT Extension and More". In: *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part III*. 2019 (cit. on pp. 1, 5, 16, 58).

[BCGIKS20] Elette Boyle et al. "Correlated Pseudorandom Functions from Variable-Density LPN". In: *61st IEEE Annual Symposium on Foundations of Computer Science, FOCS 2020, Durham, NC, USA, November 16-19, 2020*. 2020 (cit. on pp. 1, 5).

[BCMPR24] Dung Bui et al. "Fast Public-Key Silent OT and More from Constrained Naor-Reingold". In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI*. 2024 (cit. on pp. 1, 3, 5, 58).

[BDFH24] Ward Beullens et al. *The 2Hash OPRF Framework and Efficient Post-Quantum Instantiations*. Tech. rep. 450/2024. Cryptology ePrint Archive, 2024 (cit. on pp. 55, 56).

[BDOZ11] Rikke Bendlin et al. "Semi-homomorphic Encryption and Multiparty Computation". In: *Advances in Cryptology - EUROCRYPT 2011 - 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings*. 2011 (cit. on p. 59).

[Bea91] Donald Beaver. "Efficient Multiparty Protocols Using Circuit Randomization". In: *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings*. Ed. by Joan Feigenbaum. Lecture Notes in Computer Science. 1991 (cit. on pp. 1, 59).

[Bea96] Donald Beaver. "Correlated Pseudorandomness and the Complexity of Private Computations". In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*. 1996 (cit. on pp. 1, 5).

[BEPST20] Carsten Baum et al. "Efficient Protocols for Oblivious Linear Function Evaluation from Ring-LWE". In: *Security and Cryptography for Networks (SCN)*. 2020, pp. 130–149 (cit. on p. 1).

[BGG17] Dan Boneh et al. "Using Level-1 Homomorphic Encryption to Improve Threshold DSA Signatures for Bitcoin Wallet Security". In: *Progress in Cryptology – LATINCRYPT 2017*. 2017. DOI: 10.1007/978-3-030-25283-0_19 (cit. on p. 58).

[BHILM20] Marshall Ball et al. "On the Complexity of Decomposable Randomized Encodings, Or: How Friendly Can a Garbling-Friendly PRF Be?" In: *ACM Conference on Innovations in Theoretical Computer Science (ITCS)*. 2020 (cit. on p. 56).

[BIPSW18] Dan Boneh et al. "Exploring Crypto Dark Matter: - New Simple PRF Candidates and Their Applications". In: *Theory of Cryptography (TCC)*. 2018, pp. 699–729 (cit. on p. 55).

[BKW20] Dan Boneh et al. "Oblivious Pseudorandom Functions from Isogenies". In: *Annual International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*. Springer, 2020, pp. 520–550 (cit. on p. 55).

[BSG20] Ward Beullens and Cyprien Delpech de Saint Guilhem. "LegRoast: Efficient Post-quantum Signatures from the Legendre PRF". In: *PQCrypto*. 2020, pp. 130–150 (cit. on p. 56).

[Can00] Ran Canetti. "Security and Composition of Multiparty Cryptographic Protocols". In: *Journal of Cryptology* 13.1 (2000), pp. 143–202 (cit. on p. 1).

[Can01] Ran Canetti. "Universally Composable Security: A New Paradigm for Cryptographic Protocols". In: *Annual Symposium on Foundations of Computer Science (FOCS)*. 2001, pp. 136–145 (cit. on pp. 1, 13).

[CCLST19] Guilhem Castagnos et al. "Two-Party ECDSA from Hash Proof Systems and Efficient Instantiations". In: *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part III*. Ed. by Alexandra Boldyreva and Daniele Micciancio. Vol. 11694. Lecture Notes in Computer Science. Springer, 2019, pp. 191–221. DOI: 10.1007/978-3-030-26954-8_7. URL: https://doi.org/10.1007/978-3-030-26954-8_7 (cit. on pp. 4, 5).

[CCLST20] Guilhem Castagnos et al. “Bandwidth-Efficient Threshold EC-DSA”. In: *Proceedings of the 23rd International Conference on the Theory and Practice of Public-Key Cryptography (PKC), part II*. 2020 (cit. on pp. 4, 5, 58).

[CCLST23] Guilhem Castagnos et al. “Bandwidth-efficient threshold EC-DSA revisited: Online/offline extensions, identifiable aborts proactive and adaptive security”. In: *Theoretical Computer Science* 939 (2023) (cit. on p. 58).

[CDDKS24] Geoffroy Couteau et al. “QuietOT: Lightweight Oblivious Transfer with a Public-Key Setup”. In: *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part II*. Ed. by Kai-Min Chung and Yu Sasaki. Vol. 15485. Lecture Notes in Computer Science. Springer, 2024, pp. 197–231. DOI: 10.1007/978-981-96-0888-1_7. URL: https://doi.org/10.1007/978-981-96-0888-1_7 (cit. on pp. 1, 3, 5).

[CDKs24] Ran Cohen et al. “Secure Multiparty Computation with Identifiable Abort via Vindicating Release”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VIII*. 2024 (cit. on p. 59).

[CGGMP20] Ran Canetti et al. “UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts”. In: *Proceedings of the 27th ACM Conference on Computer and Communications Security (CCS)*. 2020 (cit. on pp. 4, 5, 58).

[CHIVV22] Leo de Castro et al. “Asymptotically Quasi-Optimal Cryptography”. In: *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part I*. Ed. by Orr Dunkelman and Stefan Dziembowski. Vol. 13275. Lecture Notes in Computer Science. Springer, 2022, pp. 303–334. DOI: 10.1007/978-3-031-06944-4_11. URL: https://doi.org/10.1007/978-3-031-06944-4_11 (cit. on p. 1).

[CHL22] Sílvia Casacuberta et al. “SoK: Oblivious Pseudorandom Functions”. In: *EuroS*. 2022, pp. 625–646 (cit. on p. 55).

[CJV21] Leo de Castro et al. “Fast Vector Oblivious Linear Evaluation from Ring Learning with Errors”. In: *WAHC ’21: Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography, Virtual Event, Korea, 15 November 2021*. WAHC@ACM, 2021, pp. 29–41. DOI: 10.1145/3474366.3486928. URL: https://doi.org/10.1145/3474366.3486928 (cit. on p. 1).

[CKKLMS24] Leo de Castro et al. *More Efficient Lattice-based OLE from Circuit-private Linear HE with Polynomial Overhead*. Cryptology ePrint Archive, Paper 2024/1534. 2024. URL: https://eprint.iacr.org/2024/1534 (cit. on p. 1).

[CMI93] Manuel Cerecedo et al. “Efficient and Secure Multiparty Generation of Digital Signatures Based on Discrete Logarithms”. In: *IEICE TRANSACTIONS on Fundamentals of Electronics, Communications and Computer Sciences, Vol.E76-A, No.4, pp.532-545*. 1993 (cit. on p. 58).

[Dam88] Ivan Damgard. “On the Randomness of Legendre and Jacobi Sequences”. In: *Annual International Cryptology Conference (CRYPTO)*. Ed. by Shafi Goldwasser. 1988, pp. 163–172 (cit. on pp. 3, 56).

[DGHIKSZ21] Itai Dinur et al. “MPC-Friendly Symmetric Cryptography from Alternating Moduli: Candidates, Protocols, and Applications”. In: *Annual International Cryptology Conference (CRYPTO)*. 2021, pp. 517–547 (cit. on pp. 55, 56).

[DH00] Wim van Dam and Sean Hallgren. "Efficient Quantum Algorithms for Shifted Quadratic Character Problems". In: *CoRR* (2000) (cit. on p. 56).

[DHIM25] Jack Doerner et al. "From OT to OLE with Subquadratic Communication". In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025*. ACM, 2025 (cit. on p.).

[DJNPØ20] Ivan Damgard et al. "Fast Threshold ECDSA with Honest Majority". In: *Proceedings of the 12th Conference on Security and Cryptography for Networks (SCN)*. 2020 (cit. on p. 58).

[DKLs18] Jack Doerner et al. "Secure Two-party Threshold ECDSA from ECDSA Assumptions". In: *Proceedings of the 39th IEEE Symposium on Security and Privacy (S&P)*. 2018 (cit. on pp. 1, 2, 6, 58).

[DKLs19] Jack Doerner et al. "Threshold ECDSA from ECDSA Assumptions: The Multiparty Case". In: *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 1051–1066. DOI: 10.1109/SP.2019.00024 (cit. on pp. 2, 58).

[DKLs24] Jack Doerner et al. "Threshold ECDSA in Three Rounds". In: *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P)*. 2024 (cit. on pp. 1–4, 6, 58).

[DKLsT23] Jack Doerner et al. "Threshold BBS+ Signatures for Distributed Anonymous Credential Issuance". In: *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P)*. 2023 (cit. on pp. 3, 58, 59).

[DKM12] Nico Döttling et al. "Statistically Secure Linear-Rate Dimension Extension for Oblivious Affine Function Evaluation". In: *Information Theoretic Security - 6th International Conference, ICITS 2012, Montreal, QC, Canada, August 15-17, 2012. Proceedings*. Ed. by Adam D. Smith. Vol. 7412. Lecture Notes in Computer Science. Springer, 2012, pp. 111–128. DOI: 10.1007/978-3-642-32284-6_7. URL: https://doi.org/10.1007/978-3-642-32284-6_7 (cit. on p. 3).

[DOKSS20] Anders P. K. Dalskov et al. "Securing DNSSEC Keys via Threshold ECDSA from Generic MPC". In: *Proceedings of the 25th European Symposium on Research in Computer Security (ESORICS), Part II*. 2020 (cit. on p. 58).

[DPSZ12] Ivan Damgard et al. "Multiparty Computation from Somewhat Homomorphic Encryption". In: *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*. 2012 (cit. on p. 59).

[EGL85] Shimon Even et al. "A Randomized Protocol for Signing Contracts." In: *Communications of the ACM* 28.6 (1985), pp. 637–647 (cit. on p. 1).

[FIPR05] Michael J. Freedman et al. "Keyword Search and Oblivious Pseudorandom Functions". In: *Theory of Cryptography (TCC)*. 2005, pp. 303–324 (cit. on p. 55).

[Fis05] Marc Fischlin. "Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors". In: *Annual International Cryptology Conference (CRYPTO)*. 2005, pp. 152–168 (cit. on p. 50).

[FKN94] Uriel Feige et al. "A minimal model for secure computation (extended abstract)". In: *Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1994, pp. 554–563 (cit. on p. 56).

[FO97] Eiichiro Fujisaki and Tatsuaki Okamoto. "Statistical Zero Knowledge Protocols to Prove Modular Polynomial Relations". In: *Annual International Cryptology Conference (CRYPTO)*. 1997, pp. 16–30 (cit. on p. 48).

[GG18] Rosario Gennaro and Steven Goldfeder. "Fast Multiparty Threshold ECDSA with Fast Trustless Setup". In: *Proceedings of the 25th ACM Conference on Computer and Communications Security (CCS)*. 2018 (cit. on pp. 4, 5, 58).

[GGN16] Rosario Gennaro et al. "Threshold-Optimal DSA/ECDSA Signatures and an Application to Bitcoin Wallet Security". In: *Proceedings of the 14th International Conference on Applied Cryptography and Network Security (ACNS)*. 2016 (cit. on p. 58).

[Gil99] Niv Gilboa. "Two Party RSA Key Generation". In: *Annual International Cryptology Conference (CRYPTO)*. 1999, pp. 116–129 (cit. on pp. 1, 4, 6, 15).

[GJKR96] Rosario Gennaro et al. "Robust Threshold DSS Signatures". In: *Advances in Cryptology – EUROCRYPT 1996*. 1996 (cit. on p. 58).

[GMW87] Oded Goldreich et al. "How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority". In: *Annual ACM Symposium on Theory of Computing (STOC)*. 1987, pp. 218–229 (cit. on p. 1).

[GNN17] Satrajit Ghosh et al. "Maliciously Secure Oblivious Linear Function Evaluation with Constant Overhead". In: *Annual International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*. 2017, pp. 629–659 (cit. on p. 1).

[GRRSS16] Lorenzo Grassi et al. "MPC-Friendly Symmetric Key Primitives". In: *SIGSAC*. 2016, pp. 430–443 (cit. on pp. 55, 56).

[GRS99] Oded Goldreich et al. "Chinese remaindering with errors". In: *Annual ACM Symposium on Theory of Computing (STOC)*. 1999, pp. 225–234 (cit. on p. 27).

[GS22] Jens Groth and Victor Shoup. *Design and analysis of a distributed ECDSA signing service*. Cryptology ePrint Archive, Paper 2022/506. 2022. URL: https://eprint.iacr.org/2022/506 (cit. on p. 58).

[HH22] David Harvey and Joris van der Hoeven. "Polynomial Multiplication over Finite Fields in Time \(O (n \log n) \)". In: *J. ACM* 69.2 (2022), 12:1–12:40. DOI: 10.1145/3505584. URL: https://doi.org/10.1145/3505584 (cit. on p. 1).

[HIMV19] Carmit Hazay et al. "LevioSA: Lightweight Secure Arithmetic Computation". In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*. Ed. by Lorenzo Cavallaro et al. ACM, 2019, pp. 327–344. DOI: 10.1145/3319535.3354258. URL: https://doi.org/10.1145/3319535.3354258 (cit. on p. 1).

[HL09] Carmit Hazay and Yehuda Lindell. *Efficient Oblivious Polynomial Evaluation with Simulation-Based Security*. Cryptology ePrint Archive, Paper 2009/459. 2009. URL: https://eprint.iacr.org/2009/459 (cit. on p. 5).

[HLM25] Iftach Haitner et al. *Integer Commitments, Old and New Tools*. Tech. rep. 81/2025. Cryptology ePrint Archive, 2025 (cit. on pp. 48–51).

[HLNR23] Iftach Haitner et al. *Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody*. Cryptology ePrint Archive, Paper 2018/987, Version 20230529:135032. 2023. URL: https://eprint.iacr.org/archive/2018/987/20230529:135032 (cit. on pp. 1, 58).

[HMRT22] Iftach Haitner et al. "Highly Efficient OT-Based Multiplication Protocols". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*. 2022, pp. 180–209 (cit. on pp. 2, 4, 6, 27).

[IKNP03] Yuval Ishai et al. "Extending Oblivious Transfers Efficiently". In: *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*. 2003 (cit. on pp. 1, 5).

[IKOPS11] Yuval Ishai et al. “Efficient Non-interactive Secure Computation”. In: *Advances in Cryptology - EUROCRYPT 2011 - 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings*. Ed. by Kenneth G. Paterson. Vol. 6632. Lecture Notes in Computer Science. Springer, 2011, pp. 406–425. DOI: 10.1007/978-3-642-20465-4_23. URL: https://doi.org/10.1007/978-3-642-20465-4_23 (cit. on p. 1).

[IPS08] Yuval Ishai et al. “Founding Cryptography on Oblivious Transfer - Efficiently”. In: *Advances in Cryptology - CRYPTO 2008, 28th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2008. Proceedings*. Ed. by David A. Wagner. Vol. 5157. Lecture Notes in Computer Science. Springer, 2008, pp. 572–591. DOI: 10.1007/978-3-540-85174-5_32. URL: https://doi.org/10.1007/978-3-540-85174-5_32 (cit. on p. 1).

[IPS09] Yuval Ishai et al. “Secure Arithmetic Computation with No Honest Majority”. In: *Theory of Cryptography (TCC)*. 2009, pp. 294–314 (cit. on pp. 1, 55).

[JKK14] Stanislaw Jarecki et al. “Round-Optimal Password-Protected Secret Sharing and T-PAKE in the Password-Only Model”. In: *Annual International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*. 2014, pp. 233–253 (cit. on pp. 55, 56).

[JVC18] Chiraag Juvekar et al. “GAZELLE: A Low Latency Framework for Secure Neural Network Inference”. In: *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*. Ed. by William Enck and Adrienne Porter Felt. USENIX Association, 2018, pp. 1651–1669. URL: https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar (cit. on p. 1).

[KCM24] Novak Kaluderovic et al. *A post-quantum Distributed OPRF from the Legendre PRF*. Tech. rep. 544/2024. Cryptology ePrint Archive, 2024 (cit. on p. 55).

[Kil88] Joe Kilian. “Founding Cryptography on Oblivious Transfer”. In: *Annual ACM Symposium on Theory of Computing (STOC)*. 1988, pp. 20–31 (cit. on p. 1).

[KLR10] Eyal Kushilevitz et al. “Information-Theoretically Secure Protocols and Security under Composition”. In: *SIAM Journal on Computing* 39.5 (2010), pp. 2090–2112 (cit. on p. 13).

[KOS15] Marcel Keller et al. “Actively Secure OT Extension with Optimal Overhead”. In: *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*. 2015 (cit. on pp. 1, 5).

[KOS16] Marcel Keller et al. “MASCOT: Faster Malicious Arithmetic Secure Computation with Oblivious Transfer”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2016, pp. 830–842 (cit. on pp. 1, 2, 6, 59).

[KPRR25] Vladimir Kolesnikov et al. “Stationary Syndrome Decoding for Improved PCGs”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 295. URL: https://eprint.iacr.org/2025/295 (cit. on pp. 1, 5).

[Lin17] Yehuda Lindell. “Fast Secure Two-Party ECDSA Signing”. In: *Advances in Cryptology – CRYPTO 2017, part II*. 2017 (cit. on pp. 4, 5, 58).

[LN18] Yehuda Lindell and Ariel Nof. “Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2018, pp. 1837–1854 (cit. on p. 58).

[LXYY25] Zhe Li et al. “Efficient Pseudorandom Correlation Generators for Any Finite Field”. In: *IACR Cryptol. ePrint Arch.* (2025). To appear in Eurocrypt 2025, p. 169. URL: https://eprint.iacr.org/2025/169 (cit. on pp. 1, 5).

[MR01] Philip D. MacKenzie and Michael K. Reiter. "Two-Party Generation of DSA Signatures". In: *Advances in Cryptology – CRYPTO 2001*. 2001 (cit. on p. 58).

[NP06] Moni Naor and Benny Pinkas. "Oblivious Polynomial Evaluation". In: *SIAM Journal on Computing* 35.5 (2006), pp. 1254–1281 (cit. on p. 1).

[NR04] Moni Naor and Omer Reingold. "Number-theoretic constructions of efficient pseudo-random functions". In: *Journal of the ACM* (2004) (cit. on p. 55).

[Rab81] M. O. Rabin. *How to Exchange Secrets by Oblivious Transfer*. TR-81, Harvard. 1981 (cit. on p. 1).

[Roy22] Lawrence Roy. "SoftSpokenOT: Quieter OT Extension from Small-Field Silent VOLE in the Minicrypt Model". In: *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part I*. 2022 (cit. on pp. 1, 5, 58).

[RRT23] Srinivasan Raghuraman et al. "Expand-Convolute Codes for Pseudorandom Correlation Generators from LPN". In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part IV*. 2023 (cit. on pp. 1, 5).

[RS62] J Barkley Rosser and Lowell Schoenfeld. "Approximate formulas for some functions of prime numbers". In: *Illinois Journal of Mathematics* 6.1 (1962), pp. 64–94 (cit. on p. 11).

[RT90] John H. Reif and Stephen R. Tate. "Optimal Size Integer Division Circuits". In: *SIAM J. Comput.* 19.5 (1990), pp. 912–924. DOI: 10.1137/0219064. URL: https://doi.org/10.1137/0219064 (cit. on p. 1).

[Sho06] Victor Shoup. *A computational introduction to number theory and algebra*. Cambridge University Press, 2006. ISBN: 978-0-521-85154-1 (cit. on p. 12).

[ST19] Nigel P. Smart and Younes Talibi Alaoui. "Distributing Any Elliptic Curve Based Protocol". In: *IMA International Conference on Cryptography and Coding*. 2019 (cit. on p. 58).

[YBHKR25] Yibin Yang et al. "Gold OPRF: Post-Quantum Oblivious Power-Residue PRF". In: *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*. Ed. by Marina Blanton et al. IEEE, 2025, pp. 259–278. DOI: 10.1109/SP61157.2025.00116. URL: https://doi.org/10.1109/SP61157.2025.00116 (cit. on pp. 3, 56).

[YWLZW20] Kang Yang et al. "Ferret: Fast Extension for Correlated OT with Small Communication". In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by Jay Ligatti et al. ACM, 2020, pp. 1607–1626. DOI: 10.1145/3372297.3417276. URL: https://doi.org/10.1145/3372297.3417276 (cit. on pp. 1, 16).

A Discussion of Concrete Performance

We present concrete costs for our protocols. To realize the OLE functionality over a large prime q, our protocols implement a variant of OLE in the OT-hybrid model over the divisors of a large modulus n, where n is the product of the first t primes. In the tables below, we describe (i) The minimal number of primes t required to obtain n (ii) The communication complexity in kilobytes (KB) in the *random-OT (ROT)* hybrid model (iii) The total number of OTs consumed by the protocol. For comparison, we also provide costs for Gilboa's protocol over q (and we note that $n = q$ in for this protocol).

For statistical parameter κ_s, the size of the modulus is $n \approx q^2 \cdot 2^{\kappa_s}$ for the semi-honest (CRT-SH) protocol, and $n \approx q^2 \cdot 2^{5\kappa_s}$ for the malicious receiver (CRT-MR) protocol. For the fully malicious protocol (CRT-FM), the size of n satisfies $n'/\log(n) \approx q^2 \cdot 2^{12\kappa_s}$, where n' is the product of all but the largest κ_s factors of n.

A.1 OLE

Statistical parameter $\kappa_s = 40$. In the tables below, we present the main performance benchmarks with the statistical security parameter κ_s set to 40. All protocols achieve concrete security of at least $2^{-\kappa_s}$, with no hidden constants.

$\|q\|$	Gilboa-MR	CRT-SH	CRT-MR	CRT-FM
32	1	23	49	141
64	1	32	56	143
128	1	50	72	150
256	1	80	101	177
384	1	109	128	204
512	1	136	155	229

Table A.1: Number of primes (invocations of Gilboa's protocol modulo a prime) when $\kappa_s = 40$.

$\|q\|$	Gilboa-MR	CRT-SH	CRT-MR	CRT-FM-RSA	CRT-FM-OT
32	0.13	0.08	0.28	$1.30 + c_{\text{rsa}}$	$1.30 + 4.33$
64	0.51	0.14	0.34	$1.33 + c_{\text{rsa}}$	$1.33 + 4.33$
128	2.05	0.29	0.50	$1.42 + c_{\text{rsa}}$	$1.42 + 4.70$
256	8.19	0.58	0.80	$1.77 + c_{\text{rsa}}$	$1.77 + 6.01$
384	18.43	0.90	1.14	$2.17 + c_{\text{rsa}}$	$2.17 + 7.34$
512	32.77	1.24	1.48	$2.55 + c_{\text{rsa}}$	$2.55 + 8.68$

Table A.2: Communication Cost (in KB) given random OT setup when $\kappa_s = 40$. Here, c_{rsa} denotes the additional cost between 0.52KB and 0.77KB, depending on the target security level; see Remark A.3. The communication cost in the table essentially corresponds to the total size of the sender's input in the OTs, i.e., $\sum_{i=1}^{t} \lceil \log(n_i) \rceil^2$, where $n_1, \ldots, n_t$ denote the divisors of n, plus the overhead for the commit-and-prove functionality.

Remark A.3. Additional costs for the commit-and-prove functionality (instantiated with RSA). Our fully malicious protocol requires us to instantiate a commit-and-prove functionality for a specific relation. Doing so via RSA groups adds approximately $c_{\text{rsa}} = 2n_{\text{rsa}} + \kappa_s$ bits of communication where n_{rsa} is the size of the modulus. This amounts to 0.52KB for the commonly accepted 112-bit level of computational security ($n_{\text{rsa}} = 2048$), or 0.77KB for 128-bit level of security ($n_{\text{rsa}} = 3072$). Computationally, the RSA-based commit-and-prove functionality adds 4 exponentiations in the RSA group for the sender, and two for the receiver.

$\|q\|$	Gilboa-MR	CRT-SH	CRT-MR	CRT-FM-RSA	CRT-FM-OT
32	32	119	319	1186	1268
64	64	183	377	1206	1288
128	128	327	521	1276	1358
256	256	593	786	1551	1633
384	384	866	1056	1848	1930
512	512	1136	1326	2123	2205

Table A.4: Number of underlying OTs $\approx \log(n)$ when $\kappa_s = 40$.

Statistical parameter $\kappa_s = 80$. In the tables below, we present the main performance benchmarks with the statistical security parameter κ_s set to 80.

$\|q\|$	Gilboa-MR	CRT-SH	CRT-MR	CRT-FM
32	1	29	79	233
64	1	38	82	233
128	1	55	95	238
256	1	85	122	258
384	1	113	149	284
512	1	140	175	308

Table A.5: Number of primes (invocations of Gilboa's protocol modulo a prime) when $\kappa_{\mathsf{s}} = 80$.

$\|q\|$	Gilboa-MR	CRT-SH	CRT-MR	CRT-FM-RSA	CRT-FM-OT
32	0.13	0.12	0.57	$2.61 + c_{\text{rsa}}$	$2.61 + 15.10$
64	0.51	0.19	0.60	$2.61 + c_{\text{rsa}}$	$2.61 + 15.10$
128	2.05	0.33	0.73	$2.69 + c_{\text{rsa}}$	$2.69 + 15.10$
256	8.19	0.63	1.07	$2.99 + c_{\text{rsa}}$	$2.99 + 16.81$
384	18.43	0.95	1.40	$3.38 + c_{\text{rsa}}$	$3.38 + 19.42$
512	32.77	1.29	1.74	$3.75 + c_{\text{rsa}}$	$3.75 + 22.01$

Table A.6: Communication Cost (in KB) given random OT setup when $\kappa_{\mathsf{s}} = 80$.

$\|q\|$	Gilboa-MR	CRT-SH	CRT-MR	CRT-FM-RSA	CRT-FM-OT
32	32	161	584	2167	2249
64	64	231	611	2167	2249
128	128	368	728	2222	2304
256	256	638	996	2442	2524
384	384	906	1266	2728	2810
512	512	1176	1529	2992	3074

Table A.7: Number of underlying OTs $\approx \log(n)$ when $\kappa_{\mathsf{s}} = 80$.

A.2 VOLE

We present the communication benchmarks for VOLE of length m with the statistical security parameter κ_{s} set to 40 and 80 for $m \in \{2, 3, 5, 10\}$. The main table value reports the cost per underlying OLE (i.e., per vector entry), and it does not account for the commit-and-prove functionality. The value in parentheses reports the *total* improvement (including commit-and-prove) over the generic construction that realizes a length-m VOLE using m+1 OLEs. The choice of commit-and-prove (RSA or OT-only) does not materially affect this estimate. See Appendix B for the description of the generic reduction.

$m \setminus \|q\|$	32	64	128	256	384	512
2	1.33 (~ 0.67)	1.35 (~ 0.67)	1.44 (~ 0.67)	1.80 (~ 0.67)	2.19 (~ 0.67)	2.58 (~ 0.67)
3	1.33 (~ 0.76)	1.37 (~ 0.76)	1.45 (~ 0.76)	1.81 (~ 0.76)	2.20 (~ 0.76)	2.60 (~ 0.76)
5	1.35 (~ 0.85)	1.38 (~ 0.85)	1.47 (~ 0.85)	1.83 (~ 0.85)	2.23 (~ 0.84)	2.61 (~ 0.84)
10	1.40 (~ 0.94)	1.44 (~ 0.94)	1.53 (~ 0.94)	1.89 (~ 0.94)	2.28 (~ 0.93)	2.67 (~ 0.93)

Table A.8: Amortized communication (KB) for $\kappa_{\mathsf{s}} = 40$. Improvements over the generic protocol are shown in parentheses.

$m \setminus \lvert q \rvert$	32	64	128	256	384	512
2	2.64 ($\sim$ 0.67)	2.64 ($\sim$ 0.67)	2.70 ($\sim$ 0.67)	3.02 ($\sim$ 0.67)	3.40 ($\sim$ 0.67)	3.78 ($\sim$ 0.67)
3	2.66 ($\sim$ 0.76)	2.66 ($\sim$ 0.76)	2.72 ($\sim$ 0.75)	3.04 ($\sim$ 0.76)	3.41 ($\sim$ 0.75)	3.80 ($\sim$ 0.75)
5	2.67 ($\sim$ 0.84)	2.67 ($\sim$ 0.84)	2.75 ($\sim$ 0.84)	3.05 ($\sim$ 0.84)	3.43 ($\sim$ 0.84)	3.82 ($\sim$ 0.84)
10	2.73 ($\sim$ 0.93)	2.73 ($\sim$ 0.93)	2.81 ($\sim$ 0.93)	3.11 ($\sim$ 0.93)	3.49 ($\sim$ 0.92)	3.89 ($\sim$ 0.93)

Table A.9: Amortized communication (KB) for $\kappa_{\mathsf{s}} = 80$. Improvements over the generic protocol are shown in parentheses.

B Reducing VOLE to OLE

Recall that in Section 5.4 we extend our OT-based OLE protocol to VOLE. In this section, we present a *general* information-theoretic reduction from VOLE to OLE.

Recall that a length-m VOLE takes vectors $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{Z}_q^m$ from the sender and a scalar $x \in \mathbb{Z}_q$ from the receiver and outputs $\boldsymbol{y} = \boldsymbol{a}x + \boldsymbol{b}$ to the receiver. We will realize a *random* VOLE functionality, hereafter RVOLE, with the following two differences: (1) a malicious party can choose its own output, (2) the honest party receives an output sampled from the random VOLE conditional distribution, or alternatively $\perp$. The RVOLE functionality can be used to realize chosen-input VOLE via a simple and efficient two-round protocol [BCGI18].

Functionality B.1 (Random vector-OLE $\mathsf{RVOLE}_{\mathsf{q,m}}$)**.**

Parties: S and R.

Parameters: Modulus $q \in \mathbb{N}$ and vector length 1^m.

S's input: No input for an honest S. A malicious $\widehat{\mathsf{S}}$ has inputs $\hat{\boldsymbol{a}}, \hat{\boldsymbol{b}} \in \mathbb{Z}_q^m$.

R's input: No input for an honest R. A malicious $\widehat{\mathsf{R}}$ has inputs $\widehat{x} \in \mathbb{Z}_q, \hat{\boldsymbol{y}} \in \mathbb{Z}_q^m$.

Operation: If both S and R are honest, sample $\boldsymbol{a}, \boldsymbol{b} \xleftarrow{\text{R}} \mathbb{Z}_q^m$ and $x \xleftarrow{\text{R}} \mathbb{Z}_q$, let $\boldsymbol{y} = \boldsymbol{a}x + \boldsymbol{b} \in \mathbb{Z}_q^m$, and deliver $(\boldsymbol{a}, \boldsymbol{b})$ to S and $(x, \boldsymbol{y})$ to R. If one of the parties is malicious, allow this party to choose its output, and then choose whether the other party gets an output chosen from the ideal conditional distribution (defined by the above honest experiment) or outputs $\perp$.

B.1 Malicious-Receiver Reduction

From here on, we consider VOLE and OLE over a large prime modulus q, namely over a finite field $\mathbb{F} = \mathbb{F}_q$.

Towards realizing $\mathsf{RVOLE}_{\mathsf{q,m}}$ from $\mathsf{OLE}_{\mathsf{q}}$, we start with the easier case where only the receiver can be malicious. In this case, the idea is to make use $m + 1$ separate OLE calls in which both parties choose their inputs at random, except that an honest receiver is supposed to choose the same x in each call. Let $\tilde{\boldsymbol{a}}, \tilde{\boldsymbol{b}} \in \mathbb{F}^{m+1}$ be the sender's OLE inputs.

To ensure that the receiver uses the same x in each OLE call, the sender generates a random linear combination vector $\tilde{\boldsymbol{r}} \in \mathbb{F}^{m+1}$, and reveals to the receiver $\langle \tilde{\boldsymbol{b}}, \tilde{\boldsymbol{r}} \rangle$. This allows an honest receiver to learn $\langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle$, but a malicious receiver $\widehat{\mathsf{R}}$ who uses inconsistent OLE inputs $\widehat{x}_i$ can only guess this value with $O(1/q)$ probability. A malicious receiver can make the protocol abort depending on its output (which will be allowed by the functionality). Upon receiving the correct value from the receiver, the parties project their inputs to the dual space by letting $\boldsymbol{a} = M\tilde{\boldsymbol{a}}$, $\boldsymbol{b} = M\tilde{\boldsymbol{b}}$, $\boldsymbol{y} = M(\tilde{\boldsymbol{a}}x + \tilde{\boldsymbol{b}})$, where M is a public full-rank $m \times (m+1)$ matrix whose rows are orthogonal to $\tilde{\boldsymbol{r}}$.

We now describe a protocol that (statistically) realizes the above ideal functionality in the presence of a (potentially) malicious receiver and semi-honest server.

Protocol B.2 (Malicious-receiver $\mathsf{RVOLE}_{\mathsf{q},\mathsf{m}}$ from $\mathsf{OLE}_{\mathsf{q}}$)**.**

Parties: S and R.

Parameters: Prime $q \in \mathbb{N}$ and length 1^m. Let $\mathbb{F} = \mathbb{F}_q$.

Oracle: $\mathsf{OLE}_{\mathsf{q}}$

Operation:

1. S samples $\tilde{\boldsymbol{a}}, \tilde{\boldsymbol{b}} \xleftarrow{R} \mathbb{F}^{m+1}$, R samples nonzero $x \xleftarrow{R} \mathbb{F}$.
2. The parties make $m+1$ parallel OLE calls, where the inputs to call i are $(\tilde{a}_i, \tilde{b}_i)$ and x respectively. Let $\tilde{\boldsymbol{y}} \in \mathbb{F}^{m+1}$ be the OLE outputs.
3. S: Sample $\tilde{\boldsymbol{r}} \xleftarrow{R} \mathbb{F}^{m+1}$, and send $\tilde{\boldsymbol{r}}$ and $z^{\mathsf{S}} = \langle \tilde{\boldsymbol{b}}, \tilde{\boldsymbol{r}} \rangle$.
4. R: Send $t^{\mathsf{R}} \leftarrow x^{-1} \cdot (z^{\mathsf{S}} + \langle \tilde{\boldsymbol{y}}, \tilde{\boldsymbol{r}} \rangle)$.
5. S: Verify $t^{\mathsf{R}} = \langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle$, otherwise abort.
6. Let $M \in \mathbb{F}^{m \times (m+1)}$ be a full-rank matrix whose rows are orthogonal to $\tilde{\boldsymbol{r}}$.
7. S outputs $\boldsymbol{a} \leftarrow M \times \tilde{\boldsymbol{a}}$ and $\boldsymbol{b} \leftarrow M \times \tilde{\boldsymbol{b}}$; R outputs x and $\boldsymbol{y} \leftarrow M \times \tilde{\boldsymbol{y}}$.

We will now argue that when S is honest, the protocol ϵ-securely realizes the RVOLE functionality for $\epsilon = O(1/q)$.

It is easy to see that when both parties are honest, the verification succeeds and the outputs are correctly distributed, except that the choice of a nonzero x is $(1/q)$-far from uniform over $\mathbb{F}$. Moreover, by the choice of M, the outputs $\boldsymbol{a}, \boldsymbol{b}, x, \boldsymbol{y}$ are correctly distributed even when conditioned on the view of R or S.

We define the simulator $\mathsf{Sim}_{\widehat{\mathsf{R}}}$ for a malicious $\widehat{\mathsf{R}}$ (taking the role of R in the protocol) as follows.

Algorithm B.3 ($\mathsf{Sim}_{\widehat{\mathsf{R}}}$)**.**

Ideal functionality: $\mathsf{RVOLE}_{\mathsf{q},\mathsf{m}}$ (random length-m VOLE over $\mathbb{F} = \mathbb{F}_q$).

Helper oracle: $\mathsf{OLE}_{\mathsf{q}}$ (OLE over $\mathbb{F} = \mathbb{F}_q$).

Operation:

1. Let $\widehat{x} = (\widehat{x}_1, \ldots, \widehat{x}_{m+1})$ be the inputs $\widehat{\mathsf{R}}$ provides to the $\mathsf{OLE}_{\mathsf{q}}$ oracle.
2. Sample $\tilde{\boldsymbol{y}} \xleftarrow{R} \mathbb{F}^{m+1}$, and return $\tilde{\boldsymbol{y}}$ to $\widehat{\mathsf{R}}$ as the answers of the $\mathsf{OLE}_{\mathsf{q}}$ calls.
3. Sample $\tilde{\boldsymbol{r}} \xleftarrow{R} \mathbb{F}^{m+1}$ and $z^{\mathsf{S}} \xleftarrow{R} \mathbb{F}$ and send $(\tilde{\boldsymbol{r}}, z^{\mathsf{S}})$ to $\widehat{\mathsf{R}}$ as message from S.
4. If there is $\widehat{x} \in \mathbb{F}$ such that $\widehat{x} = \widehat{x}_1 = \ldots = \widehat{x}_{m+1}$, let $t^{\mathsf{R}} = \widehat{x}^{-1} \cdot (z^{\mathsf{S}} + \langle \tilde{\boldsymbol{y}}, \tilde{\boldsymbol{r}} \rangle)$ and $\hat{t}^{\mathsf{R}}$ be the value of t^{R} sent by $\widehat{\mathsf{R}}$. If there is no such $\widehat{x}$ or $\hat{t}^{\mathsf{R}} \neq t^{\mathsf{R}}$, send $\perp$ to the ideal functionality and abort.
5. Feed $(\widehat{x}, M\tilde{y})$ to the ideal random $\mathsf{RVOLE}_{\mathsf{q},\mathsf{m}}$ functionality, where M is defined by $\tilde{\boldsymbol{r}}$ as in the protocol.

Claim B.4. *For any malicious $\widehat{\mathsf{R}}$, the emulated execution induced by the above simulator is $O(1/q)$-close to the real execution between S and $\widehat{\mathsf{R}}$.*

The correctness of the simulator follows from the next two lemmas. The first lemma formalizes the intuition that a malicious receiver can only learn a single linear combination of $\tilde{\boldsymbol{a}}$ whose coefficients are $r_i \widehat{x}_i$, for its chosen OLE inputs $\widehat{x}_i$.

Lemma B.5. *Suppose a malicious* $\widehat{\mathsf{R}}$ *feeds inputs* $\hat{\boldsymbol{x}} \in \mathbb{F}^{m+1}$ *to the* OLE *oracle. Then, it is possible to simulate the* OLE *oracle outputs* $\tilde{\boldsymbol{y}}$ *from* $\langle \hat{\boldsymbol{x}} \odot \hat{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle$*, where* $\odot$ *denotes pointwise product. More formally, for every* $\hat{\boldsymbol{x}} \in \mathbb{F}^{m+1}$ *there is a perfect simulator* S *such that* $(\tilde{\boldsymbol{a}}, \tilde{\boldsymbol{y}}) \equiv (\tilde{\boldsymbol{a}}, S(\langle \hat{\boldsymbol{x}} \odot \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle))$.

The above lemma shows that the only information about $\hat{\boldsymbol{a}}$ learned by a malicious receiver is $\langle \hat{\boldsymbol{x}} \odot \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle$. The next lemma shows that unless all entries in $\hat{\boldsymbol{x}}$ are identical, the receiver cannot guess $\langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle$ based on this information.

Lemma B.6. *Suppose* $\hat{\boldsymbol{x}} \notin \mathrm{Span}\{(1, 1, \ldots, 1)\}$*. Then, for any prediction algorithm* P*,*

$$\Pr[P(\tilde{\boldsymbol{r}}, \langle \hat{\boldsymbol{x}} \odot \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle) = \langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle] \leq 2/q,$$

where $\tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}}$ *are chosen uniformly and independently from* $\mathbb{F}^{m+1}$*.*

Proof. Except with $1/q$ probability over the choice of $\tilde{\boldsymbol{a}}$, the vectors $\hat{\boldsymbol{x}} \odot \tilde{\boldsymbol{a}}$ and $\tilde{\boldsymbol{a}}$ are linearly independent. In this event, their inner products with a random vector are uniform and independent, and the prediction probability is bounded by $1/q$. □

The above lemma shows that, conditioned on the real-world $\widehat{\mathsf{R}}$ feeding inconsistent inputs to the OLE oracle, the sender aborts except with negligible probability, as in the simulated experiment.

B.2 Fully Malicious Reduction

Protocol B.2 allows a malicious sender $\widehat{\mathsf{S}}$ to learn the honest receiver's output via the following attack. $\widehat{\mathsf{S}}$ feeds $\tilde{\boldsymbol{a}} = 0$ and a random $\tilde{\boldsymbol{b}}$ to the OLE oracle. If the protocol's execution is continued honestly, with $\widehat{\mathsf{S}}$ sending the correct value of z^{S} to R, we have $z^{\mathsf{S}} + \langle \tilde{\boldsymbol{y}}, \tilde{\boldsymbol{r}} \rangle = \langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle = 0$, which would make R send $x^{-1} \cdot 0 = 0$ to $\widehat{\mathsf{S}}$. Instead of sending the correct value of z^{S} to R, a malicious sender $\widehat{\mathsf{S}}$ can send a different value, namely $\hat{z}^{\mathsf{S}} = z^{\mathsf{S}} + \Delta$ for some $\Delta \neq 0$. Then the final message sent by R would be $t^{\mathsf{R}} = x^{-1} \cdot \Delta$, which reveals x to $\widehat{\mathsf{S}}$. Note that while choosing a fixed value of Δ (say, $\Delta = 1$) can be detected by R, a random choice of Δ cannot be distinguished by R from an honest sender behavior.

Instead, we observe that if $\Delta = 0$ then the value of t^{R} satisfies $t^{\mathsf{R}} = \langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle$, which is known to the sender; however, a nonzero value $\Delta \neq 0$ makes $t^{\mathsf{R}} = x^{-1} \cdot (x \cdot \langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle + \Delta)$ unpredictable to the sender when x is chosen at random by an honest R. This holds regardless of the choice of $\tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}}$ by $\widehat{\mathsf{S}}$. Hence, we can protect against a malicious R by just requiring the sender to commit to the value of t^{R} before receiving it from R, and to open the commitment after receiver this message. If the opened value is different from the message t^{R} sent by R, R aborts. This effectively prevents $\widehat{\mathsf{S}}$ from sending a value of $\hat{z}^{\mathsf{S}}$ which is inconsistent with its choice of $\tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}}$. We describe the full protocol below by assuming an ideal commitment oracle, which can be implemented by a single OLE call in the information-theoretic setting.

Protocol B.7 (Fully malicious $\mathsf{RVOLE}_{\mathsf{q},\mathsf{m}}$ from $\mathsf{OLE}_{\mathsf{q}}$)**.**

Parties: S and R.

Parameters: Prime $q \in \mathbb{N}$ and length 1^m. Let $\mathbb{F} = \mathbb{F}_q$.

Oracles: (1) $\mathsf{OLE}_{\mathsf{q}}$; (2) An ideal commitment oracle Com.

Operation:

1. S samples $\tilde{\boldsymbol{a}}, \tilde{\boldsymbol{b}} \xleftarrow{\mathrm{R}} \mathbb{F}^{m+1}$, and R samples nonzero $x \xleftarrow{\mathrm{R}} \mathbb{F}$.
2. The parties make $m+1$ parallel $\mathsf{OLE}_{\mathsf{q}}$ calls, where the inputs to call i are $(\tilde{a}_i, \tilde{b}_i)$ and x respectively. Let $\tilde{\boldsymbol{y}} \in \mathbb{F}^{m+1}$ be the $\mathsf{OLE}_{\mathsf{q}}$ outputs.
3. S: Sample $\tilde{\boldsymbol{r}} \xleftarrow{\mathrm{R}} \mathbb{F}^{m+1}$, and send $\tilde{\boldsymbol{r}}$ and $z^{\mathsf{S}} = \langle \tilde{\boldsymbol{b}}, \tilde{\boldsymbol{r}} \rangle$ to R.
4. S: Use Com to commit to $t^{\mathsf{S}} = \langle \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{r}} \rangle$.

5. R: Send $t^{\mathsf{R}} \leftarrow x^{-1} \cdot (z^{\mathsf{S}} + \langle \tilde{\boldsymbol{y}}, \tilde{\boldsymbol{r}} \rangle)$ to S.
6. S: Verify $t^{\mathsf{R}} = t^{\mathsf{S}}$.
7. S: Open t^{S}.
8. R: Verify $t^{\mathsf{S}} = t^{\mathsf{R}}$.
9. Let $M \in \mathbb{F}^{m \times (m+1)}$ be a full-rank matrix whose rows are orthogonal to $\tilde{\boldsymbol{r}}$.
   S outputs $\boldsymbol{a} \leftarrow M\tilde{\boldsymbol{a}}$ and $\boldsymbol{b} \leftarrow M\tilde{\boldsymbol{b}}$; R outputs x and $\boldsymbol{y} \leftarrow M\tilde{\boldsymbol{y}}$.

Claim B.8. *For any malicious receiver $\widehat{\mathsf{R}}$ or malicious sender $\widehat{\mathsf{S}}$, Protocol B.7 UC-realizes RVOLE with statistical security $O(1/q)$.*

Theorem B.9. *For any prime q and $m \in \mathbb{N}$, Protocol B.7 on parameters $(q, 1^m)$ UC-realizes $\mathsf{RVOLE}_{\mathsf{q,m}}$ in the $\{\mathsf{OLE}_{\mathsf{q}}, \mathsf{Com}\}$-hybrid model, with statistical security $O(1/q)$.*

C Missing Proofs

C.1 Proving Theorem 5.12

In this section we prove Theorem 5.12, restated below.

Theorem C.1 (Restatement of Theorem 5.12)**.** *For any prime $p > 3$, Protocol 5.11 with parameter p, UC-realizes $\mathsf{Wmult}_{\mathsf{p}}$ in the OT_{p}-hybrid model.*

Proof of Theorem C.1. In the following, fix prime $p > 3$, and let $\ell \leftarrow \lceil \log p \rceil$. Correctness follows by Claim C.2, security against malicious receiver by Claim C.4 and security against malicious send by Claim C.7. □

Claim C.2. *Protocol 5.11 is perfectly correct.*

Proof. For any inputs a, x and $\tau, c, \{\lambda_i\}$ sampled by S, it holds that

$$\begin{aligned} o_{\mathsf{S}} + o_{\mathsf{R}} &= \sum_{i=0}^{\ell-1} 2^{\tau(i)} \cdot (-\delta_i + z_i) \\ &= \sum 2^{\tau(i)} \cdot (-\delta_1 + \delta_i + \lambda_i \cdot a) \\ &= a \cdot \sum 2^{\tau(i)} \cdot \lambda_i = a \cdot (x + cp) = ax \bmod p. \end{aligned}$$

□

C.1.1 Corrupt Receivers

We define the simulator Sim for a corrupt $\widehat{\mathsf{R}}$ (taking the role of R in the protocol) as follows.

Algorithm C.3 (Sim)**.**
Oracle: $\mathsf{Wmult}_{\mathsf{p}}$.
Operation:

1. Let $\{\lambda_i\}$ be the inputs $\widehat{\mathsf{R}}$ provides to the OT_{p} calls.
2. Sample $(z_0, \ldots, z_{\ell-1}) \xleftarrow{\mathrm{R}} \mathbb{Z}_n^{\ell}$, and return them to $\widehat{\mathsf{R}}$ as the answers of the OT_n calls.

3. Let $\tau \in \mathbb{S}_\ell$ be the permutation $\widehat{\mathsf{R}}$ sends to S.
4. Let $x \leftarrow \sum_{i=0}^{\ell-1} \lambda_i \cdot 2^{\tau(i)} \bmod p$ and $o_{\mathsf{R}} \leftarrow \sum_{i=0}^{\ell-1} 2^{\tau(i)} \cdot z_i \bmod p$.
5. Send (x, o_{R}) to $\mathsf{Wmult}_{\mathsf{p}}$.

Claim C.4. *For input a of S, the emulated execution induced by the above simulator is identical to the real execution between S and $\widehat{\mathsf{R}}$.*

Proof. Since both in the real and in the emulated executions, the value of $\{z_i\}$ has the same distribution (iid in $\mathbb{Z}_n$), we only need to argue that given $\widehat{\mathsf{R}}$ view and S's input, the outputs of S in both execution are the same. Fix S's input to a and $\widehat{\mathsf{R}}$'s view to $(\{\lambda_i\}, \{z_i\}, \tau)$. In the real execution, it holds that

$$o_{\mathsf{S}} = -\sum_{i=0}^{\ell-1} 2^{\tau(i)} \cdot \delta_i = \sum 2^{\tau(i)} \cdot (\lambda_i a - z_i) = a \cdot \sum 2^{\tau(i)} \cdot \lambda_i - \sum 2^{\tau(i)} \cdot z_i \bmod p$$

which is exactly S's output in the ideal execution. □

C.1.2 Corrupt Senders

We define the simulator Sim for a corrupt $\widehat{\mathsf{S}}$ (taking the role of S in the protocol) as follows.

Definition C.5 ($g^\tau(x,c)$)**.** *For $\tau \in \mathbb{S}$, the function $g^\tau \colon \mathbb{Z}_n \times \{0,1\} \mapsto \{0,1\}^\ell$ is defined as follows:*

1. *If $x + p < 2^\ell$, let $c' \leftarrow c$. Else, let $c' \leftarrow 0$.*
2. *Let $\lambda_0, \ldots, \lambda_{\ell-1} \in \{0,1\}$ be such that $x + c' \cdot p = \sum_{i=0}^{\ell-1} \lambda_i \cdot 2^{\tau(i)}$.*
3. *Return $(\lambda_0, \ldots, \lambda_{\ell-1})$.*

That is, $g^\tau(x,c)$ computes $\lambda_0, \ldots, \lambda_{\ell-1}$ in the same way (honest) S computes them in the protocol.

Algorithm C.6 (Sim)**.**
Oracle: $\mathsf{Wmult}_{\mathsf{p}}$.
Operation:

1. Let (δ_i, σ_i) be the inputs $\widehat{\mathsf{S}}$ provides to the OT_{p} calls and let $a_i \leftarrow \sigma_i - \delta_i \bmod p$.
2. If $a_i = a$ for all i, for some $a \in \mathbb{Z}_n$:
   (a) Sample $\tau \xleftarrow{\mathrm{R}} \mathbb{S}_\ell$ and let $o_{\mathsf{S}} \leftarrow -\sum_{i=0}^{\ell-1} 2^{\tau(i)} \cdot \delta_i \bmod p$.
   (b) Send (a, o_{S}) to $\mathsf{Wmult}_{\mathsf{p}}$.

   Else,

   (a) Let f be the function that on input (b, τ, c):
      i. Let $(\lambda_0, \ldots, \lambda_{\ell-1}) \leftarrow g^\tau(b,c)$.
      ii. Return $\sum_{i=0}^{\ell-1} (\delta_i + \lambda_i \cdot a_i) \cdot 2^{\tau(i)} \bmod p$.
   (b) Send f, described as a circuit, and the proof π that f is unpredictable to $\mathsf{Wmult}_{\mathsf{p}}$.[a] Let τ be its answer.

3. Send τ to $\widehat{\mathsf{S}}$.

[a] We address the generation of π later.

Claim C.7. *For input x of R, the emulated execution induced by the above simulator is identical to the real execution between $\widehat{\mathsf{S}}$ and R.*

Proof. It is easy to verify that $\widehat{\mathsf{S}}$'s views in the real and emulated executions are identically distributed (and independent of S's input). So we conclude the proof by proving that for any fixing $(\{(\delta_i, a_i)\}, \tau)$ of $\widehat{\mathsf{S}}$'s view, and fixing b of S's input, the outputs of S in the real and emulated execution are *identically* distributed.

Assume all a_i's take the same value a. Thus, in the ideal execution Sim sends $(a, o_{\mathsf{S}} \leftarrow -\sum_{i=0}^{\ell-1} 2^{\tau(i)} \cdot \delta_i \bmod p)$ to $\mathsf{Wmult}_{\mathsf{p}}$ (f is not defined). By construction, in both executions, S's output is $a \cdot b - o_{\mathsf{S}} \bmod p$. If not all a_i's are the same, then in the real execution S's output is $f(x, \tau, c)$ for $c \xleftarrow{\mathrm{R}} \{0,1\}$, which is the output of S in the ideal execution if the function f sent by the simulator is unpredictable. So we conclude the proof by proving that if not all a_i are the same, the function f sent by Sim to $\mathsf{Wmult}_{\mathsf{p}}$ is *always* unpredictable.

By definition, for any x, τ, c and x:

$$f(b, \tau, c) - x \cdot b = \sum_{i=0}^{\ell-1} \lambda_i \cdot (a_i - x) \cdot 2^{\tau(i)} + \sum_{i=0}^{\ell-1} \delta_i \cdot 2^{\tau(i)} \tag{28}$$

for $(\lambda_0, \ldots, \lambda_{\ell-1}) \leftarrow g^{\tau}(x, c)$. For $\tau \in \mathbb{S}_\ell$, consider the the random variables $(\Lambda_0^\tau, \ldots, \Lambda_{\ell-1}^\tau) \leftarrow g^\tau(B, C)$ where $(B, C) \xleftarrow{\mathrm{R}} \mathbb{Z}_n \times \{0,1\}$, and let $F^{\tau,x} := \sum_{i=0}^{\ell-1} \Lambda_i^\tau \cdot (a_i - x) \cdot 2^{\tau(i)} \bmod p$. Given this notation, we conclude the proof by showing that

$$\mathrm{E}_{\tau \xleftarrow{\mathrm{R}} \mathbb{S}_\ell} \left[\max_{x,w} \{\Pr[F^{\tau,x} = w]\} \right] \leq 1/2 + \max\{0, \mu\} \tag{29}$$

for $\mu \leftarrow \frac{8}{15 \cdot (\ell-1)} - \frac{1}{10}$. We use the following definitions:

- $\overline{\Lambda}_{i,j}^\tau := \{\Lambda_h^\tau\}_{h \notin (\ell-1) \setminus \{i,j\}}$.
- $\mathcal{A}_{i,j}^{\tau,x} := \left\{0, a_i' \cdot 2^{\tau(i)}, a_j' \cdot 2^{\tau(j)}, a_i' \cdot 2^{\tau(i)} + a_j' \cdot 2^{\tau(j)} \bmod p\right\}$, for $a_i' \leftarrow a_i - x \bmod p$.
- $\mathcal{S}_d := \{i \colon a_i = d\}$

We continue the proof according to value of $\max_a |\mathcal{S}_a|$.

Fully heterogeneous $\{a_i\}$. In this part we prove that if all a_i are different, then

$$\mathrm{H}_\infty(F^{\tau,x}) \geq -\log(2/5) \tag{30}$$

for *any* τ and x. (Which clearly concludes the proof of this case.)

Fix x and τ. Let $a_i' \leftarrow a_i - x \bmod p$, $\mathcal{A}_{i,j} \leftarrow \mathcal{A}_{i,j}^{\tau,x}$ and $\Lambda_i \leftarrow \Lambda_i^\tau$, and assume for ease of notation that τ is the identity permutation (i.e., $\tau(i) = i$). We start by assuming that $|\mathcal{A}_{i,j}| = 4$ for some (distinct) i, j. Since $|\{\lambda_i \cdot a_i' \cdot 2^i + \lambda_j \cdot a_j' \cdot 2^j\}_{\lambda_i, \lambda_j \in \{0,1\}}| = 4$, the (characteristic) distribution of $\Lambda_i \cdot a_i' \cdot 2^i + \Lambda_j \cdot a_j' \cdot 2^j$ is the same like that of (Λ_i, Λ_j). We use the following claim (proven in Appendix C.1.3):

Claim C.8. *For any $\tau \in \mathbb{S}_\ell$, distinct $i, j \in (\ell - 1)$, and $\overline{\lambda} \in \mathrm{Supp}(\overline{\Lambda}_{i,j}^\tau)$, it holds that $\mathrm{H}_\infty(\Lambda_i^\tau, \Lambda_j^\tau \mid \overline{\Lambda}_{i,j}^\tau = \overline{\lambda}) \geq -\log(2/5)$.*

It follows that for any $\overline{\lambda} \in \mathrm{Supp}(\overline{\Lambda}_{i,j})$, $\mathrm{H}_\infty(\Lambda_i \cdot a_i' \cdot 2^i + \Lambda_j \cdot a_j' \cdot 2^j) \mid \overline{\Lambda}_{i,j} = \overline{\lambda}) \geq -\log(2/5)$. Since this holds for any fixing of $\overline{\lambda}$, by Fact C.9, proven in Appendix C.1.3, it holds also without the fixing, concluding the proof of this part.

Fact C.9. *Let A, B be random variables such that for any b: $\mathrm{H}_\infty(A \mid B = b) \geq \alpha$. Then $\mathrm{H}_\infty(A) \geq \alpha$.*

Given the above, in the following we assume $|\mathcal{A}_{i,j}| < 4$ for *all* pairs (i,j). Note that if this happens, then

1. $a'_i \cdot 2^i = \pm a'_j \cdot 2^j \bmod p$, or
2. $0 \in \{a'_i, a'_j\}$.

We assume that $a'_i = 0$ for some i, the case that all a'_i are non-zero follows the same lines. For ease of notation, assume that $a'_0 = 0$. It follows that

$$\sum_{i=0}^{\ell-1} \Lambda_i \cdot a'_i \cdot 2^i = 2 \cdot a'_1 \cdot \sum_{i=1}^{\ell-1} e_i \cdot \Lambda_i \bmod p$$

for $e_i \leftarrow 2 \cdot a'_1/(a'_i \cdot 2^i) \in \{-1, 1\}$. Since $|\{0, e_1, e_2, e_1 + e_2 \bmod p\}| = 4$ (recall $p > 3$), it holds that $\beta := \mathrm{H}_\infty(\sum_{i=1}^{\ell-1} e_i \cdot \Lambda_i \bmod p) = \mathrm{H}_\infty(\Lambda_i, \Lambda_j)$ under any fixing of $\overline{\Lambda}_{1,2}$. Hence, Claim C.8 yields that $\beta \geq -\log(2/5)$.

Non fully heterogeneous $\{a_i\}$. In this part we handle the case that $|\mathcal{S}_a| \geq 2$ for some $a \in \mathbb{Z}_n$. In the following we fix such a, and start by showing that

$$\mathrm{H}_\infty(F^{\tau,x}) \geq 1 \tag{31}$$

for any τ and $x \neq a$. Indeed,

$$\sum_{i=0}^{\ell-1} \Lambda_i^\tau \cdot (a_i - x) \cdot 2^{\tau(i)} = B \cdot (a - x) + \sum_{i \notin \mathcal{S}_a} \Lambda_i^\tau \cdot (a_i - a) \cdot 2^{\tau(i)} \bmod p$$

Since $\sum_{i \notin \mathcal{S}_a} \Lambda_i^\tau \cdot (a_i - a) \cdot 2^{\tau(i)}$ takes at most $2^{\ell - |\mathcal{S}_a|} \leq p/2$ values and $B \cdot (a - x) \bmod p$ is uniform over $\mathbb{Z}_p$, for any w:

$$\Pr\left[\sum_{i=0}^{\ell-1} \Lambda_i^\tau \cdot (a_i - x) \cdot 2^{\tau(i)} = w\right] \leq 1/2,$$

which implies Equation (31).

So in the rest of the proof we focus on the unpredictability of $F^{\tau,a}$. Let $a'_i \leftarrow a_i - a$. We start by assuming that $|\mathcal{S}_a| = \ell - 1$, and assume for ease of notation that $0 \notin \mathcal{S}_a$. It follows that $\sum_{i=0}^{\ell-1} \Lambda_i^\tau \cdot a'_i \cdot 2^{\tau(i)} = \Lambda_0^\tau \cdot a'_0 \cdot 2^0$. Thus, $\mathrm{H}_\infty(F^{\tau,a}) = \mathrm{H}_\infty(\Lambda_0^\tau)$. We use the following claim (proven in Appendix C.1.3):

Claim C.10. *For every $\tau \in \mathbb{S}_\ell$ and $i \in (\ell - 1)$: Λ_i^τ is uniform over $\{0,1\}$.*

By Claim C.10 and the above observation, we conclude that (for the case $|\mathcal{S}_a| = \ell - 1$)

$$\mathrm{H}_\infty(F^{\tau,a}) \geq 1 \tag{32}$$

So for the rest of the proof we assume $|\mathcal{S}_a| \leq \ell - 2$, and assume for ease of notation that $0, 1 \notin \mathcal{S}_a$. We do the following case analysis:

$\left|\mathcal{A}_{0,1}^{\tau,a}\right| = 4$. Similarly to the fully heterogeneous case, it holds that

$$\mathrm{H}_\infty(F^{\tau,a}) \geq -\log(2/5) \tag{33}$$

$\left|\mathcal{A}_{0,1}^{\tau,a}\right| = 4$. Since both a'_0 and a'_1 are non zero, it follows that $a'_0 \cdot 2^0 = \pm a'_1 \cdot 2^1 \bmod p$. Therefore, the distribution of $\Lambda_0^\tau \cdot a'_0 \cdot 2^{\tau(0)} + \Lambda_1^\tau \cdot a'_1 \cdot 2^{\tau(1)}$ is the distribution induced by $(\Lambda_0^\tau, \Lambda_1^\tau)$ on either $\{(0,0) \cup (1,1), (0,1), (1,0)\}$, or $\{(0,0), (0,1) \cup (1,0), (1,1)\}$. We use the following claim (proven in Appendix C.1.3):

Claim C.11. *For every $\mathcal{T} \in \{\{(0,0),(1,1)\},\{(0,1),(1,0))\}\}$ and $\overline{\lambda} \in \mathrm{Supp}(\overline{\Lambda}_{i,j})$, for every distinct $i, j \in (\ell-1)$, it holds that $\Pr\left[(\Lambda_i^\tau, \Lambda_j^\tau) \in \mathcal{T} \mid \overline{\Lambda}_{i,j} = \overline{\lambda}\right] \leq 2/3$.*

Hence, for any $\overline{\lambda} \in \mathrm{Supp}(\overline{\Lambda}_{0,1})$: $\mathrm{H}_\infty(\Lambda_0 \cdot a'_0 \cdot 2^{\tau(0)} + \Lambda_1 \cdot a'_1 \cdot 2^\tau(1)) \mid \overline{\Lambda}_{0,1} = x) \geq -\log(2/3)$, and we conclude that

$$\mathrm{H}_\infty(F^{\tau,a}) \geq -\log(2/3) \tag{34}$$

Given the above, we conclude the proof of this part using the following claim.[33]

Claim C.12. $\Pr_\tau\left[|\mathcal{A}_{0,1}^{\tau,a}| < 4\right] \leq 2/(\ell-1)$.

Proof. Since $0 \notin \{a'_0, a'_i\}$, $(|\mathcal{A}_{0,1}^{\tau,a}| < 4) \implies (a'_0 \cdot 2^{\tau(0)} = \pm a'_1 \cdot 2^{\tau(1)} \bmod p)$. Fix $\tau(0)$, and let $d \leftarrow a'_0 \cdot (a'_1)^{-1} \cdot 2^{\tau(0)} \bmod p$ (since $a'_1 \neq 0 \bmod p$, $(a'_1)^{-1} \bmod p$ is defined). So we need to bound the probability that $2^{\tau(1)} = \pm d \bmod p$. Since $\left\{2^k\right\}_{k \in (\ell-1) \bmod p}$ are all distinct values, the later happens with probability (at most) $2/(\ell-1)$. □

We conclude that (for the case $|\mathcal{S}_a| < \ell - 1$),

$$\begin{aligned} \mathrm{E}_\tau\left[\max_w\{\Pr[F^{\tau,a} = w]\}\right] &\leq \Pr_\tau\left[|\mathcal{A}_{0,1}^{\tau,a}| = 4\right] \cdot 2/5 + \Pr_\tau\left[|\mathcal{A}_{0,1}^{\tau,a}| < 4\right] \cdot 2/3 \\ &\leq \frac{\ell-3}{\ell-1} \cdot \frac{2}{5} + \frac{2}{\ell-1} \cdot \frac{2}{3} = \frac{1}{2} + \mu. \end{aligned} \tag{35}$$

Putting it together. Taking into account the different cases, reflected in Equations (30) to (32) and (35), we derive that

$$\mathrm{E}_{\tau \xleftarrow{R} \mathbb{S}_\ell}\left[\max_{x,w}\{\Pr[F^{\tau,x} = w]\}\right] \leq \max\{1/2, 1/2 + \mu\},$$

and Equation (29) follows. Since the above proof stipulates that f is verifiable, Sim can use it as the proof π it sends to Wmult (see Footnote *a* in the description of Sim). □

C.1.3 Missing Proofs

Proving Claim C.10.

Proving Proof of Claim C.10. Consider the following $2^\ell \times 2^\ell$ matrix:

$$\mathcal{M} = \begin{pmatrix} 0 & 0 & 0 & \dots & 0 \\ 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 0 & 1 & \dots & 1 \\ 0 & 1 & 1 & \dots & 1 \\ 1 & 1 & 1 & \dots & 1 \end{pmatrix}$$

where we index $\mathcal{M}$'s rows and column from 0 to $2^{\ell-1}$. Note that $\mathcal{M}$ is *anti-symmetric* when folded horizontally: $\mathcal{M}_{k,i} = 1^{2^\ell} - \mathcal{M}_{2^{\ell-1}-k,i}$ for every $k \in (2^{\ell-1}-1)$. Let $(B, C) \xleftarrow{R} \mathbb{Z}_p \times \{0,1\}$, let $C' \leftarrow C$ if $B + p \geq 2^\ell$ and

[33] Claim C.12 is the only part of were the primality of p is used.

$C' \leftarrow 0$ otherwise, and let $K \leftarrow B + p \cdot C' \bmod p$. Note that K is *symmetric*: $\Pr[K = k] = \Pr\left[K = 2^\ell - 1 - k\right]$ for every $k \in (0, 2^{\ell-1})$. Hence,

$$\begin{aligned} \Pr[\mathcal{M}_{K,i} = 1] &= \sum_{k \in (2^\ell - 1)} \Pr[K = k] \cdot (\mathcal{M}_{k,i} + \mathcal{M}_{2^\ell - 1 - k, i}) \\ &= \sum \Pr[K = k] = \Pr\left[K \in (2^\ell - 1)\right] = \frac{1}{2} \end{aligned} \tag{36}$$

for every i.

Recall that $(\Lambda_0^\tau, \ldots, \Lambda_{\ell-1}^\tau) = g^\tau(B, C)$, and note that (by construction) $K = \sum_i \Lambda^\tau \cdot 2^{\tau(i)}$. Assuming that τ is the identity permutation, since $k = \sum \mathcal{M}_{k,i} \cdot 2^i$ for every k and since the binary presentation of k is unique, we deduce that $\Lambda_i^\tau = \mathcal{M}_{K,i}$ for every i. Thus, $\Pr[\Lambda_i^\tau = 1] = \Pr[\mathcal{M}_{K,i} = 1] = \frac{1}{2}$.

For arbitrary τ, permute the columns of $\mathcal{M}$ according to τ (i.e., $\mathcal{M}^i \leftarrow \mathcal{M}^{\tau(i)}$). Note that $\mathcal{M}$ remains anti-symmetric, and that $k = \sum \mathcal{M}_{k,i} \cdot 2^{\tau(i)}$ for every k. Hence, the above argument applies. □

Proving Claims C.8 and C.11.

Proof of Claims C.8 and C.11. Fix τ, assume for ease of notation that $(i, j) = (0, 1)$, and fix $\overline{\lambda} = (\lambda_0, \lambda_1, \lambda_2, \ldots, \lambda_{\ell-1}) \in \{0,1\}^\ell$. Let K be as in the the proof of Claim C.10 and recall that $(\Lambda_0^\tau, \ldots, \Lambda_{\ell-1}^\tau) = g^\tau(B, C)$. By construction, $\Pr[K = k] \in \{1/p, 2/p\}$, and thus $\Pr\left[\overline{\Lambda}^\tau := (\Lambda_0^\tau, \ldots, \Lambda_{\ell-1}^\tau) = \overline{\lambda}\right] \in \{1/p, 1/2p\}$. For $\alpha, \beta \in \{0,1\}$, let $\mu_{\alpha,\beta} = \Pr\left[\overline{\Lambda}^\tau = (\alpha, \beta, \lambda_2, \ldots, \lambda_{\ell-1})\right]$. By the above, the vector $(\mu_{0,0}, \mu_{1,0}, \mu_{0,1}, \mu_{1,1})$ is equal (up to a permutation of the entries) to one of the following vectors:

$$\begin{aligned} &(1,1,1,1) \cdot 1/p \\ &(1,1,1,1/2) \cdot 1/p \\ &(1,1,1/2,1/2) \cdot 1/p \\ &(1,1/2,1/2,1/2) \cdot 1/p \\ &(1/2,1/2,1/2,1/2) \cdot 1/p \end{aligned} \tag{37}$$

Let $\widehat{\mu}_{\alpha,\beta} = \Pr\left[(\Lambda_0^\tau, \Lambda_1^\tau) = (\alpha, \beta) \mid \overline{\Lambda}_{0,1}^\tau = (\lambda_2, \ldots, \lambda_{\ell-1})\right]$. By Equation (37), $(\widehat{\mu}_{0,0}, \widehat{\mu}_{1,0}, \widehat{\mu}_{0,1}, \widehat{\mu}_{1,1})$ is equal (again, up to a permutation of the entries) to one of the vectors below:

$$\begin{aligned} &(1/2,1/2,1/2,1/2) \\ &(2/7,2/7,2/7,1/7) \\ &(1/3,1/3,1/6,1/6) \\ &(2/5,1/5,1/5,1/5) \end{aligned}$$

In conclusion,

$$\mathrm{H}_\infty(\Lambda_0^\tau, \Lambda_1^\tau \mid \overline{\Lambda}_{0,1}^\tau = \overline{\lambda}) = \max_{\alpha,\beta}(-\log(\widehat{\mu}_{\alpha,\beta})) \geq -\log(2/5)$$

yielding Claim C.8. For any $(\gamma, \delta) \in \{0,1\}$:

$$\begin{aligned} \Pr\left[(\Lambda_0^\tau, \Lambda_1^\tau) \in \{(\gamma,\delta), (1,1) - (\gamma,\delta)\} \mid \overline{\Lambda}_{i,j} = (\lambda_2, \ldots, \lambda_{\ell-1})\right] &\leq \max_{(\alpha,\beta) \neq (\alpha',\beta')} \{\widehat{\mu}_{\alpha,\beta} + \widehat{\mu}_{\alpha',\beta'})\} \\ &\leq 2/3, \end{aligned}$$

and Claim C.11 also follows. □

Proving Fact C.9.

Proof. For any a, b: $\Pr[A = a \mid B = b] = \Pr[A = a] \cdot \frac{\Pr[B=a|A=a]}{\Pr[B=b]}$. Since, $\frac{\Pr[B=a|A=a]}{\Pr[B=b]} \leq 1$ for some b,

$$\Pr[A = a] \leq \max_b \Pr[A = a \mid B = b].$$

Assume $\mathrm{H}_\infty(A) < \alpha$, then $\Pr[A = a] > 2^{-\alpha}$ for some a. By the above, it follows that $\Pr[A = a \mid B = b] > 2^{-\alpha}$ for some b, yielding that $\mathrm{H}_\infty(A \mid B = b) < \alpha$. □